SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)
DIRECTORATE OF DISTANCE EDUCATION
DEPARTMENT OF COMPUTER APPLICATIONS

MACROVA: AN INTELLIGENT DIET, MACRONUTRIENT TRACKING AND AI FOOD RECOMMENDATION SYSTEM

A full-stack web application built with Angular 19, Spring Boot 3.4 and MongoDB

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

Submitted by

KARTHICK S

Register No.: _____

Under the guidance of

Dr. A. THIRUMURTHI RAJA M.C.A., M.E., Ph.D.
Assistant Professor, SRMIST-DDE
Department of Computer Applications

ACADEMIC YEAR 2026 – 2027
OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"MACROVA: AN INTELLIGENT DIET, MACRONUTRIENT TRACKING AND AI FOOD RECOMMENDATION SYSTEM"** is a bonafide record of the project work carried out by **Mr. KARTHICK S, Register No. _____**, a student of the Department of Computer Applications, Directorate of Distance Education, SRM Institute of Science and Technology, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications**, under my supervision and guidance during the academic year 2026–2027.

The work reported herein was examined at the review stages and is found to be original. The source code, database design and documented interface behaviour were verified against the submitted repository.

PROJECT GUIDE

Dr. A. Thirumurthi Raja
M.C.A., M.E., Ph.D.
Assistant Professor, SRMIST-DDE

HEAD OF THE DEPARTMENT

Dr. A. Thirumurthi Raja
Programme Coordinator
Department of Computer Applications (DDE), SRMIST

INTERNAL EXAMINER

_____

EXTERNAL EXAMINER

_____

Date: _____ Place: Chennai

DECLARATION

I, **KARTHICK S** (Register No. _____), hereby declare that the project report entitled **“MACROVA: AN INTELLIGENT DIET, MACRONUTRIENT TRACKING AND AI FOOD RECOMMENDATION SYSTEM”** submitted to SRM Institute of Science and Technology in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications is a record of original software engineering work carried out by me under the supervision and guidance of **Dr. A. Thirumurthi Raja**, M.C.A., M.E., Ph.D., Assistant Professor, Department of Computer Applications, Directorate of Distance Education, SRMIST.

I further declare that every architectural description, database structure, algorithm, interface rendering and test observation documented in this report was derived directly from the source repository submitted with the project, and that this report or any part thereof has not been submitted earlier to any other University or Institution for the award of any degree, diploma or fellowship.

KARTHICK S

(Signature of the Candidate)

Date: _____ Place: Chennai

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of SRM Institute of Science and Technology for providing the academic environment, laboratory access and library resources that made this project possible.

I am deeply grateful to **Dr. A. Thirumurthi Raja**, Programme Coordinator, BCA/MCA, Directorate of Distance Education, and my project guide, for defining a rigorous standard for technical documentation, for insisting that every claimed feature be traceable to source code, and for reviewing the architecture and data model of this project at every stage.

I thank the faculty members of the Department of Computer Applications for their comments during the internal reviews, particularly on the nutritional computation logic and on the security of the session model.

I acknowledge the open-source community whose libraries made this project practical within a single semester: the Spring Boot and Spring Security projects, the Angular team, the MongoDB documentation team, and the maintainers of the Chart.js and java-jwt libraries. The nutrition formulae implemented in this project follow the published equations of Mifflin and St Jeor and the energy requirement guidance of the FAO/WHO/UNU expert consultation.

Finally, I thank my family and friends for their patience during the long debugging sessions that produced the working system documented here.

KARTHICK S

ABSTRACT

Dietary adherence fails less often because people lack motivation than because they lack a numeric plan they can act on. A person who wishes to lose eight kilograms needs three numbers that ordinary fitness applications seldom compute together: a daily energy budget derived from that individual's own body metrics, a macronutrient split consistent with that budget, and a per-meal share of the split that can be compared against what was actually eaten. In practice these numbers are produced in spreadsheets, copied from generic online calculators, or estimated by mobile applications that either hide the underlying formula or present static advice that never changes as the user's weight changes.

Macrova is a web-based diet planning and macronutrient tracking platform that computes, stores and continuously re-derives these quantities for each registered user. The system is implemented as a two-tier web application: a single-page front end built with Angular 19 and TypeScript 5.7, and a stateless REST back end built with Spring Boot 3.4.4 on Java 21 and persisted in MongoDB. The back end is organised as a conventional layered Spring application (controllers, services, repositories and documents) inside the package `org.karthick.macrova`, with dedicated modules for user identity, diet planning, nutritional computation, and AI-assisted food recommendation.

The nutritional engine implements the Mifflin–St Jeor basal metabolic rate equation, applies activity multipliers from 1.2 to 1.9 to obtain total daily energy expenditure, derives a calorie target through a goal-dependent deficit or surplus, splits that target into protein (30 %), fat (30 %) and carbohydrate (40 %) allocations, and finally apportions the macronutrients across breakfast (30 %), lunch (30 %), snack (15 %) and dinner (25 %). Every plan is validated against two independent safety rules: the target weight must correspond to a healthy body-mass index, and the requested duration must fall inside a window derived from the mass to be gained or lost, bounded by seven and twenty-eight days per kilogram.

A second contribution is the daily tracking model. Each tracking day is stored as a single self-contained document that embeds the calorie and macronutrient targets, the values recorded so far, and the per-meal breakdown. Recording a meal recomputes the daily totals and the calorie deficit server-side, so the dashboard, the progress screen and the historical archive always display the same derived figures without any client-side arithmetic. The application also exposes an AI-assisted meal suggestion feature that forwards a filtered prompt to a large language model through the OpenRouter inference API, with the calorie budget for the requested meal taken from the active plan.

Security is handled with JSON Web Tokens signed using HMAC512. A 60-minute access token is held only in browser memory, while a 24-hour refresh token is stored in an HttpOnly, SameSite=Lax cookie; a custom authentication filter, a JWT authentication filter and a dedicated exception filter form the request-security pipeline, and passwords are stored as BCrypt digests. The report documents the requirements, architecture, data-flow and UML models, database design, implementation of every module with source excerpts, twelve interface renderings, and a catalogue of functional, security and performance test cases together with the observed behaviour of the implemented endpoints.

Keywords: diet planning, macronutrient tracking, Mifflin–St Jeor equation, total daily energy expenditure, REST API, Angular 19, Spring Boot 3.4, MongoDB, JSON Web Token, BCrypt, large language model, OpenRouter.

TABLE OF CONTENTS

ABSTRACT . . . . . **v**
TABLE OF CONTENTS . . . . . **vi**
LIST OF FIGURES . . . . . **viii**
LIST OF TABLES . . . . . **ix**
ABBREVIATIONS AND ACRONYMS . . . . . **x**
Chapter 1: INTRODUCTION **1**
1.1 Overview of the Project 1
1.2 Background of the Study 2
1.3 Problem Statement 3
1.4 Objectives of the Project 4
1.5 Scope of the Project 5
1.6 Project Motivation 5
1.7 Assumptions and Constraints 6
1.8 Organisation of the Report 6
Chapter 2: LITERATURE REVIEW **8**
2.1 Introduction 8
2.2 Review of Existing Systems and Related Literature 8
2.3 Comparative Study of Related Work 11
2.4 Research Gaps Identified 11
2.5 Summary of Literature Review 12
Chapter 3: SYSTEM ANALYSIS **13**
3.1 Existing System Analysis 13
3.2 Limitations of the Existing System 13
3.3 Proposed System 14
3.4 Advantages of the Proposed System 14
3.5 Software Requirements Specification 16
3.6 Feasibility Study 17
3.7 Summary of System Analysis 18
Chapter 4: SYSTEM DESIGN **19**
4.1 Design Overview and Architectural Principles 19
4.2 System Architecture 19
4.3 Data Flow Diagrams 21
4.4 Use Case Diagram and Actor Specifications 24
4.5 Object-Oriented and Behavioural UML Models 27
4.6 Entity–Relationship Model 33
4.7 Database Design and Data Dictionary 33
4.8 Input Design, Output Design and Interface Design System 36
Chapter 5: TECHNOLOGIES AND TOOLS **38**
5.1 Introduction 38
5.2 Software Requirements 38
5.3 Hardware Requirements 38
5.4 Programming Languages and Data Formats 39
5.5 Frameworks and Libraries 39
5.6 Development and Runtime Environment 40
5.7 Summary of the Technology Selection 41
Chapter 6: IMPLEMENTATION **42**
6.1 Implementation Overview 42
6.2 Module Descriptions 42
6.3 Core Algorithms and Pseudocode 46
6.4 Source Code Excerpts 48
6.5 Screen Walkthroughs 54

Chapter 7: TESTING AND RESULTS 62
7.1 Testing Objectives and Strategy 62
7.2 Test Environment 62
7.3 Verification of the Nutritional Engine 62
7.4 Unit-Level Verification of the Utility Class 64
7.5 Endpoint Verification 64
7.6 Acceptance Test Catalogue 66
7.7 Security Verification 67
7.8 Interface State Verification 68
7.9 Performance Observations 68
7.10 Discussion of Results 69
Chapter 8: CONCLUSION AND FUTURE SCOPE 70
8.1 Summary of the Work 70
8.2 Achievement of Objectives 70
8.3 Limitations of the Present Work 71
8.4 Future Scope 71
8.5 Conclusion 72
REFERENCES 73
APPENDIX A — REST API REFERENCE 74
APPENDIX B — INSTALLATION AND USER MANUAL 76
B.1 Prerequisites 76
B.2 Back-End Setup 76
B.3 Front-End Setup 76
B.4 First-Run Checklist 76
B.5 Operating Notes 77
APPENDIX C — SAMPLE DOCUMENTS 78
APPENDIX D — GLOSSARY 80

Page numbers refer to the printed folio shown in the footer of each page; preliminary pages are numbered in lower-case roman numerals and the main body starts at page 1 with Chapter 1.

LIST OF FIGURES

Figure 1.1 End-to-end user journey through the implemented Macrova application 1
Figure 3.1 Comparison of the existing manual workflow with the proposed system workflow 13
Figure 4.1 Three-tier architecture of the Macrova application with the external inference service 20
Figure 4.2 Package and component structure of the Spring Boot application tier 21
Figure 4.3 Data-flow diagram, level 0 — context of the Macrova system 22
Figure 4.4 Data-flow diagram, level 1 — functional decomposition 23
Figure 4.5 Data-flow diagram, level 2 — decomposition of daily tracking and progress 24
Figure 4.6 Unified use case diagram of the Macrova system 25
Figure 4.7 Domain class diagram of documents, value objects and enumerations 28
Figure 4.8 Sequence diagram — authentication and the access-token handshake 29
Figure 4.9 Sequence diagram — diet-plan creation and day-1 track seeding 30
Figure 4.10 Sequence diagram — AI food suggestion via OpenRouter 31
Figure 4.11 Access and refresh token lifetimes and the refresh handshake 31
Figure 4.12 Activity diagram — plan creation with client and server validation 32
Figure 4.13 State machine — lifecycle of a diet plan 32
Figure 4.14 Document entity–relationship model of the Macrova database 33
Figure 5.1 Technology stack of the delivered system, layer by layer 41
Figure 6.1 Interface rendering — login screen 55
Figure 6.2 Interface rendering — sign up screen 55
Figure 6.3 Interface rendering — dashboard screen 56
Figure 6.4 Interface rendering — progress screen 57
Figure 6.5 Interface rendering — diet plan form screen 58
Figure 6.6 Interface rendering — meals screen 58
Figure 6.7 Interface rendering — plan history screen 59
Figure 6.8 Interface rendering — plan overview screen 59
Figure 6.9 Interface rendering — tracked day screen 60
Figure 6.10 Interface rendering — no active plan screen 60
Figure 6.11 Interface rendering — page not found screen 61
Figure 6.12 Interface rendering — session states screen 61

LIST OF TABLES

Table 1.1 High-level identity and architectural profile of Macrova . . . 2
Table 1.2 Functional scope matrix of the implemented release . . . 5
Table 2.1 Comparative analysis of existing diet-planning approaches against Macrova . . . 11
Table 3.1 Functional requirements specification (FR-01 to FR-22) . . . 16
Table 3.2 Non-functional requirements specification (NFR-01 to NFR-10) . . . 17
Table 3.3 Economic feasibility summary . . . 18
Table 4.1 Layer responsibilities and key types . . . 21
Table 4.2 Process descriptions of the level-1 data-flow diagram . . . 23
Table 4.3 Actor-to-use-case responsibility matrix . . . 25
Table 4.4 Use case inventory . . . 26
Table 4.5 Use case specification — UC-02 log in . . . 26
Table 4.6 Use case specification — UC-05 create or update a diet plan . . . 26
Table 4.7 Use case specification — UC-06 log daily meal macronutrients . . . 27
Table 4.8 Use case specification — UC-10 request AI food suggestions . . . 27
Table 4.9 Collection overview of the macrova database . . . 33
Table 4.10 Data dictionary — collection users . . . 34
Table 4.11 Data dictionary — collection diet_plans . . . 34
Table 4.12 Data dictionary — collection diet_plan_tracks . . . 35
Table 4.13 Data dictionary — collection ai_food_suggests . . . 35
Table 4.14 Embedded value objects and their serialised shapes . . . 35
Table 4.15 Repository query inventory and access patterns . . . 36
Table 4.16 Input design and two-level validation of the diet-plan form . . . 36
Table 4.17 Representative request and response bodies . . . 37
Table 4.18 Interface design tokens of the client application . . . 37
Table 5.1 Software requirements of the development and runtime environment . . . 38
Table 5.2 Hardware requirements for development and for deployment . . . 38
Table 5.3 Frameworks and libraries with declared versions . . . 40
Table 5.4 Runtime configuration items and their overrides . . . 40
Table 6.1 Module decomposition of the implementation . . . 42
Table 7.1 Environment used for verification . . . 62
Table 7.2 Input profiles used for the nutritional verification . . . 62
Table 7.3 Recomputation of the nutritional chain for the three profiles . . . 63
Table 7.4 Unit-level verification of NutrientsCalculator and MacrosConstants . . . 64
Table 7.5 Endpoint verification record . . . 65
Table 7.6 Acceptance test catalogue (AT-01 to AT-28) . . . 66
Table 7.7 Security verification record . . . 67
Table 7.8 Interface state verification . . . 68
Table 7.9 Indicative response times . . . 68
Table 8.1 Traceability of objectives to delivered capability and evidence . . . 70
Table A.1 User and session endpoints . . . 74
Table A.2 Diet-plan and tracking endpoints . . . 74
Table A.3 Suggestion endpoint and error mapping . . . 75
Table D.1 Glossary of terms used throughout the report . . . 80

ABBREVIATIONS AND ACRONYMS

AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
BCrypt	Blowfish-based adaptive password hashing function
BMI	Body Mass Index (kg/m²)
BMR	Basal Metabolic Rate (kcal/day)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DTO	Data Transfer Object
ER	Entity–Relationship
HMAC	Hash-based Message Authentication Code
HTTP / HTTPS	HyperText Transfer Protocol / HTTP Secure
JSON	JavaScript Object Notation
JSP	Jakarta Server Pages (not used in this project)
JWT	JSON Web Token (RFC 7519)
kcal	Kilocalorie
LLM	Large Language Model
MVC	Model–View–Controller
NoSQL	Not Only SQL (document database family)
NFR / FR	Non-Functional Requirement / Functional Requirement
ORM	Object–Relational Mapping
OWASP	Open Worldwide Application Security Project
PAL	Physical Activity Level multiplier
REST	Representational State Transfer
RBAC	Role-Based Access Control
SDK	Software Development Kit
SPA	Single-Page Application
SRS	Software Requirements Specification
TDEE	Total Daily Energy Expenditure (kcal/day)
UML	Unified Modeling Language
URI / URL	Uniform Resource Identifier / Locator
UAT	User Acceptance Testing
XSS	Cross-Site Scripting

CHAPTER 1

INTRODUCTION

Macrova is a web-based diet planning and macronutrient tracking system. It converts a small set of body measurements — age, gender, height, current weight, activity level, a goal and a target weight — into a complete, numerically consistent nutrition plan, and it then tracks that plan day by day. The application is delivered as a single-page Angular 19 client served on port 4200 and a stateless Spring Boot 3.4.4 service listening on port 8080, with MongoDB as the document store. The back end is packaged as the Maven artifact `org.karthick:macrova:0.0.1-SNAPSHOT` and the front end as the npm project `macrova-ui`.

Functionally the system covers five user-facing areas: account registration and authentication, creation and revision of a diet plan, a daily macronutrient log with automatic recomputation of totals, a dashboard that summarises progress and renders a seven-day weight trend, and an archive of past plans together with read-only historical day entries. A sixth area, AI-assisted meal suggestion, combines the calorie budget of a selected meal with three user-chosen filters (meal type, dietary preference and cuisine) and obtains a list of candidate foods from a large language model through the OpenRouter inference API.

The distinguishing property of the implementation is that all nutritional arithmetic is performed on the server, inside a single utility class (`NutrientsCalculator`), and that the client never receives a number it did not receive from the server. Consequently the dashboard, the progress screen and the archived day view cannot disagree, and the numerical behaviour of the system can be verified by unit-testing one class.

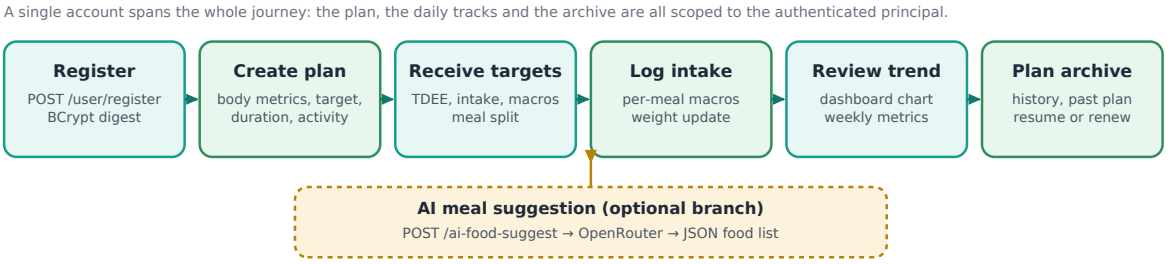


Figure 1.1: End-to-end user journey through the implemented Macrova application

Figure 1.1 summarises the journey that the remainder of this report documents. Each stage corresponds to one or more concrete endpoints, services and documents that are described in Chapters 4 and 6.

1.1 Overview of the Project

Macrova began as an attempt to answer a narrow question honestly: what exactly should a person eat tomorrow, in grams, given today's weight and a target date? Answering it requires a chain of computations whose order is fixed by physiology. The basal metabolic rate is estimated from body mass, stature, age and gender; it is multiplied by an activity factor to obtain total daily energy expenditure; a goal-dependent deficit or surplus converts that expenditure into an energy target; the target is divided into protein, fat and carbohydrate budgets; and the budgets are apportioned across meals so that each meal has an actionable target.

The project therefore consists of three cooperating concerns. The **identity concern** registers a user, stores a BCrypt digest of the password and issues a short-lived access token together with a longer-lived refresh token. The **planning concern** validates the requested plan, computes the energy and macronutrient targets, persists the plan header and immediately seeds the first daily tracking document. The **tracking concern** lets the user record a new weight and the macronutrients consumed at each meal, recomputes the daily totals and the calorie deficit, and exposes aggregate metrics for the dashboard.

Two design choices shape the implementation more than any other. First, the system stores *derived* values as well as *input* values: a tracking document contains both the targets that applied on that day and the values recorded against them, so a later change to the plan does not silently rewrite history. Second, the system keeps the plan header and the daily track in separate collections and links them with a plain identifier, which allows an arbitrary number of tracking days per plan without growing a single document beyond the 16 MB BSON limit and without requiring joins on the read path.

Table 1.1: High-level identity and architectural profile of Macrova

Attribute	Specification in the current code base
Application name	Macrova — diet, macronutrient tracking and AI food recommendation platform
Back-end artifact	org.karthick:macrova:0.0.1-SNAPSHOT (folder `backend/`, Maven wrapper included)
Front-end project	macrova-ui (folder `frontend/`, Angular CLI application builder)
Architectural style	Two-tier client–server architecture: a stateless REST service with a layered internal structure (controller → service → repository → document)
Runtime and framework	Java 21, Spring Boot 3.4.4, Spring Web MVC, Spring Security 6, Spring Data MongoDB 4
Data store	MongoDB, database `macrova`; collections `users`, `diet_plans`, `diet_plan_tracks`, `ai_food_suggests`
Client technology	Angular 19 standalone components, TypeScript 5.7, RxJS 7.8, Chart.js 4.4, CSS custom properties
Authentication	JWT signed with HMAC512; 60-minute access token in memory; 24-hour refresh token in an HttpOnly cookie; BCrypt password digests
External integration	OpenRouter chat-completions API (configurable model, temperature 0.3, JSON-only response contract)
Documentation artefacts	12 interface renderings, 19 architecture, DFD, UML and database figures, 30 data and test tables

1.2 Background of the Study

Energy balance remains the most reliable predictor of short-term weight change. When energy intake is held below total daily energy expenditure, body mass falls; when it exceeds expenditure, body mass rises. The practical difficulty is that neither side of the equation is directly observable. Expenditure must be estimated from body metrics and habitual activity, and intake must be estimated from meals. Both estimates carry error, which is why tracking tools emphasise consistency rather than precision: a plan that is approximately right and is followed is more useful than a plan that is precisely computed and abandoned.

The predictive-equation literature offers a well-established route from body metrics to energy expenditure. The Mifflin–St Jeor equation, published in 1990 [1], remains among the most accurate simple predictors of resting energy expenditure for non-obese and obese adults, and later systematic reviews recommended it as the default choice when indirect calorimetry is unavailable. Multiplying the resting value by a physical-activity-level factor converts resting expenditure into total daily expenditure; the FAO/WHO/UNU expert

consultation formalised the multiplier approach [3], and the Compendium of Physical Activities provides the underlying intensity values [5]. Macrova adopts this chain without modification, and its activity enumeration stores the multipliers recommended in that literature.

The composition of intake matters as much as its total. Protein, fat and carbohydrate differ in energy density — approximately 4, 9 and 4 kcal per gram respectively — and in their effect on satiety and lean-mass retention. A tracking system that reports only calories denies the user the ability to check whether protein intake is adequate during a deficit. Hence Macrova reports and stores a three-way macronutrient split and apportions it across meals.

The software-engineering background of the project lies in the shift from server-rendered page applications to stateless APIs consumed by single-page clients. REST, as characterised by Fielding [10], treats the server as a resource hierarchy manipulated by uniform methods; the client holds session state. That model suits a tracking application well, because almost every screen is a view of the same small set of resources (the plan, today's track, the archive) and because it allows the same API to serve a future mobile client. Token-based authentication, standardised as JSON Web Token in RFC 7519 [7], complements the model: the server need not keep a session table, and the token can carry the subject and an expiry that the client can inspect before use.

Finally, the domain background is cultural as much as technical. Indian diets are predominantly home-cooked, regionally varied and served in composite meals rather than in labelled portions. A recommendation engine that suggests Western single-item foods is therefore of limited practical value. The AI suggestion feature of Macrova deliberately exposes cuisine and dietary-preference filters so that suggestions can be constrained to Indian, Chinese, Japanese or Thai traditions, and vegetarian, non-vegetarian or low-carbohydrate preferences.

1.3 Problem Statement

The problems that motivated the project are summarised below. Each is stated in terms of the behaviour of the implemented system that addresses it.

1. **Fragmented arithmetic.** Energy and macronutrient targets are computed in spreadsheets or on general-purpose websites, then lost; the tracking application used afterwards has no knowledge of the plan, so the two disagree. Macrova stores the computed targets inside the plan and copies them into every tracking day.
2. **Opaque formulae.** Commercial applications present a calorie number without the equation, the activity factor or the macro split that produced it, which prevents any audit of the recommendation. Macrova evaluates a single documented class whose methods correspond one-to-one with published equations.
3. **Static targets despite changing weight.** A plan computed once for a weight of 76.5 kg becomes wrong as the user loses mass. Each weight update in Macrova recomputes expenditure, intake and macro targets from the new weight and stores the result in that day's track.
4. **No per-meal granularity.** Calories consumed are commonly recorded as a single daily number, which gives no guidance for the next meal. Macrova splits each macro across breakfast, lunch, snack and dinner and tracks them separately.

5. **Coarse validation of user goals.** A user may request a target weight or a duration that is physiologically unsafe. Macrova rejects targets whose body-mass index lies outside the healthy band for the chosen direction and durations outside a derived window of seven to twenty-eight days per kilogram.
6. **Volatile history.** Plans and past days are often overwritten when a new plan starts. Macrova keeps each plan as a separate document, marks the active one with a boolean flag, and exposes the archive through dedicated read-only endpoints.
7. **Weak session handling in simple implementations.** Storing long-lived tokens in browser storage exposes them to script injection. Macrova keeps the access token in memory only and the refresh token in an HttpOnly cookie, with a server-side verification filter on every protected request.

1.4 Objectives of the Project

The objectives below were fixed at the start of the project and each one is traceable to a module described in Chapter 6.

- **O1 — Deterministic nutritional engine:** implement the Mifflin–St Jeor basal metabolic rate, the activity-multiplier total daily energy expenditure, the goal-dependent energy target, the 30/30/40 macronutrient split and the 30/30/15/25 meal split in a stateless, unit-testable Java utility.
- **O2 — Safe plan validation:** reject plan requests whose target weight implies an unhealthy body-mass index or whose duration falls outside the derived minimum and maximum window, and communicate the permitted range to the client.
- **O3 — Daily tracking model:** represent every tracking day as one self-contained document, create it on demand, and recompute daily totals, the calorie deficit and the per-meal breakdown whenever a weight or a meal entry changes.
- **O4 — Aggregate progress reporting:** expose a single metrics resource that carries the plan title, elapsed days, planned duration, current weight, calorie and macro targets and the seven-day weight series used by the dashboard chart.
- **O5 — Plan lifecycle:** maintain exactly one active plan per user, retire a plan automatically when its duration has elapsed, and allow a past plan to be cloned into a new plan.
- **O6 — Token-based security:** authenticate with a JSON Web Token signed using HMAC512, issue a 60-minute access token and a 24-hour refresh token, store passwords as BCrypt digests, and translate every failure into a JSON error body.
- **O7 — AI-assisted suggestions:** generate a filtered prompt from the active plan's meal budget and a three-part filter, call a large language model through a provider-agnostic service interface, and parse the JSON reply into typed objects.
- **O8 — Responsive interface:** deliver a single-page client with guarded routes, an interceptor-based token handshake, a consistent teal design system and a charted progress view.

1.5 Scope of the Project

The functional scope of the release documented here is defined by the eleven Angular routes and the twenty-three request handlers that exist in the repository — four on the user controller, seventeen on the plan controller, one on the suggestion controller and the authentication endpoint implemented as a filter. Table 1.2 maps the scope onto the modules of the implementation.

Table 1.2: Functional scope matrix of the implemented release

Area	Routes / endpoints	Capabilities implemented
Identity	`/login`, `/signup`; `POST /user/register`, `POST /authenticate`, `POST /user/refresh`, `POST /user/logout`, `GET /user/username`	Registration with duplicate-username and duplicate-e-mail checks, BCrypt hashing, access and refresh token issue, cookie-based refresh, logout, authenticated username lookup.
Plan authoring	`/diet-plan-form`, `/no-active-plan`; `POST /diet-plan`, `PUT /diet-plan/update`, `POST /diet-plan/tdee`	Client-side field validation, server-side BMI and duration validation, plan creation with day-1 track seeding, plan update, TDEE preview.
Daily tracking	`/progress`; `GET /diet-plan/track/plan/{id}`, `POST /diet-plan/add-weight/{id}`, `POST /diet-plan/{category}/{id}`, `GET /diet-plan/meal-kcal/{id}`	On-demand creation of the current day's track, weight update with full recomputation, per-meal macro entry for breakfast, lunch, snack and dinner, meal calorie breakdown.
Progress reporting	`/dashboard`; `GET /diet-plan/metrics`, `GET /diet-plan/status`, `GET /diet-plan/active-plans`	Streak, current weight, calorie intake and macro targets, seven-day weight series, automatic retirement of expired plans, active-plan detection.
Archive	`/history`, `/history/:id`, `/history/track/:id`; `GET /diet-plan/history`, `GET /diet-plan/past-plan`, `GET /diet-plan/overview/{id}`	Plan list with status badges, projected day entries, read-only historical day log, resume-past-plan handshake.
AI suggestions	`/meals`; `POST /ai-food-suggest`	Filter selection, prompt construction with the meal calorie budget, OpenRouter invocation, JSON parsing into food objects, error state on HTTP 410.

The following capabilities are deliberately **outside** the current release: integration with wearable devices or fitness trackers, photographic food recognition, barcode scanning of packaged food, recipe generation with ingredient quantities, social or coaching features, payment and subscription handling, multi-language localisation, and any clinical recommendation beyond arithmetic. The exclusion of the last item is a design decision rather than an omission: the system produces arithmetic targets from a documented equation and does not present medical advice.

1.6 Project Motivation

Three observations motivated the work. The first was that the arithmetic of diet planning is simple but error-prone when performed by hand: converting a weight difference into a duration, converting a calorie target into grams of three macronutrients, and apportioning those grams across four meals requires eleven dependent calculations. Doing this on paper for sixty consecutive days is impractical, yet doing it once and freezing the result defeats the purpose of tracking.

The second observation concerned software structure. A diet tracker is small enough to be implemented as a single class, and many tutorials do exactly that. The project was an opportunity to demonstrate that even a small domain benefits from explicit architectural layers, a documented data model, defensive validation at both boundaries, and a security

model that does not depend on the client's good behaviour. Every endpoint in the delivered system derives the owner of the data from the authenticated principal rather than from a request parameter.

The third observation concerned the responsible use of generative models. A large language model can propose meals that respect a calorie budget, but its output is free text. The project therefore treats the model as an unreliable external service: the prompt fixes the response schema, the temperature is lowered to 0.3, the reply is parsed into typed objects, and a parse failure is surfaced to the user as an explicit error rather than a partially rendered list.

1.7 Assumptions and Constraints

1.7.1 Assumptions

- The user reports body metrics honestly; no measurement device is integrated.
- The Mifflin–St Jeor estimate is an acceptable proxy for measured resting energy expenditure in a non-clinical application.
- The activity multiplier selected by the user describes habitual activity for the whole plan period and does not change daily.
- One active plan per user is sufficient; concurrent plans are not modelled.
- The user tracks intake on the same calendar date on which the food is consumed; the tracking document is keyed by the server's local date.
- The language model returns a JSON array in the schema requested by the system prompt; any deviation is reported as an error.
- The deployment target is a single application instance with a single MongoDB instance; horizontal scaling is out of scope.

1.7.2 Constraints

- **Time:** the project was developed within one academic Semester.
- **Team size:** a single developer, which limited the breadth of the automated test suite to a Spring context-load test and a manually executed acceptance catalogue.
- **Environment:** the reference deployment uses a local MongoDB instance on port 27017 and the Angular development server on port 4200 with a proxy that rewrites `/api` to port 8080.
- **Secrets:** the JWT signing secret and the OpenRouter key are supplied as environment variables (`JWT_SECRET`, `OPENROUTER_API_KEY`, `OPENROUTER_MODEL`) and are never committed.
- **Currency of literature:** the nutritional formulae are deliberately taken from long-established publications rather than recent web content.
- **Browser support:** evergreen Chromium and Firefox releases; no legacy browser support is claimed.

1.8 Organisation of the Report

The report is organised as follows.

- **Chapter 1 — Introduction** states the problem, objectives, scope and constraints, and profiles the delivered system.
- **Chapter 2 — Literature Review** surveys the twelve sources that inform the nutritional and software design, compares five classes of existing systems and identifies the gaps

addressed here.

- **Chapter 3 — System Analysis** analyses the existing manual practice, derives the proposed system, and records functional and non-functional requirements with a feasibility study.
- **Chapter 4 — System Design** presents the architecture, data-flow diagrams, use cases, UML class, sequence, activity and state models, the document database design with data dictionaries, and the input/output design.
- **Chapter 5 — Technologies and Tools** lists software, hardware, languages, frameworks and the development environment.
- **Chapter 6 — Implementation** describes every module, the core algorithms with pseudocode and source excerpts, and walks through twelve interface renderings.
- **Chapter 7 — Testing and Results** records the test strategy, a twenty-eight case acceptance catalogue, endpoint verification and a performance and security analysis.
- **Chapter 8 — Conclusion and Future Scope** summarises the work, states its limitations candidly and lists enhancements.
- **Appendices** provide the API reference, configuration keys, sample documents, the installation and user manual, and the glossary.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Macrova sits at the intersection of two bodies of knowledge. The first is nutritional physiology: how energy expenditure is estimated from body metrics, how energy balance determines weight change, and why macronutrient composition matters. The second is software engineering: how a small information system should be layered, how its data should be modelled, how its endpoints should be secured and how its interface should communicate state. This chapter reviews twelve sources drawn from both bodies, records the specific decision each source influenced, compares the delivered system with five classes of existing software, and states the gaps that the project addresses.

The sources were selected by a simple rule: a source was included only if a concrete artefact in the repository could be traced back to it. Consequently the review is deliberately narrow. It does not attempt a survey of machine learning for dietary recommendation, of wearable sensing or of clinical nutrition therapy, because the implemented system makes no claim in those areas.

2.2 Review of Existing Systems and Related Literature

2.2.1 Literature Survey 1

TITLE: Predictive Equations for Resting Energy Expenditure in Healthy Adults

YEAR OF PUBLICATION: 1990

AUTHOR(S): M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty and Y. O. Koh

ABSTRACT AND RELEVANCE: The paper introduces the equation that is now cited as Mifflin–St Jeor: a resting energy expenditure of 10w + 6.25h − 5a + 5 kcal/day for men and 10w + 6.25h − 5a − 161 kcal/day for women, where w is mass in kilograms, h is stature in centimetres and a is age in years. It was derived from indirect calorimetry on 498 adults and reported closer agreement with measured values than the older Harris–Benedict equation. Macrova implements both branches verbatim in `NutrientsCalculator.findBMR`. **[1]**

2.2.2 Literature Survey 2

TITLE: Comparison of Predictive Equations for Resting Metabolic Rate in Healthy Non-obese and Obese Adults

YEAR OF PUBLICATION: 2005

AUTHOR(S): J. Frankenfield, L. Roth-Yousey and C. Compher

ABSTRACT AND RELEVANCE: A systematic review that compares predictive equations against measured resting metabolic rate and concludes that the Mifflin–St Jeor equation is the most reliable general-purpose predictor when calorimetry is unavailable, while noting that individual error of 10 per cent is expected. The review justified the decision to use one documented equation rather than a proprietary formula, and to display targets as guidance rather than prescription. **[2]**

2.2.3 Literature Survey 3

TITLE: Human Energy Requirements: Report of a Joint FAO/WHO/UNU Expert Consultation

YEAR OF PUBLICATION: 2004

AUTHOR(S): Food and Agriculture Organization, World Health Organization and United Nations University

ABSTRACT AND RELEVANCE: The consultation formalises the factorial method in which total energy expenditure is expressed as a multiple of resting expenditure according to a physical activity level. It classifies habitual activity into sedentary, light, moderate, vigorous and extremely vigorous bands. Macrova's `Activity` enumeration stores the five multipliers 1.2, 1.375, 1.55, 1.725 and 1.9 and its labels mirror these bands. **[3]**

2.2.4 Literature Survey 4

TITLE: Indices of Relative Weight and Obesity

YEAR OF PUBLICATION: 1972

AUTHOR(S): A. Keys, F. Fidanza, M. J. Karvonen, N. Kimura and H. L. Taylor

ABSTRACT AND RELEVANCE: The classical paper that established the body-mass index as a stature-independent index of relative weight. Macrova uses the index twice: to validate that a requested target weight is inside a healthy band, and to reject unsafe plans. The implementation rejects a weight-loss target whose index falls below 18 and a weight-gain target whose index exceeds 25. **[4]**

2.2.5 Literature Survey 5

TITLE: Compendium of Physical Activities: A Second Update of Codes and MET Values

YEAR OF PUBLICATION: 2011

AUTHOR(S): B. E. Ainsworth, W. L. Haskell, S. D. Herrmann, N. Meckes, D. R. Bassett and others

ABSTRACT AND RELEVANCE: The compendium tabulates metabolic equivalents for dozens of activities and provides the empirical basis for the activity multipliers used in factorial estimation. It supports the design decision to expose activity as five discrete levels rather than a free numeric field, which reduces both input error and unrealistic multiplier values. **[5]**

2.2.6 Literature Survey 6

TITLE: JSON Web Token (JWT), RFC 7519

YEAR OF PUBLICATION: 2015

AUTHOR(S): M. Jones, J. Bradley and N. Sakimura

ABSTRACT AND RELEVANCE: The specification defines a compact, self-describing token consisting of a header, a claim set and a signature. Macrova uses the `java-jwt` implementation to sign tokens with HMAC-SHA512 over a secret supplied through the `JWT_SECRET` environment variable, to place the username in the subject claim and to bound validity with the expiry claim. **[7]**

2.2.7 Literature Survey 7

TITLE: The OAuth 2.0 Authorization Framework, RFC 6749

YEAR OF PUBLICATION: 2012

AUTHOR(S): D. Hardt (editor)

ABSTRACT AND RELEVANCE: Although Macrova implements its own password grant rather than delegating to an authorisation server, RFC 6749 supplies the vocabulary the implementation uses: a short-lived bearer credential for resource access and a longer-lived refresh credential used solely to mint new access credentials. The lifetime ratio adopted here — one hour against twenty-four hours — follows the guidance that the refresh credential should outlive the access credential by at least an order of magnitude. **[8]**

2.2.8 Literature Survey 8

TITLE: OWASP Application Security Verification Standard, Version 4.0.3

YEAR OF PUBLICATION: 2021

AUTHOR(S): OWASP Foundation

ABSTRACT AND RELEVANCE: The standard enumerates verifiable security requirements for authentication, session management, access control and data protection. The project adopts its guidance on adaptive password hashing (BCrypt with per-password salt), on generic authentication failure messages that do not disclose whether a username exists, and on server-side authorisation decisions. Section 7.7 records the conformance position honestly, including the requirements that remain unmet in this release. **[9]**

2.2.9 Literature Survey 9

TITLE: Architectural Styles and the Design of Network-based Software Architectures

YEAR OF PUBLICATION: 2000

AUTHOR(S): R. T. Fielding

ABSTRACT AND RELEVANCE: Fielding's dissertation defines representational state transfer and the constraints that distinguish it: client–server separation, statelessness, cacheability, a uniform interface and layered components. Macrova's API follows the style by exposing resource-oriented paths under `/user`, `/diet-plan` and `/ai-food-suggest`, by keeping no server session and by returning JSON representations. **[10]**

2.2.10 Literature Survey 10

TITLE: Patterns of Enterprise Application Architecture

YEAR OF PUBLICATION: 2002

AUTHOR(S): M. Fowler

ABSTRACT AND RELEVANCE: The catalogue supplies three patterns that structure the back end: the layering of presentation, domain and data source responsibilities; the data transfer object used to decouple the wire representation from persistence objects; and the repository that encapsulates query construction. Macrova implements them as controllers, DTO records, services and Spring Data repository interfaces. **[11]**

2.2.11 Literature Survey 11

TITLE: Designing Data-Intensive Applications

YEAR OF PUBLICATION: 2017

AUTHOR(S): M. Kleppmann

ABSTRACT AND RELEVANCE: The book compares relational and document models and observes that the document model suits data that is read and written as a self-contained unit, while normalisation suits many-to-many relationships. Macrova stores each tracking day as one document with embedded macronutrient objects precisely because the dashboard reads that unit whole, and it keeps the plan header separate because the archive pages the plan list without its tracks. **[12]**

2.2.12 Literature Survey 12

TITLE: Usability Engineering and the Heuristic Evaluation of Interactive Systems

YEAR OF PUBLICATION: 1994

AUTHOR(S): J. Nielsen

ABSTRACT AND RELEVANCE: Nielsen's heuristics — visibility of system status, match between system and the real world, user control, consistency and prevention of errors — inform the interface design described in Section 4.8. The loading state on the suggestion screen, the confirmation of active versus inactive plans, the persistent navigation bar and the inline validation messages are direct applications of these heuristics. **[13]**

2.3 Comparative Study of Related Work

Table 2.1 positions the delivered system against five classes of software that a student or fitness enthusiast might use for the same purpose. The comparison is qualitative and is based on the features each class typically exposes rather than on a measured benchmark.

Table 2.1: Comparative analysis of existing diet-planning approaches against Macrova

Criterion	Paper or spreadsheet	Generic calorie web calculator	Consumer tracking app	Clinical dietitian software	Macrova (this work)
Energy target derived from body metrics	Manual	Yes, formula hidden	Yes, formula hidden	Yes, supervised	Yes, documented equation
Macronutrient split stored	Occasionally	Rarely	Yes	Yes	Yes (30/30/40)
Per-meal targets	Manual	No	Sometimes	Yes	Yes (30/30/15/25)
Server-side recomputation on weight change	No	No	Partial	By clinician	Yes, on every update
Plan validation (BMI, duration window)	No	No	No	Clinician judgement	Yes, automated
Immutable daily history	No	No	Partial	Yes	Yes, one document per day
Resume a previous plan	Manual	No	Sometimes	Yes	Yes, cloned by id
AI meal suggestion with cuisine filters	No	No	Rarely	No	Yes, via OpenRouter
Auditable open arithmetic	Yes, by hand	No	No	Partially	Yes, one utility class
Token-based stateless session	Not applicable	No	Vendor-specific	No	Yes, JWT + refresh cookie

2.4 Research Gaps Identified

1. **Traceability of computed targets.** None of the surveyed consumer tools exposes the equation, the activity table and the macro ratios that produced a target. An auditable chain from inputs to grams per meal is absent, which prevents academic verification of the recommendation.

2. **Recomputation on weight change.** Tools that store a plan usually freeze its targets. The literature on energy balance implies that targets must be recomputed as mass changes; the implemented system re-derives expenditure, intake and macros on every weight update.
3. **Per-meal granularity of all three macronutrients.** Applications tend to track calories per meal and macronutrients per day. Splitting protein, fat and carbohydrate across four meals, as Macrova does, closes this gap.
4. **Server-side validation of goal safety.** Consumer applications accept any target weight and duration. A derived duration window and a body-mass index guard are simple, deterministic checks that are rarely implemented.
5. **Controlled use of generative models.** Where meal suggestions are AI-assisted, the model output is typically inserted into the page without schema validation. Treating the model as an unreliable service with a fixed JSON contract and an explicit failure state is not common practice in student projects, and is demonstrated here.

2.5 Summary of Literature Review

The review establishes four conclusions that shaped the implementation. First, an energy model constructed from a published predictive equation and a documented activity table is defensible for a non-clinical application, provided the estimate is presented as guidance. Second, a document store is an appropriate substrate for a tracking workload that reads and writes one self-contained day at a time. Third, stateless token authentication with a short access lifetime and a longer refresh lifetime is a sound default for a JSON API. Fourth, interface consistency and explicit system status measurably reduce user error, which justifies the investment in a design system rather than ad-hoc styling. The next chapter converts these conclusions into requirements.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System Analysis

Before the application existed, the same task was performed with a spreadsheet and a printed calculator. The workflow had six steps: measure weight; choose a target weight and date; compute a basal metabolic rate and multiply by an activity factor; subtract or add a percentage to obtain a calorie target; split the target into protein, fat and carbohydrate; and write the grams into a notebook at each meal. Recording intake required a second pass in which the macronutrients of each meal were summed by hand and compared with the targets.

The workflow is arithmetically correct and requires no software. Its weaknesses are operational: the eleven dependent calculations are repeated for every plan revision; the spreadsheet is not available at the dining table; the notebook carries no derived totals; and nothing links a plan to the days it governed. Because the target is frozen at the start of the plan, a user who loses three kilograms in the first month continues to eat against a budget computed for the original weight.

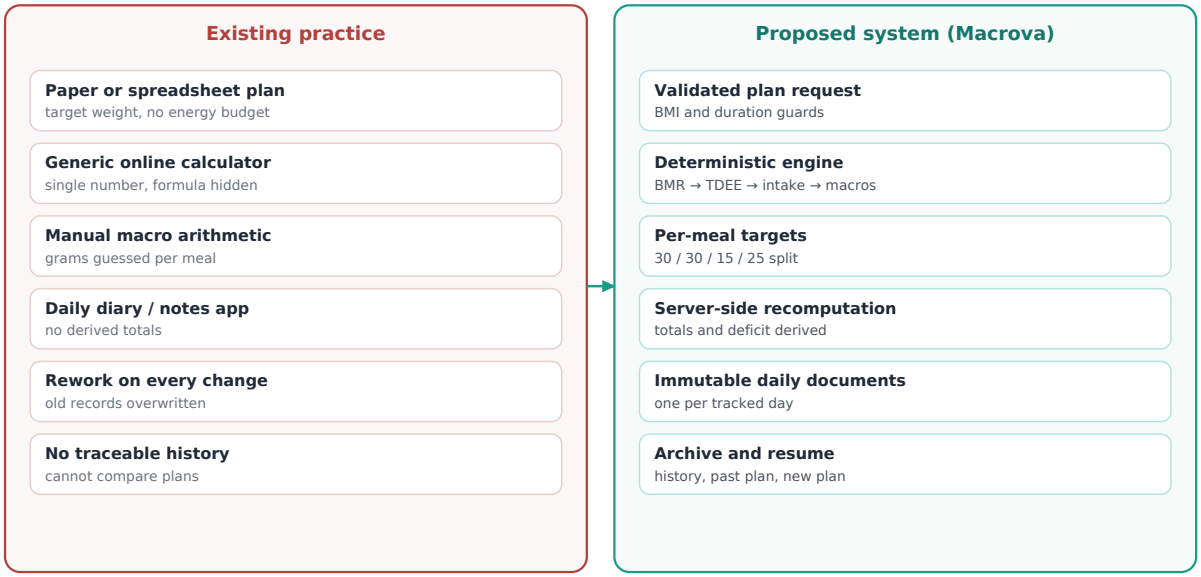


Figure 3.1: Comparison of the existing manual workflow with the proposed system workflow

3.2 Limitations of the Existing System

- Repeated manual computation invites arithmetic and unit errors, particularly when converting a calorie allocation into grams of protein, fat and carbohydrate.
- The plan and the record are physically separate artefacts, so the record cannot be validated against the plan.
- No authoritative history exists: a revised spreadsheet overwrites the previous plan, and paper pages are easily lost.
- There is no validation of the requested target weight or duration, so physiologically implausible goals are accepted.

- Targets do not adapt to the measured weight, so the deficit or surplus drifts away from the intended value.
- The per-meal split, if performed at all, is performed once and not updated.
- No access control exists; the notebook is either private or shared, with no middle ground.
- No reporting: producing a weekly weight trend requires manual plotting.
- No suggestion capability: the user must invent meals that fit the remaining budget.
- No portability: records cannot be consulted from another device.

3.3 Proposed System

The proposed system automates the arithmetic, persists every artefact and exposes the plan and its history through a guarded web interface. It is implemented as a two-tier application. The presentation tier is an Angular single-page application that renders eleven routes; the application tier is a Spring Boot service that exposes twenty-three request handlers (four user, seventeen plan, one suggestion and the authentication filter) and persists to MongoDB.

A plan is accepted only if two safety conditions hold. The body-mass index computed from the requested target weight and the stated height must be consistent with the direction of the plan: at least 18 kg/m² for weight loss and at most 25 kg/m² for weight gain. The requested duration must lie between the minimum number of days derived at the fastest safe rate (seven days per kilogram for loss, fourteen days per kilogram for gain) and the maximum derived at the slowest useful rate (twenty-eight days per kilogram). Both conditions are evaluated server-side and reported to the client as a human-readable range.

Once a plan is stored, the service immediately creates the first tracking document for the current date. That document embeds the computed calorie target, the three macronutrient targets, and the four meal allocations. When the user records a meal, the service writes the recorded values into the appropriate meal object, re-sums the three macronutrient totals across all four meals, and recomputes the calorie value actually consumed from the standard energy densities. When the user records a new weight, the service updates the plan's current weight and re-runs the entire energy chain for that day.

3.4 Advantages of the Proposed System

- Every displayed number is computed once, on the server, from a documented formula; the client performs no nutritional arithmetic.
- Targets adapt to the measured weight, so the intended deficit or surplus is preserved across the plan.
- Plans are validated against two independent safety rules before persistence.
- The daily record is immutable in structure and complete in itself, which makes the archive auditable and the dashboard a single-document read.
- The archive preserves every plan and every tracking day, and allows a past plan to be cloned into a new one.
- Session handling separates a short-lived access token from a long-lived refresh credential and keeps the refresh credential out of reach of page scripts.
- A single design system gives consistent typography, spacing and colour semantics across all screens.

- AI suggestions are constrained by the remaining calorie budget of the selected meal and by cuisine and dietary filters, rather than being unconstrained text.

3.5 Software Requirements Specification

3.5.1 Functional Requirements

Table 3.1: Functional requirements specification (FR-01 to FR-22)

ID	Requirement	Acceptance criterion
FR-01	The system shall register a user with a unique username, a unique e-mail address and a password.	Duplicate username or e-mail returns HTTP 400 with a descriptive message; the stored document contains a BCrypt digest, never the raw password.
FR-02	The system shall authenticate a user and issue an access token and a refresh token.	POST /authenticate with valid credentials returns 200 and a Set-Cookie header containing an HttpOnly Refresh-Token cookie.
FR-03	The system shall renew an access token from a valid refresh cookie.	POST /user/refresh returns {accessToken}; a missing or expired cookie returns HTTP 401.
FR-04	The system shall terminate a session on request.	POST /user/logout returns a cookie with an empty value and max-age 0.
FR-05	The system shall report the authenticated username.	GET /user/username returns the subject of the validated access token.
FR-06	The system shall validate a plan request on the client before submission.	Empty title, age outside 18–65, height outside 140–210 cm, weight outside 40–170 kg, target weight on the wrong side of the current weight, or duration outside 14–730 days raises an inline toast.
FR-07	The system shall validate a plan request on the server.	A plan whose target-weight body-mass index violates the direction rule, or whose duration is outside the derived window, returns HTTP 400 with the permitted range.
FR-08	The system shall compute basal metabolic rate, total daily energy expenditure, calorie target and macronutrient targets.	The computed values match the published equations to the nearest whole kcal and gram.
FR-09	The system shall split each macronutrient target across four meals.	Breakfast and lunch receive 30 per cent each, snack 15 per cent and dinner 25 per cent of each target.
FR-10	The system shall persist a plan with its owner and creation date and mark it active.	The stored document contains the authenticated user identifier, active = true and an ISO date.
FR-11	The system shall create the first tracking document when a plan is created.	A document exists for the current date with day count 1 and populated targets.
FR-12	The system shall return the tracking document for a given plan and date, creating it on demand.	A request on a date without a document creates and returns one; a request for another user's plan returns HTTP 404.
FR-13	The system shall record a new weight and recompute the day's targets.	The stored track contains the new weight, the recomputed expenditure, target and macronutrients.
FR-14	The system shall record per-meal macronutrient intake and recompute the daily totals.	The response contains updated per-meal and aggregate values; the consumed calories equal 4P + 9F + 4C.
FR-15	The system shall expose a metrics resource for the dashboard.	The response contains the plan title, elapsed day, duration, current weight, targets and a seven-entry day and weight series.
FR-16	The system shall retire a plan whose duration has elapsed.	GET /diet-plan/status returns true and sets active = false once the elapsed day count exceeds the duration.
FR-17	The system shall list the plans of the authenticated user with status summaries.	GET /diet-plan/history returns one entry per plan ordered by creation date, with id, title, plan type, weights, duration and active flag.
FR-18	The system shall return every tracking day of a plan for archival display.	GET /diet-plan/overview/{id} returns weight, expenditure, intake and macro values with dates in descending order.
FR-19	The system shall allow a past plan to be resumed.	The client passes the past plan identifier through navigation state; the form is pre-filled and the update endpoint replaces the active plan.
FR-20	The system shall suggest foods using a language model constrained by the meal budget and three filters.	POST /ai-food-suggest returns a JSON array of five objects; a parse failure returns HTTP 410 and the client displays an error state.
FR-21	The system shall protect every resource except registration and refresh.	A request carrying no bearer token is rejected by the authorisation rule and returns HTTP 403 from the default entry point; an expired or unverifiable token returns HTTP 401 with the plain-text reason 'Access token expired' or 'JWT verification failed'.
FR-22	The system shall return a uniform JSON error body.	Errors are returned as {"error": ["message"]} with the appropriate status code.

3.5.2 Non-Functional Requirements

Table 3.2: Non-functional requirements specification (NFR-01 to NFR-10)

ID	Category	Requirement and target
NFR-01	Performance	Every REST handler completes in under 250 ms against a local MongoDB instance, excluding the external language-model call.
NFR-02	Scalability	The service is stateless; additional instances may be placed behind a load balancer without session affinity.
NFR-03	Security	Passwords are stored only as BCrypt digests; tokens are signed with HMAC512; the refresh token is HttpOnly and SameSite=Lax.
NFR-04	Reliability	A missing tracking document is created on demand rather than raising an error, so a day boundary cannot break the dashboard.
NFR-05	Usability	All screens use one design system; every form error is explained in words rather than codes.
NFR-06	Portability	The back end runs on any JDK 21 platform; the client runs on evergreen browsers.
NFR-07	Maintainability	Layering is enforced by package structure; nutritional arithmetic is isolated in one class.
NFR-08	Auditability	Each plan and each tracking day records the values that were in force on that date.
NFR-09	Observability	Exceptions are mapped to explicit status codes and messages; unhandled runtime exceptions return a generic body.
NFR-10	Data integrity	Queries are scoped by the authenticated principal; the client cannot supply an owner identifier.

3.5.3 External Interface Requirements

- **User interfaces:** HTML5 rendered by Angular 19; eleven routes; responsive down to 640 px where metric cards reflow to two columns.
- **Software interfaces:** JSON over HTTP/1.1; MongoDB wire protocol on port 27017; the OpenRouter chat-completions REST API over HTTPS.
- **Communication interfaces:** bearer-token authorisation header, HttpOnly refresh cookie, and a development proxy that rewrites the `/api` prefix to the service root.

3.6 Feasibility Study

3.6.1 Technical Feasibility

Every component of the system is available in a mature, freely licensed form. Java 21 and Spring Boot 3.4.4 provide the web, security and persistence infrastructure; MongoDB provides a schema-flexible document store that suits a tracking workload whose document shape is stable but whose embedded objects vary in count; Angular 19 provides a component model with guarded routing and typed HTTP clients. The nutritional equations require no numerical library beyond the Java standard library. The only external dependency with a variable cost is the language-model call, which is abstracted behind the `LLMService` interface so that the provider can be replaced without touching the food-suggestion module.

The developer's existing competence in Java, TypeScript and REST design was sufficient; the learning effort concentrated on the newer Angular control-flow syntax, on Spring Security's filter chain and on the Jackson mapping of nested generic types returned by the model.

3.6.2 Economic Feasibility

The direct cost of the delivered system is zero: all frameworks, the database and the development tools are open source. The only recurring cost is the inference charge of the language-model provider, which is incurred per suggestion request. Table 3.3 summarises the cost position for a single-developer academic deployment and for a hypothetical hundred-user institute deployment.

Table 3.3: Economic feasibility summary

Cost head	Academic deployment (current)	Institute deployment (100 users)
Runtime licences	Nil (open-source stack)	Nil (open-source stack)
Application hosting	Student laptop, port 8080	One small virtual instance
Database hosting	Local MongoDB instance	Managed MongoDB tier
Language-model inference	Pay per suggestion request	Budget cap per month
Development effort	One semester, one developer	Not applicable
Subscription fees for users	Nil	Nil

Because the application replaces a manual spreadsheet rather than a licensed product, the relevant comparison is the value of eliminated arithmetic and of the preserved plan history, not a licence saving. The language-model feature is optional: the remainder of the system functions when no API key is configured, and the suggestion screen is the only component that degrades.

3.6.3 Operational Feasibility

The interface follows conventions that users of consumer applications already know: a persistent top navigation bar with four destinations, a single primary action per screen, inline validation with plain-language messages, and a consistent teal palette in which the primary colour always denotes the forward action. No training is required beyond the user manual in Appendix B. The administrator's operational burden is limited to starting two processes and providing three environment variables.

Social and organisational feasibility were assessed during internal reviews: the system does not collect health data beyond what the user volunteers, does not provide medical advice, and stores no payment information. Its outputs are presented as arithmetic guidance derived from published equations, which is consistent with the academic framing of the project.

3.7 Summary of System Analysis

The analysis converts a manual, error-prone workflow into twenty-two functional requirements and ten non-functional requirements, all of which are traceable to concrete handlers or components. The feasibility study finds the solution technically achievable with a free tool-chain, economically neutral apart from optional inference charges, and operationally simple. Chapter 4 now designs the system that satisfies these requirements.

CHAPTER 4

SYSTEM DESIGN

4.1 Design Overview and Architectural Principles

The design follows six principles, each of which is visible in the package structure. **Separation of concerns** places web handling, business rules and persistence in distinct packages so that a controller never constructs a query and a repository never performs arithmetic, which is the classical layered decomposition described in [14]. **Single source of truth for arithmetic** keeps every equation in `NutrientsCalculator`, a final class of static methods with no dependencies, so that the numerical behaviour of the system can be reasoned about and tested in isolation. **Server-side authority** derives the owner of every resource from the authenticated principal rather than from the request, and performs every validation that protects the user on the server as well as in the browser.

Denormalisation by read unit stores each tracking day as one self-contained document that embeds its targets, its recorded values and its per-meal breakdown, because the dashboard, the progress screen and the archive all read that unit whole, exactly as the document-model argument in [12] predicts. **Explicit contracts** restricts the wire format to DTOs and to the document classes that the client already understands, and never leaks internal fields such as the password digest. **Fail visibly** maps each exception family to one HTTP status and one JSON body so that the client can react deterministically instead of guessing.

4.2 System Architecture

Macrova is a two-tier client–server system with an external inference dependency. Figure 4.1 shows the three logical layers of the delivered implementation and the external service. The presentation tier is the Angular application; the application tier is the Spring Boot service; the data tier is MongoDB; and the language model is reached only through the `LLMService` abstraction inside the application tier, which prevents any controller from depending on a vendor SDK.

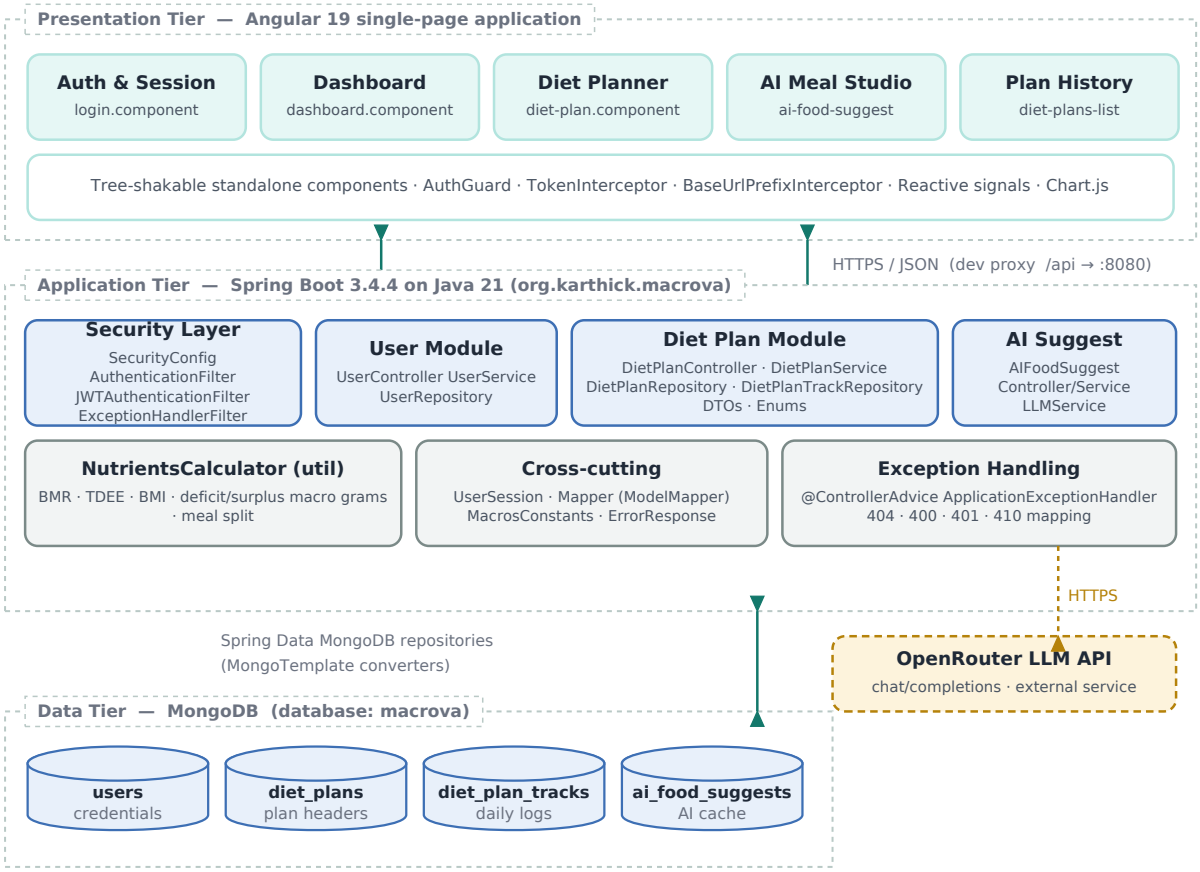


Figure 4.1: Three-tier architecture of the Macrova application with the external inference service

The package view in Figure 4.2 shows how the application tier is decomposed. The root package `org.karthick.macrova` contains nine sub-packages. The dependency direction is downward and inward: controllers depend on services, services depend on repositories and on the utility class, and no repository depends on a controller. The `security` package is consumed by the service layer through `UserSession`, which resolves the current principal from the security context.

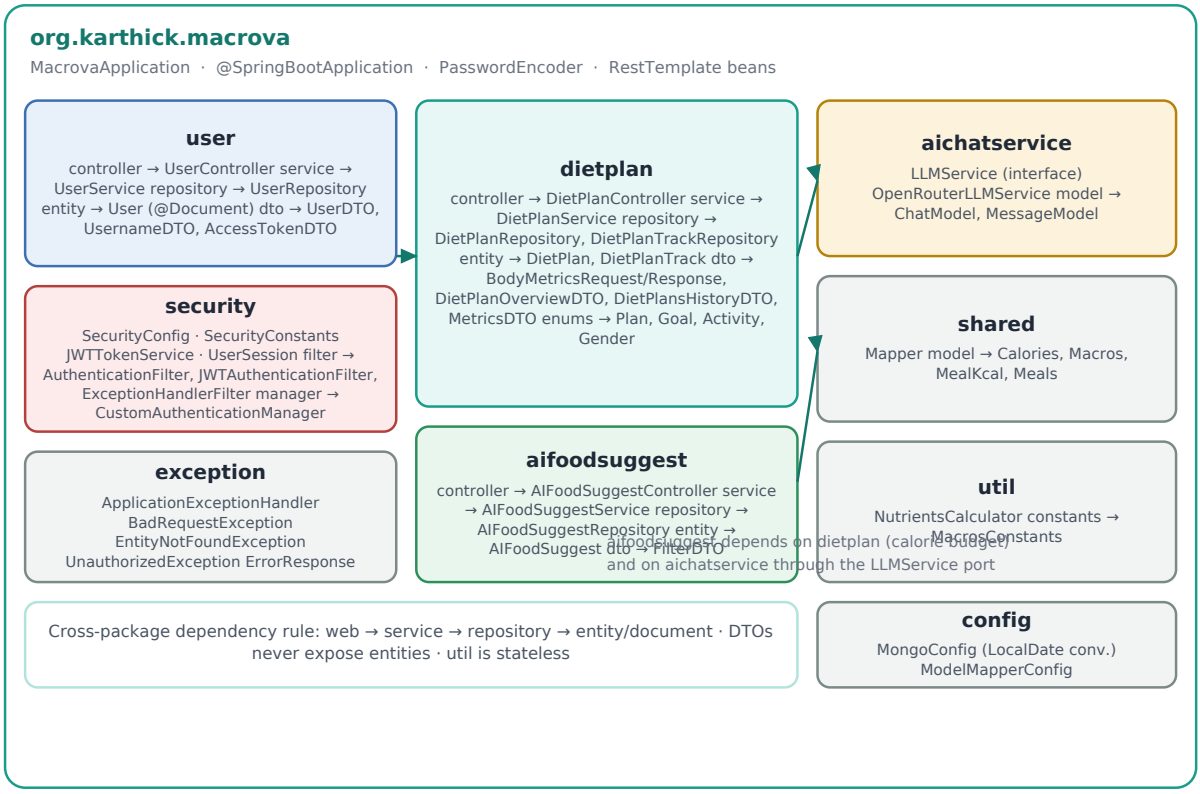


Figure 4.2: Package and component structure of the Spring Boot application tier

Table 4.1: Layer responsibilities and key types

Layer	Responsibilities	Key types
Presentation (Angular)	Routing, guarded navigation, template rendering, client-side validation, HTTP interception, chart rendering	AppComponent, HeaderComponent, LoginComponent, DashboardComponent, DietPlanComponent, DietPlanFormComponent, AIFoodSuggestComponent, DietPlansListComponent, DietPlanOverviewComponent, DietPlanTrackOverviewComponent, NoActiveDietPlanComponent, PageNotFoundComponent, ToastComponent, WeightTrackComponent
Web (Spring MVC)	Request binding, response serialisation, status codes, exception translation	UserController, DietPlanController, AIFoodSuggestController, ApplicationExceptionHandler
Security	Authentication, token issue and verification, principal resolution, CORS policy	SecurityConfig, AuthenticationFilter, JWTAuthenticationFilter, ExceptionHandlerFilter, CustomAuthenticationManager, JWTTokenService, UserSession
Service	Plan validation, energy computation, track lifecycle, metrics aggregation, prompt construction	UserService, DietPlanService, AIFoodSuggestService, OpenRouterLLMService
Domain	Document entities, embedded value objects, enumerations	User, DietPlan, DietPlanTrack, AIFoodSuggest, Calories, Macros, MealKcal, Plan, Goal, Activity, Gender
Persistence	Query derivation, projections, aggregation of weekly series	UserRepository, DietPlanRepository, DietPlanTrackRepository, AIFoodSuggestRepository
Utility and configuration	Stateless arithmetic, constants, date conversion, bean wiring	NutrientsCalculator, MacrosConstants, MongoConfig, ModelMapperConfig, Mapper

4.3 Data Flow Diagrams

Three levels of data-flow diagram describe the movement of information. The context diagram fixes the system boundary; the level-1 diagram decomposes the system into five processes and four data stores; the level-2 diagram decomposes the daily-tracking process, which carries the most intricate logic.

4.3.1 Level 0 — Context Diagram

At the context level, Macrova exchanges two flows with the user and two flows with the inference service. The user supplies credentials, plan parameters, meal entries and filter selections; the system returns an authenticated session, computed targets, dashboard metrics, history and food suggestions. The inference service receives a prompt containing the meal calorie budget and returns a JSON array.



Inputs: registration, login, body metrics, target weight, duration, daily weight and meal macro entries, filter selections.

Outputs: authenticated session, computed energy targets, daily macro dashboard, weekly weight trend, plan history, meal recommendations.

Level 0 (context) data-flow diagram of the Macrova system.

Figure 4.3: Data-flow diagram, level 0 — context of the Macrova system

4.3.2 Level 1 — Functional Decomposition

The level-1 diagram introduces the four collections as data stores and the five processes that constitute the system. Process 3.0 is shared: it is invoked by plan configuration when a plan is created, and by daily tracking whenever a weight changes.

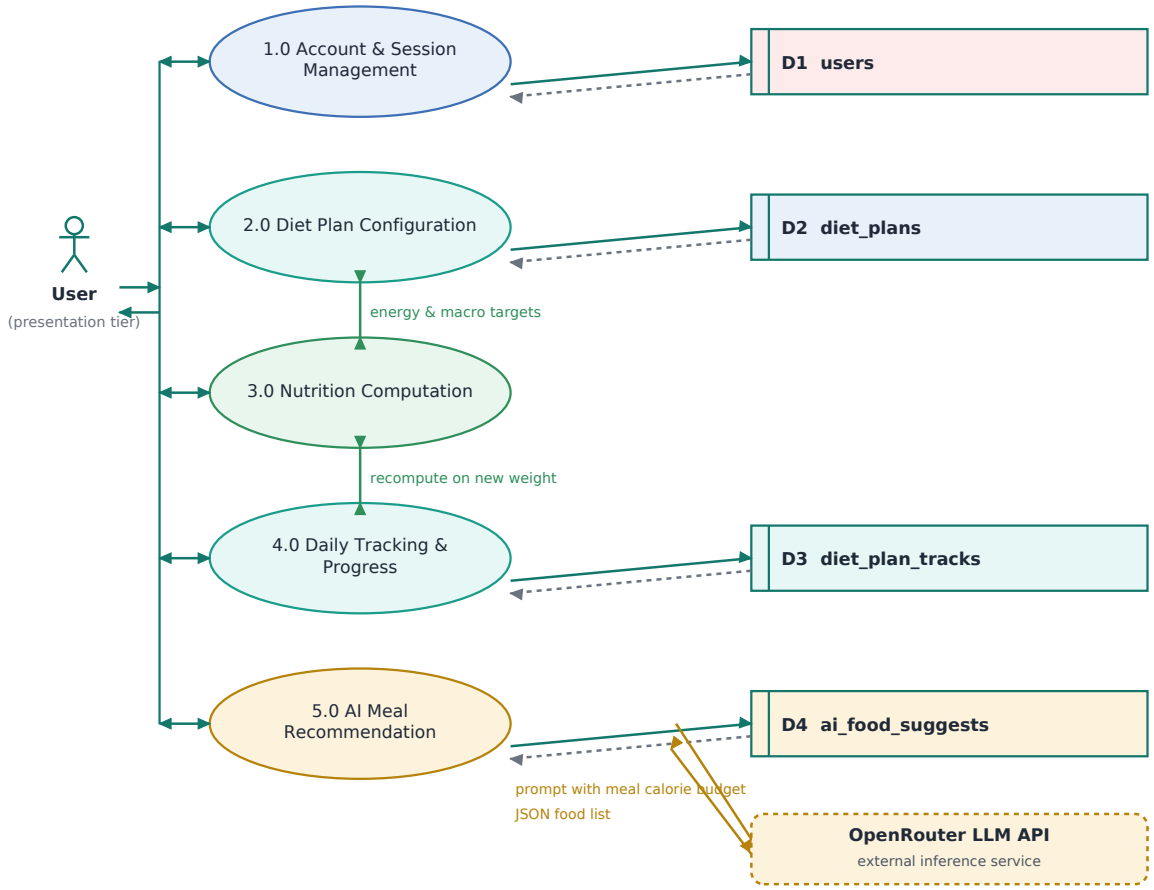


Figure 4.4: Data-flow diagram, level 1 — functional decomposition

Table 4.2: Process descriptions of the level-1 data-flow diagram

Process	Description	Realised by
1.0 Account and session management	Registers a user, verifies credentials, issues and refreshes tokens, terminates a session and resolves the authenticated principal.	UserService, CustomAuthenticationManager, JWTTokenService, AuthenticationFilter, JWTAuthenticationFilter, UserController
2.0 Diet plan configuration	Accepts plan parameters, validates them, delegates the arithmetic to process 3.0, persists the plan header and seeds the first tracking day.	DietPlanController.createDietPlan, DietPlanService.validateDietPlan, DietPlanService.createDietPlan
3.0 Nutrition computation	Computes basal metabolic rate, total daily energy expenditure, calorie target, macronutrient grams and the four meal allocations.	NutrientsCalculator (findBMR, findTDEE, findDeficit, findSurplus, findProtein, findFat, findCarbs, macrosPercentage)
4.0 Daily tracking and progress	Resolves the day's tracking document, records weight and meal entries, recomputes totals and deficit, aggregates the seven-day weight series.	DietPlanService.addWeight, updateMacros, getMetricsByDateRange, DietPlanTrackRepository
5.0 AI meal recommendation	Builds a prompt from the selected meal's calorie budget and the three filters, calls the language model and parses the response.	AIFoodSuggestService, OpenRouterLLMService

4.3.3 Level 2 — Daily Tracking Decomposition

Process 4.0 decomposes into five sub-processes. Sub-process 4.1 resolves the document for the current date and creates it if the user has not yet opened the application that day; 4.2 applies a weight update and re-enters process 3.0; 4.3 writes the recorded grams into the selected meal object; 4.4 re-sums the three macronutrients and recomputes consumed calories; and 4.5 assembles the metrics resource from the current document and the seven-day weight series.

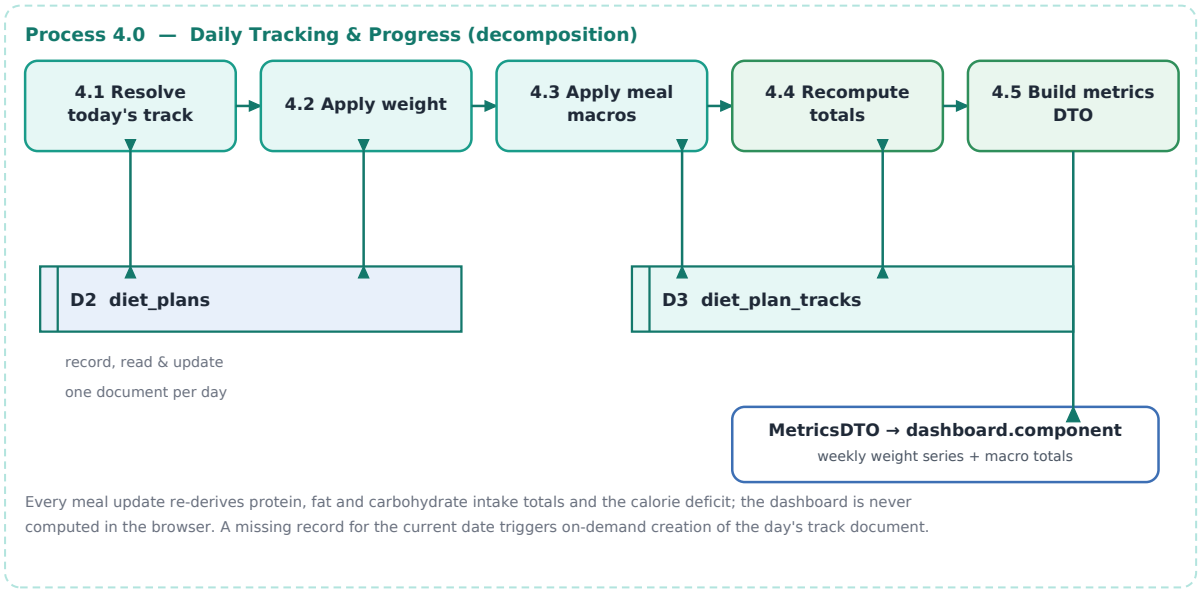


Figure 4.5: Data-flow diagram, level 2 — decomposition of daily tracking and progress

4.4 Use Case Diagram and Actor Specifications

The system has two human actors and one secondary (system) actor. The visitor may register and authenticate; the registered user inherits every other use case; the inference service participates only in the suggestion use case.

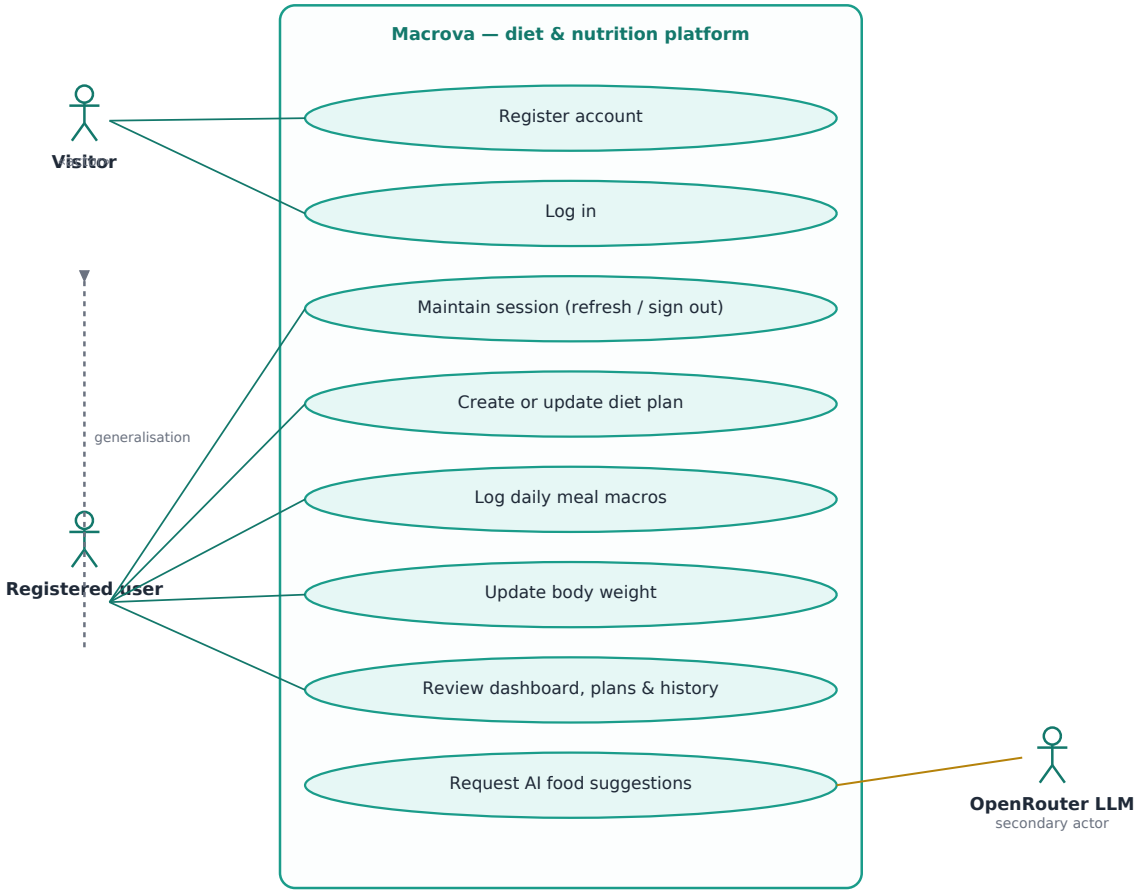


Figure 4.6: Unified use case diagram of the Macrova system

Table 4.3: Actor-to-use-case responsibility matrix

Actor	Type	Goal	Use cases
Visitor	Primary, unauthenticated	Create an account or start a session	Register account, Log in
Registered user	Primary, authenticated	Plan, track and review nutrition	Maintain session, Create or update plan, Log daily macros, Update weight, Review dashboard and history, Request AI suggestions
OpenRouter LLM	Secondary, external	Return structured food suggestions	Request AI food suggestions

Table 4.4: Use case inventory

ID	Use case	Primary actor	Trigger
UC-01	Register account	Visitor	Submission of the sign-up form
UC-02	Log in and establish a session	Visitor	Submission of the log-in form
UC-03	Refresh the access token	Registered user	Expiry detected by the client interceptor
UC-04	Log out	Registered user	Selection of Sign Out in the profile menu
UC-05	Create or update a diet plan	Registered user	Submission of the plan form
UC-06	Log daily meal macronutrients	Registered user	Selection of Update in a meal card
UC-07	Update body weight	Registered user	Selection of Update Weight
UC-08	Review dashboard metrics	Registered user	Navigation to /dashboard
UC-09	Browse plan history and archived days	Registered user	Navigation to /history
UC-10	Request AI food suggestions	Registered user	Selection of Get Suggestions
UC-11	Resume a past plan	Registered user	Selection of Continue this plan

4.4.1 Use Case Specification — UC-02: Log In and Establish a Session

Table 4.5: Use case specification — UC-02 log in

Item	Specification
Actors	Visitor (primary); AuthenticationFilter, CustomAuthenticationManager, UserService, JWTTokenService (supporting)
Preconditions	The visitor has a registered account and the service is reachable through the development proxy.
Trigger	The visitor submits username and password on /login.
Main flow	1. AuthService.login posts the credentials with credentials enabled. 2. AuthenticationFilter parses the JSON body into a User and builds a UsernamePasswordAuthenticationToken. 3. CustomAuthenticationManager loads the user and compares the raw password with the stored BCrypt digest. 4. On success the filter generates a refresh token with a 24-hour expiry and writes it to an HttpOnly, SameSite=Lax cookie. 5. The client fetches the username, which triggers the interceptor to call /user/refresh and obtain a 60-minute access token, then retries the request with the bearer header. 6. The router navigates to /dashboard.
Alternate flow	If the username is unknown, CustomAuthenticationManager raises BadCredentialsException so that the response does not reveal whether the account exists; the filter writes HTTP 401 and the client shows 'Invalid username or password!'.
Postconditions	A refresh cookie is stored by the browser; an access token is held in memory; the header displays the user's initial.

4.4.2 Use Case Specification — UC-05: Create or Update a Diet Plan

Table 4.6: Use case specification — UC-05 create or update a diet plan

Item	Specification
Actors	Registered user (primary); DietPlanFormComponent, DietPlanService, DietPlanController, NutrientsCalculator (supporting)
Preconditions	No active plan exists (for creation) or a past plan identifier is supplied through navigation state (for resumption).
Main flow	1. The component validates the fields and the direction of the target weight. 2. It posts the plan to /diet-plan. 3. The controller delegates to the service, which validates the body-mass index of the target weight and the duration window. 4. The service stamps the owner identifier, sets today's weight, marks the plan active and saves it. 5. It creates the day-1 tracking document, computing expenditure, target, macronutrient grams and the four meal allocations. 6. The client navigates to /dashboard.
Alternate flow	A target weight whose body-mass index is below 18 for weight loss (or above 25 for weight gain) raises BadRequestException; a duration outside the derived window raises BadRequestException carrying the permitted range. Both are rendered as a toast.
Postconditions	One plan document with active = true and one tracking document for the current date exist for the authenticated user.

4.4.3 Use Case Specification — UC-06: Log Daily Meal Macronutrients

Table 4.7: Use case specification — UC-06 log daily meal macronutrients

Item	Specification
Actors	Registered user (primary); DietPlanComponent, DietPlanService, DietPlanController (supporting)
Preconditions	An active plan exists and the day's tracking document is loaded.
Main flow	1. The user edits the protein, carbohydrate and fat fields of a meal card. 2. The component posts the three figures to /diet-plan/{category}/{dietPlanId}. 3. The controller validates the category against the four permitted constants. 4. The service loads today's document, writes the recorded values into the selected meal, re-sums the three macronutrients across the four meals and recomputes consumed calories as 4P + 9F + 4C. 5. The updated document is returned and re-rendered.
Alternate flow	An unknown category returns HTTP 400 'Invalid macro category'; a missing document returns HTTP 404.
Postconditions	The document records per-meal and aggregate values; the calorie bar is coloured by the relationship between consumed, target and expenditure.

4.4.4 Use Case Specification — UC-10: Request AI Food Suggestions

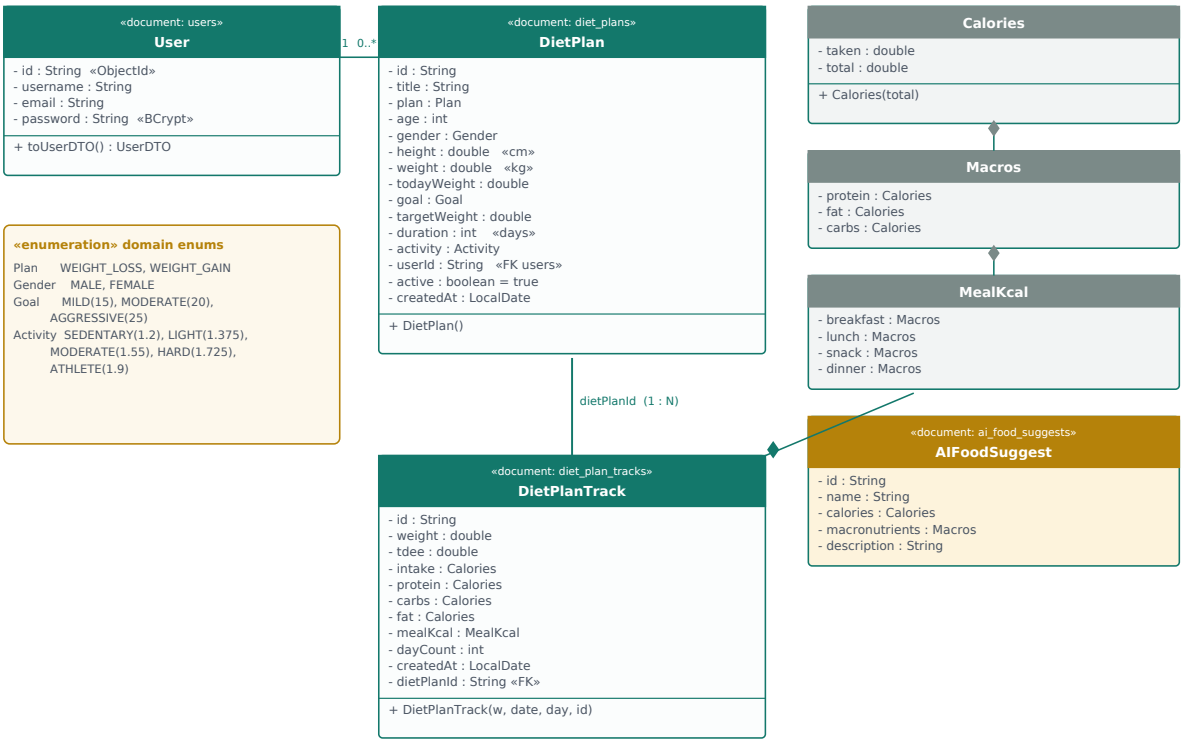
Table 4.8: Use case specification — UC-10 request AI food suggestions

Item	Specification
Actors	Registered user (primary); AIFoodSuggestComponent, AIFoodSuggestService, DietPlanService, OpenRouterLLMService, external model (secondary)
Preconditions	An active plan and a tracking document exist for the current date.
Main flow	1. The user selects a meal, a dietary preference and a cuisine and submits the form. 2. The service obtains the calorie budget of the selected meal from the active plan's daily document. 3. It composes the prompt 'Suggest me 5 healthy foods for under calories'. 4. The LLM service posts a two-message conversation with a JSON-only system instruction, temperature 0.3 and streaming disabled. 5. The content of the first choice is parsed into a list of food objects and returned.
Alternate flow	If no active plan exists the calorie budget defaults to 500 kcal. If parsing fails, JsonProcessingException is translated to HTTP 410 and the component displays an inline error message.
Postconditions	Five food cards with calories and macronutrients are rendered, or an explicit error state is shown.

4.5 Object-Oriented and Behavioural UML Models

4.5.1 Class Model

Figure 4.7 presents the persistent and embedded classes. Four document classes map one-to-one onto collections; three value objects are embedded rather than referenced. The value objects are deliberately simple: `Calories` carries a total and a recorded value, `Macros` groups three calorie objects, and `MealKcal` groups four macro objects. The composition chain *track → meal → macros → calories* is what allows a single document to answer every question the tracking screens ask.

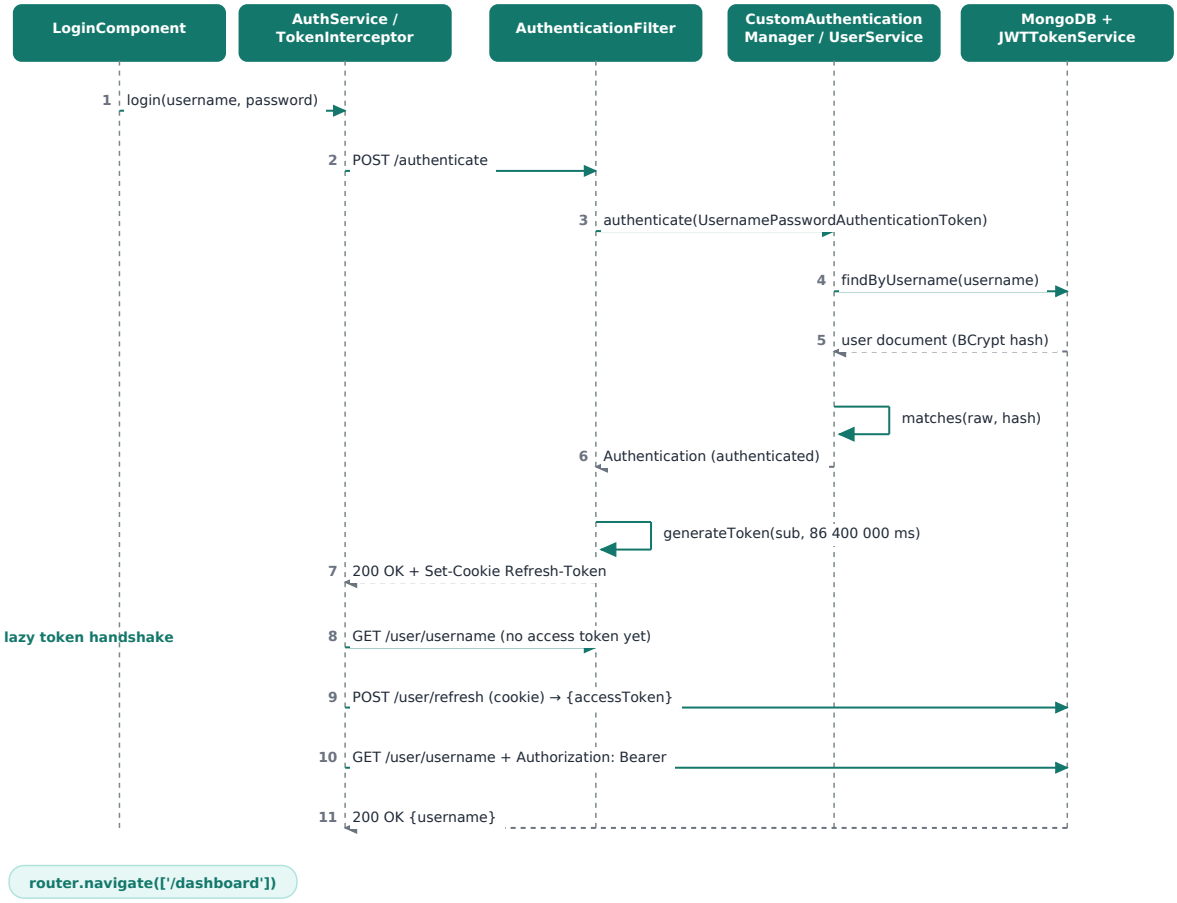


Composition: a DietPlanTrack owns one MealKcal, which aggregates four Macros; each Macros embeds three Calories value objects (taken/total). Association: every track document stores the identifier of the owning plan. AIFoodSuggest reuses Macros for the macronutrient figures returned by the language model.

Figure 4.7: Domain class diagram of documents, value objects and enumerations

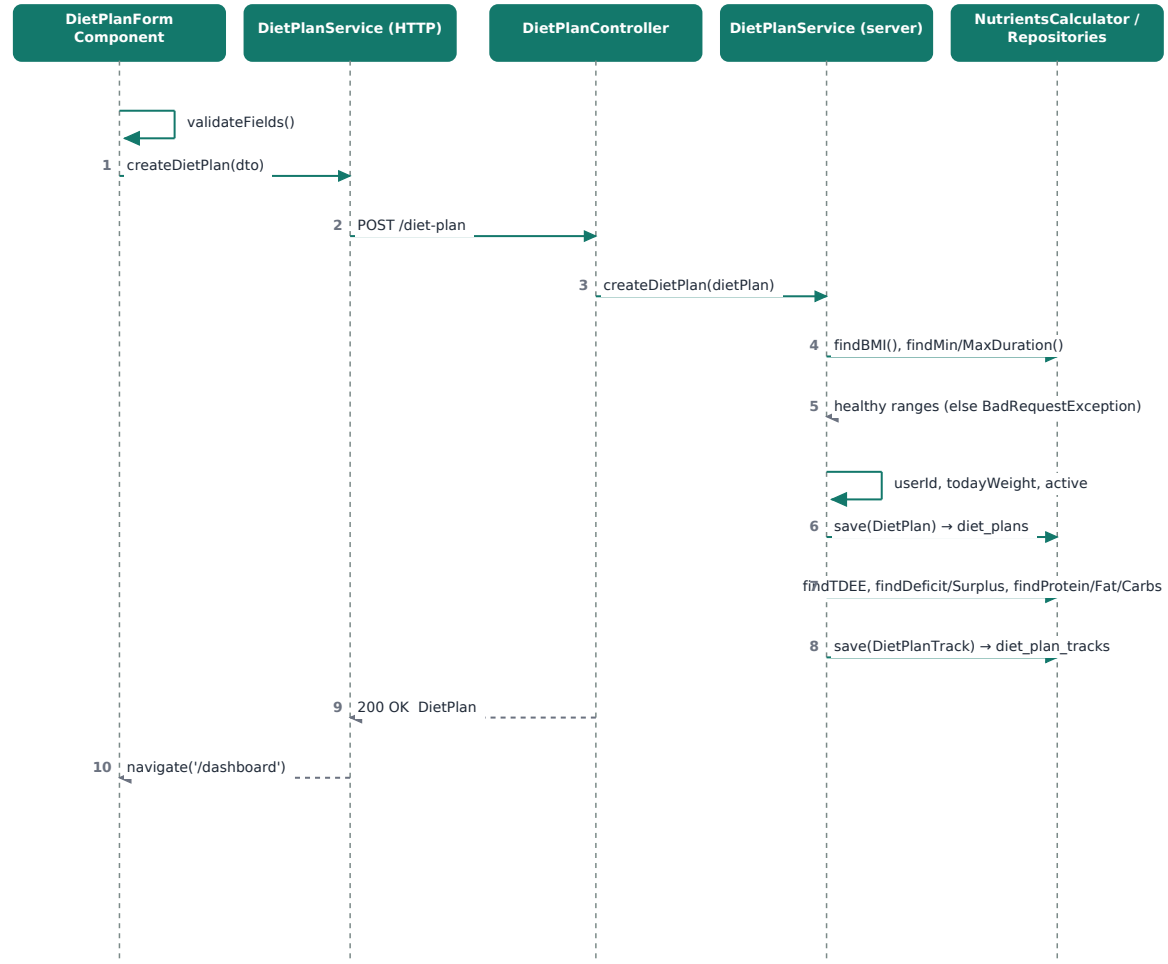
4.5.2 Sequence Models

Three sequence diagrams cover the flows with the most interaction. Figure 4.8 shows authentication, including the lazy acquisition of the access token: the authentication filter writes only the refresh cookie, and the client obtains an access token on the first protected request through the interceptor-driven refresh handshake. Figure 4.9 shows plan creation and the immediate seeding of the first tracking day. Figure 4.10 shows the suggestion flow, which is the only flow that leaves the process boundary.



Both tokens are signed with HMAC512. The refresh token exists only inside an HttpOnly cookie; the access token exists only in Angular memory.

Figure 4.8: Sequence diagram — authentication and the access-token handshake



The day-1 track document is created inside the same request, so the dashboard is immediately populated.

Figure 4.9: Sequence diagram — diet-plan creation and day-1 track seeding

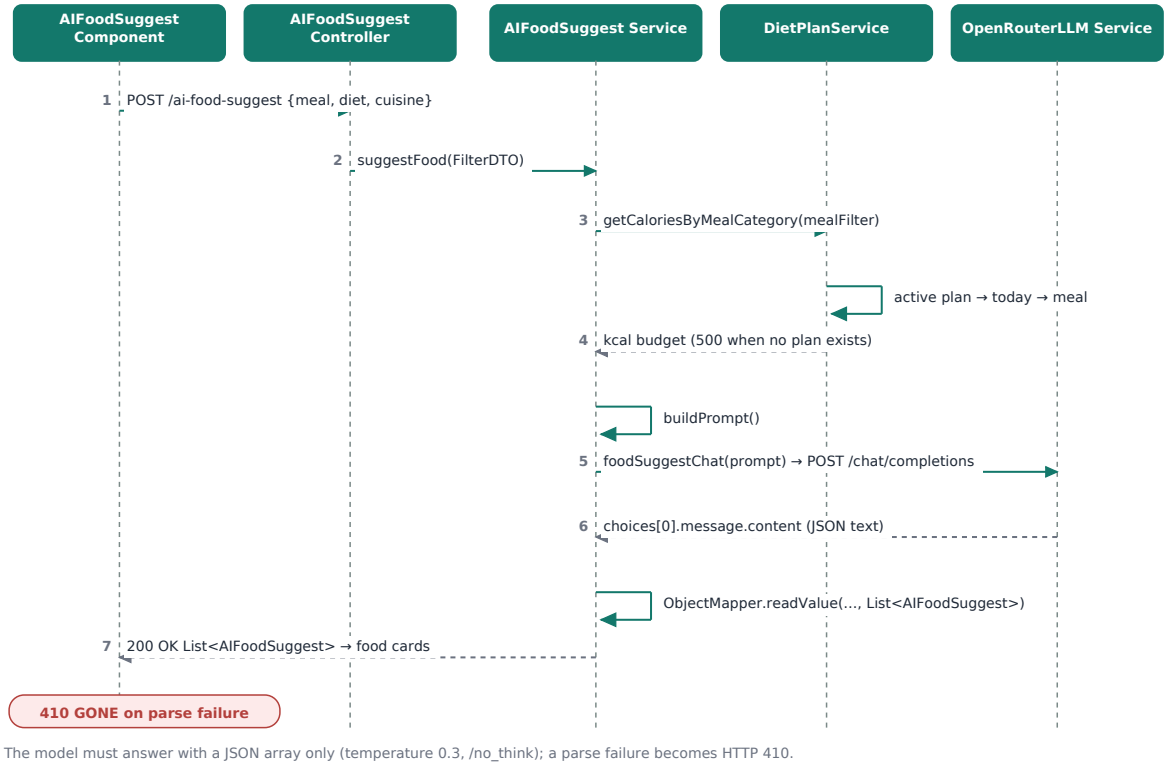
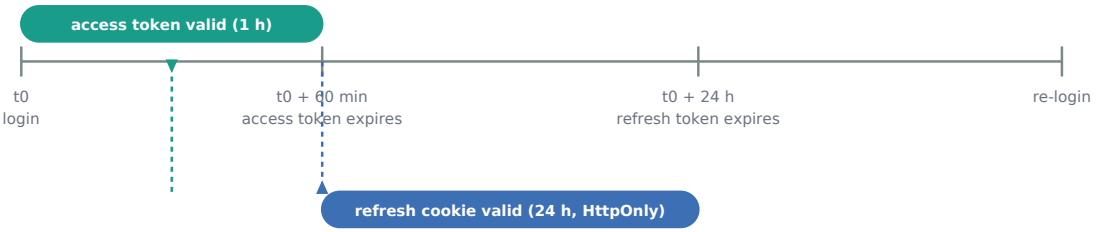


Figure 4.10: Sequence diagram — AI food suggestion via OpenRouter

Figure 4.11 relates the two token lifetimes. The access token is deliberately short so that a leaked value has limited value; the refresh token is longer lived but is inaccessible to page scripts, and the client never stores either token in local storage.



1. POST /authenticate → AuthenticationFilter issues the access token in the response body and the refresh token as a SameSite=Lax, HttpOnly cookie (path /, max-age 86 400 s).
2. TokenInterceptor verifies expiry locally with jwt-decode; an expired token triggers POST /user/refresh, which re-issues a fresh 60-minute access token from the cookie.
3. POST /user/logout clears the cookie (max-age 0) and the SPA drops the in-memory token.

Figure 4.11: Access and refresh token lifetimes and the refresh handshake

4.5.3 Activity Model

Figure 4.12 describes the happy path and both rejection paths of plan creation. The diagram makes an important property of the design explicit: client-side validation and server-side validation are independent, and the server never trusts the client's decision.

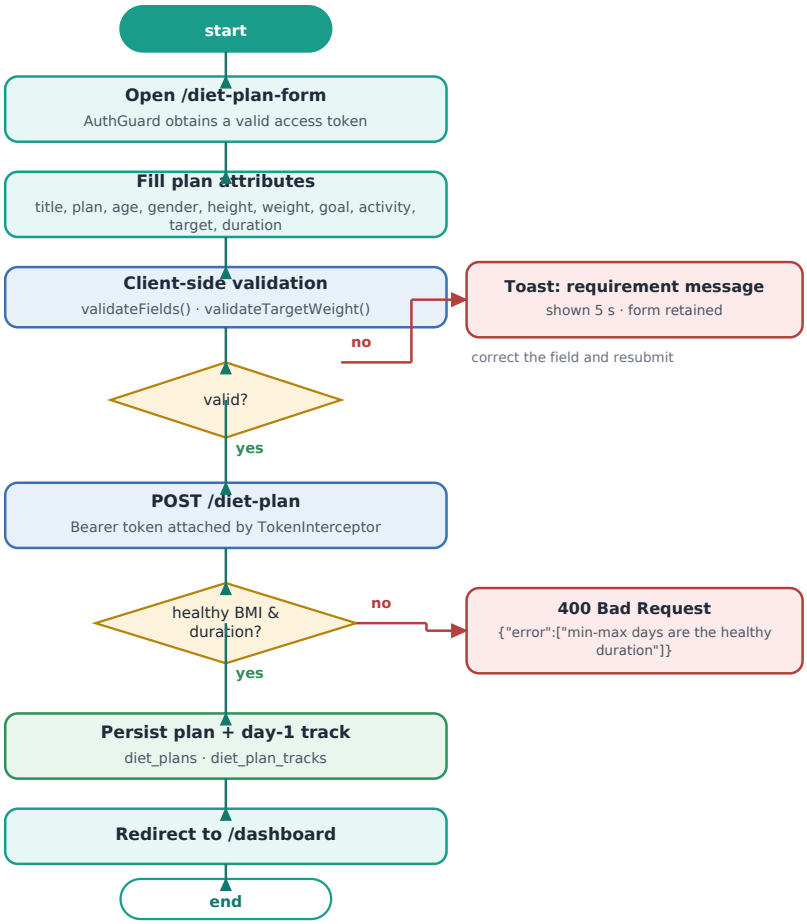
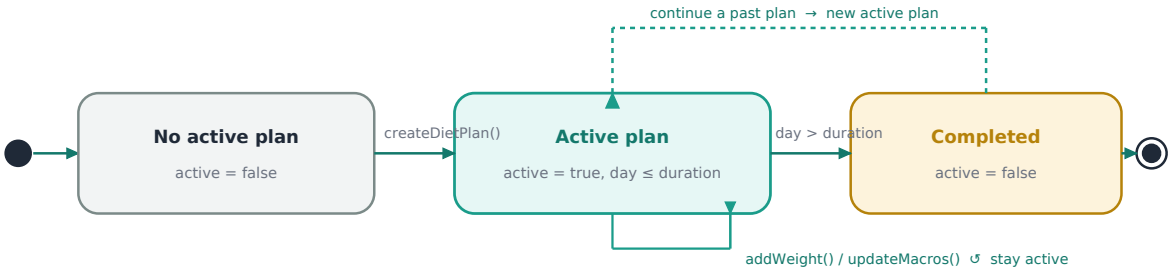


Figure 4.12: Activity diagram — plan creation with client and server validation

4.5.4 State Model

A plan has three observable states. It is created active, it remains active while the elapsed day count does not exceed its duration, and it becomes completed when the status endpoint detects that the duration has been exceeded. A completed plan can be cloned into a new active plan, which is the mechanism behind the 'Continue this plan' action of the no-active-plan screen.



A plan is created in the active state; once the elapsed day count exceeds the planned duration the server flips the flag to false on the next status check. Archived plans remain readable through the history screens and can be cloned into a new plan, which re-seeds a fresh day-1 track document. Terminal state is reached when the user abandons tracking altogether.

Figure 4.13: State machine — lifecycle of a diet plan

4.6 Entity–Relationship Model

Although MongoDB does not enforce referential integrity, the data model has a clear logical relationship structure in the sense of [15]: one user owns zero or more plans, and one plan owns zero or more tracking days. Both relationships are realised as identifier fields, and the service layer enforces ownership by always combining the identifier with the authenticated principal in the query predicate.

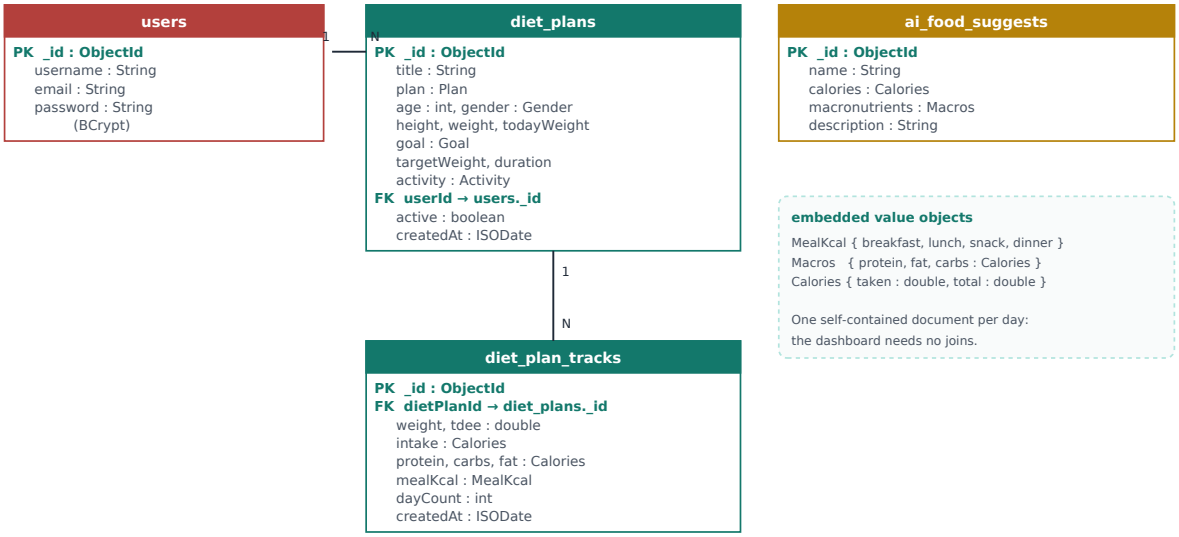


Figure 4.14: Document entity–relationship model of the Macrova database

4.7 Database Design and Data Dictionary

The database `macrova` contains four collections. Collections are created implicitly on first write; there is no schema-migration tool in this release, and the Java document classes act as the schema of record. Table 4.9 summarises the collections; the following tables give the data dictionary for each.

Table 4.9: Collection overview of the macrova database

Collection	Purpose	Document count per user	Growth pattern
users	One document per registered account, holding credentials	1	One per registration
diet_plans	One document per plan, holding body metrics, goal parameters and the active flag	n ≥ 1	One per plan creation or resumption
diet_plan_tracks	One document per tracked calendar day of one plan, holding targets, recorded values and the meal breakdown	one per plan per tracked day	One per day, per plan
ai_food_suggests	Model output mapped to a document class; the repository exists for optional caching of suggestions	0 in normal operation	Only when suggestions are persisted explicitly

4.7.1 users

Table 4.10: Data dictionary — collection users

Field	BSON type	Constraint / origin	Purpose
_id	ObjectId	Generated by the driver	Primary key; referenced by diet_plans.userId
username	String	Unique in practice; checked by existsByUsername before insertion	Login identifier and the JWT subject claim
email	String	Unique in practice; checked by existsByEmail	Contact identifier shown to the user
password	String	BCrypt digest produced by PasswordEncoder.encode	Credential verification; never returned by any endpoint

4.7.2 diet_plans

Table 4.11: Data dictionary — collection diet_plans

Field	BSON type	Constraint / domain	Purpose
_id	ObjectId	Driver-generated	Plan identifier used by every track path
title	String	Client-validated non-empty	User-facing plan name
plan	String (enum)	WEIGHT_LOSS, WEIGHT_GAIN	Direction of the energy target
age	Int32	18–65 (client and form constraints)	BMR term
gender	String (enum)	MALE, FEMALE	Selects the BMR constant
height	Double	140–210 cm	BMR term and BMI denominator
weight	Double	40–180 kg	Mass at plan creation
todayWeight	Double	Updated by addWeight	Most recent measured mass
goal	String (enum)	MILD (15), MODERATE (20), AGGRESSIVE (25)	Deficit or surplus percentage
targetWeight	Double	Direction-checked and BMI-checked	Goal mass
duration	Int32	Within the derived 7–28 day/kg window	Planned length in days
activity	String (enum)	SEDENTARY 1.2 … ATHLETE 1.9	Multiplier applied to BMR
userId	String	Owner identifier taken from the security context	Tenancy key for every query
active	Boolean	true on creation	Single active plan per user
createdAt	Date	LocalDate.now() at creation	Day-count origin and archive ordering

4.7.3 diet_plan_tracks

Table 4.12: Data dictionary — collection diet_plan_tracks

Field	BSON type	Constraint / domain	Purpose
_id	ObjectId	Driver-generated	Track identifier
weight	Double	Copied from the plan and updated by addWeight	Mass on that day
tdee	Double	findTDEE(age, weight, height, gender, activity)	Expenditure for that day
intake	Embedded { taken, total }	total = calorie target; taken = 4P + 9F + 4C	Calorie budget and consumption
protein / carbs / fat	Embedded { taken, total }	total in grams from the macro split	Macro budget and consumption
mealKcal	Embedded MealKcal	breakfast, lunch, snack, dinner, each a Macros object	Per-meal targets and entries
dayCount	Int32	ChronoUnit.DAYS.between(createdAt, today) + 1	Streak counter
createdAt	Date	Date of the tracking day	Day key and archive ordering
dietPlanId	String	Identifier of the owning plan	Join key and ownership check

4.7.4 ai_food_suggests

Table 4.13: Data dictionary — collection ai_food_suggests

Field	BSON type	Constraint / domain	Purpose
_id	ObjectId	Driver-generated	Suggestion identifier
name	String	Returned by the model	Food name rendered as a card title
calories	Embedded { taken, total }	Model figure mapped to the shared value object	Energy content
macronutrients	Embedded Macros	protein, fat, carbs	Macro figures
description	String	Returned by the model	One-line preparation summary

4.7.5 Embedded Value Objects

Table 4.14: Embedded value objects and their serialised shapes

Value object	Composition	Serialised shape
Calories	taken : double, total : double	`{"taken": 1148.0, "total": 1850.0}`
Macros	protein, fat, carbs : Calories	`{"protein": {…}, "fat": {…}, "carbs": {…}}`
MealKcal	breakfast, lunch, snack, dinner : Macros	`{"breakfast": {…}, "lunch": {…}, …}`
MetricsDTO	title, taken, duration, currentDay, currentWeight, intake, protein, carbs, fat, days[], weights[]	Flat projection returned by GET /diet-plan/metrics

4.7.6 Query Inventory and Access Patterns

Table 4.15: Repository query inventory and access patterns

Repository method	Predicate / projection	Used by
UserRepository.findByUsername	username equality	Authentication, username lookup
UserRepository.existsByUsername / existsByEmail	Existence check	Registration duplicate detection
DietPlanRepository.findByUserId	userId equality	GET /diet-plan/all
DietPlanRepository.findByIdAndUserId	id and owner equality	Every plan-scoped read
DietPlanRepository.findByUserIdAndActiveIsTrue	userId equality and active = true	Active-plan resolution
DietPlanRepository.findByActiveTrue	active = true	Any-active-plan probe
DietPlanRepository.findDietPlansHistoryByUserId	@Query projection of id, title, plan, weight, targetWeight, duration, active, sorted by createdAt descending	History list
DietPlanRepository.findFirstDietPlansHistoryByUserId	Same projection, first document	Resume a past plan
DietPlanTrackRepository.findFirstByCreatedAtAndDietPlanId	date and plan equality	Current day's document
DietPlanTrackRepository.findByDateRangeAndDietPlanId	@Query with $gte and $lte on createdAt, sorted ascending	Seven-day weight series
DietPlanTrackRepository.findAllDietPlanOverview	@Query projection of weight, tdee, intake, protein, carbs, fat, createdAt sorted descending	Archived day list

4.8 Input Design, Output Design and Interface Design System

4.8.1 Input Design

Input is collected through five forms. Table 4.16 lists the fields of the plan form, which is the most constrained, together with the validation applied in the component and the independent validation applied on the server. The remaining forms (registration, log-in, meal entry and weight update) are single-purpose and are described in the module section of Chapter 6.

Table 4.16: Input design and two-level validation of the diet-plan form

Field	Control	Client validation	Server validation
title	Text input	Non-empty	Not validated (informational)
plan	Two-state toggle	One of the two enumerations	Drives the direction rules
age	Number input	18 ≤ age ≤ 65	Used directly in the BMR equation
gender	Two-state toggle	MALE or FEMALE	Selects the BMR constant
height	Number input	140 ≤ height ≤ 210 cm	Denominator of the BMI check
weight	Number input	40 ≤ weight ≤ 180 kg	Numerator of the BMI check
goal	Three-state toggle	MILD, MODERATE or AGGRESSIVE	Percentage of deficit or surplus
activity	Five-state toggle	One of the five enumerations	Multiplier 1.2 to 1.9
targetWeight	Number input	Lower than current weight for loss, higher for gain; ≥ 40 kg and ≤ 110 kg	BMI of the target must be ≥ 18 or ≤ 25
duration	Number input	14 ≤ duration ≤ 730 days	Must lie inside the derived 7–28 day/kg window

4.8.2 Output Design

Output is delivered in three forms: JSON resources consumed by the client, derived visual components (progress bars, metric cards and a line chart) and textual messages carried by the toast component. Table 4.17 shows representative request and response bodies for the two most important resources.

Table 4.17: Representative request and response bodies

Endpoint	Request body	Response body
POST /diet-plan	`{title, plan, age, gender, height, weight, goal, targetWeight, duration, activity}`	`{id, title, plan, …, userId, active, createdAt}`
POST /diet-plan/{category}/{id}	`{protein:{taken,total}, fat:{taken,total}, carbs:{taken,total}}`	`{id, weight, tdee, intake, protein, carbs, fat, mealKcal, dayCount, createdAt, dietPlanId}`
GET /diet-plan/metrics	None (principal resolved from the token)	`{title, taken, duration, currentDay, currentWeight, intake, protein, carbs, fat, days[], weights[]}`
POST /ai-food-suggest	`{mealFilter, foodFilter, cuisineFilter}`	`[{name, calories:{taken,total}, macronutrients:{protein,fat,carbs}, description}]`
Error (any handler)	Not applicable	`{"error": ["message"]}` with 400, 401, 404 or 410

4.8.3 Interface Design System

The client uses a single stylesheet that declares design tokens as CSS custom properties; every component stylesheet consumes those tokens rather than literal colours. The palette is teal-based, with the primary colour reserved for forward actions and for the active navigation item. Table 4.18 lists the tokens that appear most often in the interface.

Table 4.18: Interface design tokens of the client application

Token	Value	Applied to
--color-primary	#1a9c8a	Primary buttons, active navigation, progress fill, avatar
--color-primary-dark	#13786b	Hover state, section headings
--color-primary-light	#e6f7f5	Selected chips, pale panels, active nav background
--color-text / --color-text-muted	#1f2937 / #6b7280	Body copy and secondary labels
--color-bg / --color-surface	#f8fafb / #ffffff	Page background and cards
--color-border	#e2e8e6	Card outlines, dividers, inputs
--color-danger / --color-warning	#dc2626 / #d97706	Error toast, over-target calorie bar
--radius-sm / -md / -lg	6 / 8 / 12 px	Inputs, buttons, cards
--space-xs … --space-2xl	4 … 48 px	Consistent rhythm across sections
--font-family	Inter, system-ui, sans-serif	All interface text
--shadow-sm / -md	Subtle elevation	Auth card, profile drop-down
Card measure	max-width 52 rem, centred	Dashboard and form containers

> **Note on figures.** Interface figures in Chapter 6 are renderings reconstructed from the implemented Angular templates and their scoped stylesheets. They reproduce the layout, wording and colour tokens of the delivered screens; they are not photographic captures of a running browser session, because the documentation environment used to produce this report had no Java runtime or MongoDB instance available. The distinction is stated here once and is not repeated for every figure.

CHAPTER 5

TECHNOLOGIES AND TOOLS

5.1 Introduction

This chapter records the software, hardware, languages, frameworks and tools used to build and operate the system. Every version listed in Table 5.3 was read from the two build descriptors of the repository — `backend/pom.xml` and `frontend/package.json` — rather than recalled from general knowledge, so the table describes the delivered artefact rather than a typical stack. Version numbers for the frameworks are cross-checked against their reference documentation [16], [17], [18].

5.2 Software Requirements

Table 5.1: Software requirements of the development and runtime environment

Component	Requirement	Used for
Operating system	Windows 10/11, macOS 13+ or a Linux distribution with a 64-bit JDK	Development and deployment host
Java Development Kit	JDK 21 (the Maven compiler release is 21)	Compiling and running the back end
Maven	Maven 3.9 (wrapper included as mvnw/mvnw.cmd)	Dependency resolution, build, test, packaging
Node.js	18.19+, 20.11+ or 22.x, as required by Angular 19	Building and serving the client
npm	10.x	Dependency installation, script execution
MongoDB	A local or hosted instance reachable on port 27017, database `macrova`	Persistence
MongoDB Compass / mongosh	Optional	Inspecting collections and documents during development
Web browser	Chromium 120+ or Firefox 120+	Running the client and the developer tools
Git and a GitHub account	Any recent version	Version control and remote backup
Postman or curl	Optional	Exercise of endpoints independently of the client

5.3 Hardware Requirements

Table 5.2: Hardware requirements for development and for deployment

Resource	Minimum	Recommended	Remark
Processor	Dual-core x86-64, 2.0 GHz	Quad-core, 2.5 GHz or faster	The Angular development server and the JVM compete for cores during a rebuild
Memory	4 GB	8 GB or more	Maven plus the JVM plus the Node process exceed 4 GB when both hot reload and tests run
Free disk	3 GB	10 GB	Maven dependencies, node_modules and the MongoDB data directory
Display	1366 × 768	1920 × 1080	The dashboard is designed for a content column of 52 rem
Network	Not required for local use	4G or broadband	Required only for dependency resolution and for the language-model call
Server (deployment)	One vCPU, 1 GB RAM	Two vCPU, 2 GB RAM	The service is stateless and memory-light; the database dominates the footprint

5.4 Programming Languages and Data Formats

5.4.1 Java 21

The back end is written entirely in Java 21. The language level is exploited in three specific ways. Records are used for every request and response payload (`LoginRequest`, `MetricsDTO`, `DietPlanHistoryDTO`, `TrackingOverviewDTO` and others), which removes boilerplate and makes immutability the default. Text blocks are not used, but switch expressions and pattern matching for *instanceof* appear in the service layer. The standard `java.time` API supplies `LocalDate` and `ChronoUnit`, which the day-count arithmetic depends on.

5.4.2 TypeScript 5.7

The client is written in TypeScript with strict template checking and `strict` compiler options. Every HTTP payload has an interface in `core/models` (`DietPlan`, `DietPlanTrack`, `Metrics`, `Macros`, `AIFoodSuggest`, `Toast`), and the services declare typed observables so that a mismatch between the server contract and the template is a compile error rather than a runtime undefined.

5.4.3 HTML5, CSS3 and JSON

Templates are HTML5 with Angular's control-flow blocks (`@if`, `@for`) and two-way binding through `ngModel`. Styling is CSS3 with custom properties, flexbox and grid; no CSS framework is used, which keeps the bundle small and the design system explicit. JSON is the sole wire format: requests and responses are JSON, the MongoDB driver stores BSON that is isomorphic to the Java classes, and the language model is instructed to answer in JSON.

5.5 Frameworks and Libraries

Table 5.3 lists the frameworks and libraries with the versions declared in the build descriptors. Where a version is managed by a parent rather than declared, the table says so.

Table 5.3: Frameworks and libraries with declared versions

Library / framework	Version	Module	Role in the system
Spring Boot	3.4.4 (parent)	back end	Auto-configuration, embedded Tomcat, dependency management
spring-boot-starter-web	managed	back end	Spring MVC, Jackson, validation infrastructure
spring-boot-starter-data-mongodb	managed	back end	MongoTemplate, repository proxies, BSON mapping
spring-boot-starter-security	managed	back end	Filter chain, password encoder, security context
spring-boot-starter-test	managed (test scope)	back end	JUnit 5, AssertJ, Spring test context
java-jwt (Auth0)	4.5.0	back end	HMAC512 signing and verification of access tokens
Lombok	1.18.38	back end	Getters, setters, constructors and builders on documents and DTOs
ModelMapper	3.2.2	back end	Mapping between documents and DTOs
Angular	19.2.x	front end	Component framework, routing, dependency injection, signals
RxJS	7.8.x	front end	Observable HTTP pipelines and operator composition
Chart.js	4.4.9	front end	Seven-day weight line chart on the dashboard
jwt-decode	4.0.0	front end	Reading the exp claim before use of a token
zone.js	0.15.x	front end	Change detection
Angular CLI / build-angular	19.2.x (dev)	front end	Build, serve, proxy and test harness
Karma and Jasmine	6.4 / 5.6 (dev)	front end	Unit-test runner and assertion library
OpenRouter chat-completions API	v1 (external)	integration	Language-model inference behind the LLMService interface

5.6 Development and Runtime Environment

Development was carried out on a single workstation with the back end started through the Maven wrapper and the client through the Angular CLI. The two processes run on ports 8080 and 4200 respectively; a development proxy rewrites the `/api` prefix so that the browser sees one origin and the refresh cookie is treated as first-party. MongoDB runs locally on its default port. Table 5.4 records the configuration values that the runtime expects.

Table 5.4: Runtime configuration items and their overrides

Configuration item	Default in the repository	Override
Server port	8080 (Spring Boot default)	server.port
MongoDB URI	mongodb://localhost:27017 (Spring Boot default; the property in application.properties is commented out)	spring.data.mongodb.uri
Database name	macrova (set in MongoConfig through MongoClientSettings)	Source change
JWT signing secret	Read from the JWT_SECRET environment variable	Environment
Access-token lifetime	60 minutes (constant in JWTTokenService)	Source change
Refresh-token lifetime	24 hours (constant in AuthenticationFilter)	Source change
Refresh cookie	Refresh-Token, HttpOnly, SameSite=Lax, path /	Source change
OpenRouter key	Read from OPENROUTER_API_KEY	Environment
OpenRouter model	Read from OPENROUTER_MODEL with a code default	Environment
Allowed client origin	http://localhost:4200	Source change (CORS configuration)
Client base prefix	/api (dev proxy target http://localhost:8080)	proxy.conf.json

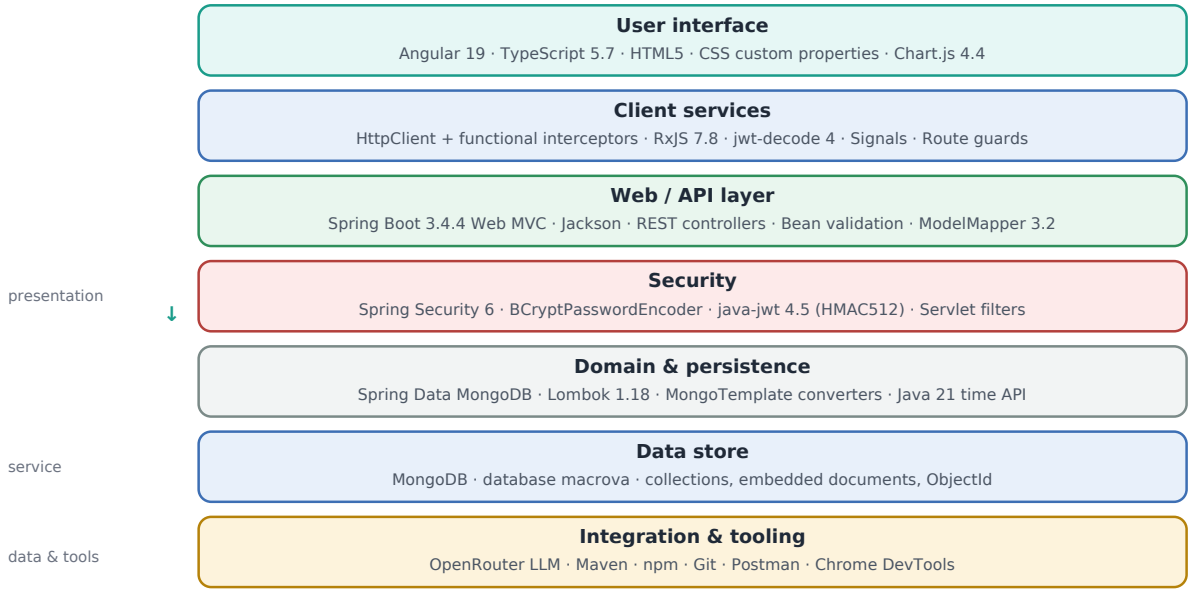


Figure 5.1: Technology stack of the delivered system, layer by layer

5.7 Summary of the Technology Selection

The selection was driven by three constraints: zero licence cost, a single language per tier and a mature ecosystem for the two risky areas, namely security and document persistence. Spring Boot satisfies the second and third constraints for the back end because security, JSON binding and Mongo repositories are first-party starters; Angular satisfies them for the client because routing guards, HTTP interceptors and typed forms are core features rather than add-ons. Chart.js is the single third-party visual dependency, chosen because its canvas renderer produces a small, dependency-free line chart. The language model is deliberately the only component that depends on a remote service, and it is isolated behind an interface so that the system remains fully functional when the service is unavailable.

CHAPTER 6

IMPLEMENTATION

6.1 Implementation Overview

The implementation is organised into eleven modules. Nine belong to the back end and two to the client; the boundary between them is the JSON contract described in Section 4.8. Table 6.1 lists the modules with the concrete artefacts that realise them and the number of source files each occupies, as counted in the repository.

Table 6.1: Module decomposition of the implementation

Module	Tier	Principal artefacts	Files
M1 Account registration and identity	back end	UserController, UserService, User, UserRepository, UserDTO	6
M2 Authentication, tokens and session	back end	SecurityConfig, AuthenticationFilter, JWTAuthenticationFilter, ExceptionHandlerFilter, CustomAuthenticationManager, JWTTokenService, SecurityConstants, UserSession	8
M3 Plan authoring and validation	back end	DietPlanController, DietPlanService (validation half), DietPlan, BodyMetricsRequest, BodyMetricsResponse	6
M4 Nutritional computation	back end	NutrientsCalculator, MacrosConstants, Calories, Macros, MealKcal	5
M5 Daily tracking and recomputation	back end	DietPlanService (tracking half), DietPlanTrack, DietPlanTrackRepository	3
M6 Metrics, status and reporting	back end	DietPlanService (metrics half), MetricsDTO, DietPlansHistoryDTO, DietPlanOverviewDTO	4
M7 AI food suggestion	back end	AIFoodSuggestController, AIFoodSuggestService, AIFoodSuggest, FilterDTO, LLMService, OpenRouterLLMService, ChatModel, MessageModel	8
M8 Error handling, mapping and configuration	back end	ApplicationExceptionHandler, ErrorResponse, BadRequestException, EntityNotFoundException, UnauthorizedException, Mapper, ModelMapperConfig, MongoConfig	9
M9 Client shell, routing and interception	front end	AppComponent, app.routes.ts, authGuard, TokenInterceptor, BaseUrlPrefixInterceptor, HeaderComponent, ToastComponent	12
M10 Client feature screens	front end	LoginComponent, SignupComponent, DashboardComponent, WeightTrackComponent, DietPlanComponent, DietPlanFormComponent, AIFoodSuggestComponent, DietPlansListComponent, DietPlanOverviewComponent, DietPlanTrackOverviewComponent, NoActiveDietPlanComponent, PageNotFoundComponent	24
M11 Service layer and models	front end	AuthService, DietPlanService, AIFoodSuggestService, DietPlanModel, Macros, Metrics, Toast, enumerations	14

6.2 Module Descriptions

6.2.1 M1 — Account Registration and Identity

Registration is handled by `POST /user/register`. The controller receives a `UserDTO`, and the service performs two independent existence checks before hashing: `existsByUsername` and `existsByEmail`. A duplicate raises `BadRequestException` with the offending field named in the message. The password is encoded with `BCryptPasswordEncoder` and only the digest is persisted. The endpoint is one of the two paths permitted without authentication, and it deliberately returns the created document without the password field.

For the client, the sign-up form performs its own validation before any request is issued, and each rule has its own message. The username must be between three and twenty characters, must contain only letters and digits and must not begin with a digit. The e-mail

address must match a simple `local@domain.tld` pattern. The password must be between eight and twenty characters and alphanumeric. A violation raises the toast component with the specific message, for example 'Password must be at least 8 characters long'. The form posts to the public registration path, so the token interceptor passes the request through untouched. On success the component navigates to `/dashboard`, but registration does not establish a session: the route guard then finds no valid access token and no refresh cookie, and redirects to the log-in screen, so the effective behaviour of a successful sign-up is a return to log-in with the account created.

6.2.2 M2 — Authentication, Tokens and Session

Authentication is implemented as a Spring Security filter pipeline of three custom filters placed around the framework's own. `ExceptionHandlerFilter` is registered before the authentication filter and wraps the remainder of the chain in a try-catch that translates token failures into plain-text responses: an expired token becomes HTTP 401 with the body 'Access token expired', any other verification failure becomes 401 with 'JWT verification failed', and any other runtime exception becomes 400. `AuthenticationFilter` extends `UsernamePasswordAuthenticationFilter`, is bound to the path `/authenticate`, parses the JSON body into a `User`, and delegates the credential comparison to `CustomAuthenticationManager`. On success it does not return an access token: it generates the refresh token and writes it into an HttpOnly, SameSite=Lax cookie named `Refresh-Token` with a 24-hour max-age. The access token is obtained separately from `POST /user/refresh`, which reads the cookie, verifies it and returns a 60-minute token in the response body.

`JWTAuthenticationFilter` runs after the authentication filter on every request; it skips the two public paths, extracts the bearer header, verifies the signature and populates the security context with a `UsernamePasswordAuthenticationToken` whose principal is the username. Downstream, `UserSession` turns that principal into an owner identifier by looking the user up and returning the string form of the `ObjectId`; every repository predicate is built from that value. This is the single mechanism by which tenancy is enforced.

The session is stateless: `SessionCreationPolicy.STATELESS` is configured, CSRF is disabled because no cookie-authenticated state-changing endpoint exists apart from refresh and log-out, and CORS is opened only to `http://localhost:4200` with credentials allowed.

6.2.3 M3 — Plan Authoring and Validation

Plan creation is a single service method, `DietPlanService.createDietPlan`, which sequences five steps: validate, stamp today's weight, stamp the owner and the active flag, persist the plan, and seed the first tracking document. The validation step is described in Algorithm 6.2. Because `updateDietPlan` simply delegates to `createDietPlan`, a plan update is semantically a new plan with the same fields: it receives a new identifier, a new creation date and a new day-1 track. The client uses this to implement resumption of an archived plan.

The controller exposes the plan endpoints with explicit status semantics: `POST /diet-plan` returns the stored document, `PUT /diet-plan/update` the replaced document, `POST /diet-plan/tdee` a preview of expenditure without persistence, and the read endpoints return either a document or a list of projections. The TDEE preview exists so that the form can show the user the expenditure implied by the entered metrics before any commitment is made.

6.2.4 M4 — Nutritional Computation

`NutrientsCalculator` is a final class with a private constructor and twelve static methods; it holds no state and has no injected dependency, which makes it directly callable from a unit test. The methods fall into four groups: body indices (`findBMR`, `findBMI`, `findTDEE`), duration bounds (`findMinimumDuration`, `findMaximumDuration`), energy targets (`findDeficit`, `findSurplus`, `roundCalories`) and macronutrient allocation (`findProtein`, `findFat`, `findCarbs`, `macrosPercentage`, `calcTakenDeficit`, `calcTotalDeficit`). The enumerations carry the domain constants: `Activity` maps each level to its multiplier, `Goal` maps mild, moderate and aggressive to 15, 20 and 25, and `Gender` selects the sign term of the Mifflin–St Jeor equation.

6.2.5 M5 — Daily Tracking and Recomputation

Tracking is built on one invariant: a tracking document always exists for the current date when a screen needs one. The resolver `findDietPlanTrackByDietPlanId` first verifies ownership of the plan, then looks up the document for `LocalDate.now()`, and if the lookup is empty it calls `createDietPlanTrack` instead of raising an error. The new document copies the plan's current weight, computes the elapsed day count as the number of days between the plan's creation date and today plus one, and is then filled by `processDietPlan`.

`processDietPlan` is the point at which the two halves of the system meet. It computes expenditure from the plan's age, height, gender and activity together with the *track*'*s* weight, converts expenditure into an intake target through the goal, derives the three macronutrient gram targets from that intake, and finally splits each target across the four meals. Because the method reads the weight from the track rather than the plan, a weight update on a given day changes that day's targets while leaving earlier days untouched.

Meal entry is handled by `updateMacros`. It resolves today's document, selects the meal object by category using a switch expression over the four constants in `MacrosConstants`, copies the three recorded values, re-sums the daily totals in `adjustMacros` and finally recomputes consumed calories with the 4-9-4 conversion. An unknown category is not rejected by a switch expression with a default branch pointing at the snack; the controller validates the category against `MacrosConstants.validateMacro` before the service is called, so a typo becomes HTTP 400 rather than a silent write.

6.2.6 M6 — Metrics, Status and Reporting

The dashboard is served by one resource, `GET /diet-plan/metrics`, whose assembly is described in Algorithm 6.5. The service resolves the active plan, ensures today's document exists, obtains the seven-day window in the Asia/Kolkata zone, maps each document in that window to a three-letter weekday label and a weight, and finally projects the current document into a flat `MetricsDTO`. The percentage of the calorie goal is computed on the server as `round(taken / total × 100)` so that the client never divides.

Two further endpoints support the plan lifecycle. `GET /diet-plan/status` compares the elapsed day count with the planned duration and, when the duration has been exceeded, sets `active = false` and returns true; the client uses this to route the user to the no-active-plan screen. `GET /diet-plan/active-plans` answers whether any plan in the database is active, which the router guard uses on a cold start. The archive is served by the history projection, the past-plan lookup and the overview query described in Table 4.15.

6.2.7 M7 — AI Food Suggestion

The suggestion module is a thin orchestration layer over an interface. `AIFoodSuggestService.suggestFood` obtains the calorie budget for the requested meal from `DietPlanService.getCaloriesByMealCategory`, builds the prompt, calls `LLMService.foodSuggestChat` and parses the response into a list of `AIFoodSuggest` objects with Jackson's `TypeReference`. The budget calculation re-uses the meal split: the snack, for example, is 15 per cent of each macro target, and the method converts those grams back into kilocalories with `calcTotalDeficit`. When no active plan exists the method catches `EntityNotFoundException` and returns a default budget of 500 kcal, which keeps the suggestion screen usable before a plan is created.

`OpenRouterLLMService` is the only class that knows the provider. It posts a two-message conversation to `https://openrouter.ai/api/v1/chat/completions` with a bearer key read from configuration, a system message that fixes the JSON schema and ends with `/no_think` to suppress reasoning output, a temperature of 0.3 and streaming disabled. It then walks the response tree `choices[0].message.content` and returns the text of that node. Any malformed content surfaces as `JsonProcessingException`, which the global handler maps to HTTP 410 and the client renders as an inline error beneath the filter row.

6.2.8 M8 — Error Handling, Mapping and Configuration

Four exception types carry the domain's failure vocabulary: `EntityNotFoundException` (404), `BadRequestException` (400), `UnauthorizedException` (401) and Jackson's `JsonProcessingException` (410). `ApplicationExceptionHandler` is a `@ControllerAdvice` that maps each to an `ErrorResponse` record whose single field is a list of strings, so the body is always `{"error":["…"]}`. Client-side failures are handled separately by the toast component, which holds one message and one severity and dismisses itself after a timeout.

Cross-cutting configuration is minimal by design. `ModelMapperConfig` registers a single mapper bean used to convert entities to DTOs where the shapes differ; `MongoConfig` builds a `MongoClientSettings` that names the database `macrova` explicitly, so the database name does not depend on the connection string; and `Mapper` in the shared package converts between `LocalDate` and `Date` for the range queries.

6.2.9 M9 and M10 — Client Shell and Feature Screens

The client is a standalone-component Angular application with no NgModule. The root component renders the header and a single router outlet. The header is hidden on the two authentication screens, which is why the log-in page is a full-height centred card rather than a page inside the shell. Routes are declared in `app.routes.ts`: the empty path redirects to `/dashboard`, the wildcard path redirects to the `not-found` route, and every component is imported eagerly rather than lazily, which is a deliberate simplification for an application of this size. Six routes are protected by `AuthGuard`: dashboard, progress, meals, history, diet-plan-form and no-active-plan. The two archive detail routes (`history/:id` and `history/track/:id`) carry no guard; they remain safe because the token interceptor still attaches the credential and because the service scopes every read to the authenticated owner, but the omission is an inconsistency that a later revision should correct.

Two functional interceptors form the client half of the session protocol. `BaseUrlPrefixInterceptor` prefixes every relative URL with `/api`, which the development proxy rewrites to the service root. `TokenInterceptor` passes the three public paths through untouched; for every other request it asks `AuthService.isAuthenticated()`, which refreshes the access token when it is absent or expired, then clones the request with the bearer header and `withCredentials: true`. The combination is what makes the refresh cookie effective while

keeping the access token in memory only.

Feature state is held in Angular signals rather than in a store library. Each screen keeps its own signal for the loaded resource and derives its displays from it, which keeps the data flow one-directional and removes the need for a global store in an application of this size. Shared presentation is limited to the header, the toast and the design tokens; every other component owns its stylesheet.

6.3 Core Algorithms and Pseudocode

Six algorithms carry the domain logic. They are presented as pseudocode in the order in which they execute during the life of a plan, and each is followed by the corresponding source excerpt in Section 6.4.

6.3.1 Algorithm 6.1 — Derivation of Daily Targets

Listing 6.1: Pseudocode for Algorithm 6.1

```
INPUT : age, gender, height (cm), weight (kg), activity, goal, plan
OUTPUT: tdee, intake, protein, fat, carbs (grams), mealKcal
1  bmr   <- 10*weight + 6.25*height - 5*age + (gender = MALE ? 5 : -161)
2  tdee  <- bmr * activity.multiplier
3  if plan = WEIGHT_LOSS then
4      intake <- round50(round(tdee) - pct(tdee, goal.percent))
5  else
6      intake <- round50(round(tdee) + pct(tdee, goal.percent - 10))
7  protein <- round(pct(intake, 30) / 4)
8  fat     <- round(pct(intake, 30) / 9)
9  carbs   <- round(pct(intake, 40) / 4)
10 mealKcal <- for each meal m in (breakfast 30, lunch 30, snack 15, dinner 25):
11                scale(protein, fat, carbs) by m.percent
12 return tdee, intake, protein, fat, carbs, mealKcal
```

Three details of the pseudocode deserve comment because they are easy to misread in the source. First, the surplus branch subtracts ten percentage points from the goal before applying it, so a mild goal of 15 per cent produces a surplus of 5 per cent of expenditure; the offset exists because a gain plan also adds the energy cost of tissue synthesis, which the equation does not model. Second, the calorie target is rounded to the nearest fifty or hundred so that targets read as 1,850 rather than 1,847. Third, the four meal percentages sum to one hundred, so the meal targets always reconstruct the daily target exactly; rounding is applied per macronutrient and may introduce a difference of one or two grams, which is visible in the interface and is acceptable.

6.3.2 Algorithm 6.2 — Validation of a Plan Request

Listing 6.2: Pseudocode for Algorithm 6.2

```
INPUT : plan request (plan, height, weight, targetWeight, duration)
OUTPUT: accepted, or BadRequestException carrying the permitted range
1  bmi <- weight_target / (height/100)^2
2  if plan = WEIGHT_LOSS and bmi < 18 then
3      reject(targetWeight + "kg is not the healthy weight")
4  if plan = WEIGHT_GAIN and bmi > 25 then
5      reject(targetWeight + "kg is not the healthy weight")
6  mass <- |weight - targetWeight|
7  min  <- plan = WEIGHT_LOSS ? ceil(mass * 7) : ceil(mass * 14)
8  max  <- ceil(mass * 28)
9  if duration < min or duration > max then
10     reject(min + "-" + max + " days are the healthy duration")
11 accept
```

The duration bounds are derived rather than configured. A loss plan permits one kilogram per week at the fastest, which is seven days per kilogram, and a quarter of a kilogram per

week at the slowest, which is twenty-eight days per kilogram; a gain plan permits half a kilogram per week at the fastest, which is fourteen days per kilogram, and shares the same slow bound. For a user of 76.5 kg aiming at 70 kg, the mass difference is 6.5 kg, so the permitted window is 46 to 182 days; a request for 30 days is rejected with the message '46-182 days are the healthy duration'.

6.3.3 Algorithm 6.3 — Resolution and Seeding of the Current Day

Listing 6.3: Pseudocode for Algorithm 6.3

```
INPUT : dietPlanId
OUTPUT: today's DietPlanTrack (existing or newly created)
1  plan <- findPlanByIdAndOwner(dietPlanId)      // 404 if absent or foreign
2  day  <- LocalDate.now()
3  track <- repository.findFirstByCreatedAtAndDietPlanId(day, plan.id)
4  if track is present then return track
5  dayCount <- DAYS.between(plan.createdAt, day) + 1
6  track <- new DietPlanTrack(plan.todayWeight, day, dayCount, plan.id)
7  processDietPlan(plan, track)                  // Algorithm 6.1
8  return repository.save(track)
```

Step 1 is the ownership gate. Because the lookup combines the plan identifier with the authenticated user identifier, a request for another user's plan behaves exactly like a request for a plan that does not exist, and the client receives 404 rather than a hint that the identifier is valid. Step 7 is the only place where a tracking document obtains its targets.

6.3.4 Algorithm 6.4 — Entry of a Meal and Recomputation of Totals

Listing 6.4: Pseudocode for Algorithm 6.4

```
INPUT : dietPlanId, category, macros{protein,fat,carbs}.taken
OUTPUT: updated DietPlanTrack
1  track <- resolveCurrentDay(dietPlanId)        // Algorithm 6.3
2  meal  <- switch category of
3             "breakfast" -> track.mealKcal.breakfast
4             "lunch"     -> track.mealKcal.lunch
5             "dinner"    -> track.mealKcal.dinner
6             "snack"     -> track.mealKcal.snack
7  meal.protein.taken <- input.protein
8  meal.fat.taken     <- input.fat
9  meal.carbs.taken   <- input.carbs
10 for each macro in (protein, fat, carbs):
11     track.macro.taken <- sum over the four meals of macro.taken
12 track.intake.taken  <- 4 * protein.taken + 9 * fat.taken + 4 * carbs.taken
13 return repository.save(track)
```

The recomputation is deliberately total rather than incremental: the service sums all four meals on every write instead of adding the difference. A sum over four objects is inexpensive, and it makes the operation idempotent, so a repeated submission of the same values cannot compound an error. Line 12 uses the energy densities rather than the calorie targets, so the recorded calorie figure is a function of the recorded grams alone.

6.3.5 Algorithm 6.5 — Assembly of the Dashboard Metrics

Listing 6.5: Pseudocode for Algorithm 6.5

```
INPUT : authenticated user
OUTPUT: MetricsDTO
1  plan  <- findActivePlanOfUser()               // 404 if none
2  track <- resolveCurrentDay(plan.id)           // Algorithm 6.3
3  (from, to) <- [today - 7 days 00:00, today 23:59:59.999] in Asia/Kolkata
4  days <- [], weights <- []
5  for each t in repository.findByDateRangeAndDietPlanId(from, to, plan.id):
6      days.append(shortWeekdayName(t.createdAt))
7      weights.append(t.weight)
8  taken <- round(track.intake.taken / track.intake.total * 100)
9  return { title: plan.title, taken, duration: plan.duration,
10          currentDay: track.dayCount, currentWeight: track.weight,
11          intake: track.intake.total, protein: track.protein.total,
12          carbs: track.carbs.total, fat: track.fat.total, days, weights }
```

The seven-day window is computed with an explicit zone rather than with the server's default zone. This matters because the tracking documents are keyed by `LocalDate`: without an explicit zone the window can shift by a day on a host configured for UTC, and a Sunday entry can appear under Monday. The series therefore contains one point per tracked day inside the window, which may be fewer than seven if the user did not open the application on some days; the chart renders the points it receives rather than fabricating gaps.

6.3.6 Algorithm 6.6 — Construction and Parsing of the Suggestion Call

Listing 6.6: Pseudocode for Algorithm 6.6

```
INPUT : mealFilter, foodFilter, cuisineFilter
OUTPUT: list of AIFoodSuggest
1  kcal   <- caloriesOfMeal(mealFilter)          // 500 when no active plan
2  prompt <- "Suggest me 5 healthy {cuisine} cuisine {food} foods for {meal}
              under {kcal} calories."
3  system <- "Respond in JSON with this format: '{ name: string, calories: number,
              macronutrients: { protein, carbs, fat }, description: string }[]'.
              No other output. /no_think"
4  response <- POST openrouter/chat/completions
5                { model, messages: [system, prompt], temperature: 0.3, stream: false }
6  text <- response.choices[0].message.content
7  foods <- deserialise(text, List<AIFoodSuggest>)
8  return foods                       // JsonProcessingException -> HTTP 410
```

The system message is doing three jobs at once: it fixes the schema, forbids surrounding prose, and suppresses the model's reasoning trace. The temperature of 0.3 trades variety for schema compliance, and the absence of a token limit means the provider applies its own default. Because the contract is enforced only by the prompt, the parse step is the real validation: a reply wrapped in a Markdown fence fails to deserialise, and the failure is surfaced rather than silently dropped.

6.4 Source Code Excerpts

The excerpts below are reproduced from the repository with only whitespace and import statements removed. Each is followed by a short explanation of the decision it embodies.

6.4.1 The Nutritional Engine

Listing 6.7: Basal metabolic rate, body-mass index and total daily energy expenditure

```
public static double findBMR(int age, double weight, double height, Gender gender) {
  return 10 * weight + 6.25 * height - 5 * age
      + (gender == Gender.MALE ? 5 : -161);
}
public static double findBMI(double height, double weight) {
  return (weight / (height * height)) * 10_000;
}
public static double findTDEE(int age, double weight, double height,
                              Gender gender, Activity activity) {
  return findBMR(age, weight, height, gender) * activity.getValue();
}
```

The three methods are deliberately free of rounding: they return the raw double and leave rounding to the caller that needs it. `findBMI` multiplies by ten thousand to convert centimetres to metres, so the method's unit contract is stated in its parameter names rather than assumed.

6.4.2 Energy Targets and Duration Bounds

Listing 6.8: Duration window, calorie rounding and the deficit and surplus targets

```
public static int findMinimumDuration(double mass, Plan plan) {
  return (int) Math.ceil(Plan.WEIGHT_LOSS.equals(plan) ? mass * 7 : (mass / 0.5) * 7);
}
public static int findMaximumDuration(double mass) {
  return (int) Math.ceil((mass / 0.25) * 7);
}
public static long roundCalories(long kcal) {
  long round = (kcal / 100) * 100;
  long difference = kcal - round;
  return round + (difference > 50 ? 100 : (difference > 0 ? 50 : 0));
}
public static long findDeficit(double tdee, int percent) {
  return roundCalories(Math.round(tdee - percentageOf(tdee, percent)));
}
public static long findSurplus(double tdee, int percent) {
  return roundCalories(Math.round(tdee) + percentageOf(tdee, percent - 10));
}
```

The rounding helper is the most idiosyncratic method in the class: a remainder above fifty rounds up to the next hundred, a remainder between one and fifty rounds up to the next fifty, and a remainder of zero is left alone. Targets are therefore always multiples of fifty, which the interface exploits by formatting them without decimals. The surplus method's subtraction of ten percentage points is the only asymmetry between the two directions and is the reason a moderate gain plan of twenty per cent produces a ten per cent surplus.

6.4.3 Macro Targets and Meal Split

Listing 6.9: Macronutrient gram targets and proportional scaling for a meal

```
public static double findProtein(double deficit) {
  return Double.valueOf(Math.round(percentageOf(deficit, 30) / 4.0));
}
public static double findFat(double deficit) {
  return Double.valueOf(Math.round(percentageOf(deficit, 30) / 9.0));
}
public static double findCarbs(double deficit) {
  return Double.valueOf(Math.round(percentageOf(deficit, 40) / 4.0));
}
public static Macros macrosPercentage(Macros macros, int percent) {
  return new Macros(
    percentageOf(macros.getProtein().getTotal(), percent),
    percentageOf(macros.getFat().getTotal(), percent),
    percentageOf(macros.getCarbs().getTotal(), percent));
}
```

Each macro method takes the calorie budget and returns grams, dividing by the energy density of the nutrient: four for protein and carbohydrate, nine for fat. `macrosPercentage` performs the meal split by scaling the *total* field of each macro rather than the taken field, which guarantees that meal targets and daily targets are consistent by construction.

6.4.4 Plan Validation and Creation

Listing 6.10: Validation and creation of a diet plan

```
private void validateDietPlan(DietPlan dietPlan) {
  double bmi = NutrientsCalculator.findBMI(dietPlan.getHeight(), dietPlan.getTargetWeight());
  if ((Plan.WEIGHT_LOSS.equals(dietPlan.getPlan()) && bmi < 18) ||
      (Plan.WEIGHT_GAIN.equals(dietPlan.getPlan()) && bmi > 25)) {
    throw new BadRequestException(dietPlan.getTargetWeight() + "kg is not the healthy weight");
  }
  double mass = dietPlan.getWeight() - dietPlan.getTargetWeight();
  int min = NutrientsCalculator.findMinimumDuration(mass, dietPlan.getPlan());
  int max = NutrientsCalculator.findMaximumDuration(mass);
  if (dietPlan.getDuration() < min || dietPlan.getDuration() > max) {
    throw new BadRequestException(min + "-" + max + " days are the healthy duration");
  }
}
public DietPlan createDietPlan(DietPlan dietPlan) {
  validateDietPlan(dietPlan);
  dietPlan.setTodayWeight(dietPlan.getWeight());
  dietPlan.setUserId(userSession.getAuthenticatedUserId());
  dietPlan.setActive(true);
  dietPlannerRepository.save(dietPlan);
  createDietPlanTrack(dietPlan);
  return dietPlan;
}
```

The signed difference `mass` is passed to the duration methods without an absolute value; both bound methods multiply by a positive factor, and a negative mass can only arise when the target lies on the wrong side of the current weight, which the client prevents and which the body-mass index rule rejects in most cases. The creation method mutates the incoming entity rather than constructing a copy, which is safe because the entity is request-scoped and is not referenced after the response.

6.4.5 Track Seeding and Target Processing

Listing 6.11: On-demand seeding of a tracking day and derivation of its targets

```
public DietPlanTrack findDietPlanTrackByDietPlanId(String dietPlanId) {
  DietPlan dietPlan = findDietPlanById(dietPlanId);
  return dietPlanTrackRepository
      .findFirstByCreatedAtAndDietPlanId(LocalDate.now(), dietPlanId)
      .orElseGet(() -> createDietPlanTrack(dietPlan));
}
private void processDietPlan(DietPlan dietPlan, DietPlanTrack dietPlanTrack) {
  dietPlanTrack.setTdee(NutrientsCalculator.findTDEE(
      dietPlan.getAge(), dietPlanTrack.getWeight(), dietPlan.getHeight(),
      dietPlan.getGender(), dietPlan.getActivity()));
  double intake = getIntake(dietPlanTrack.getTdee(), dietPlan.getGoal(), dietPlan.getPlan());
  dietPlanTrack.setIntake(new Calories(intake));
  dietPlanTrack.setProtein(new Calories(NutrientsCalculator.findProtein(intake)));
  dietPlanTrack.setFat(new Calories(NutrientsCalculator.findFat(intake)));
  dietPlanTrack.setCarbs(new Calories(NutrientsCalculator.findCarbs(intake)));
  dietPlanTrack.setMealKcal(splitForMealKcal(dietPlanTrack));
}
```

The single-argument constructor of `Calories` initialises the target and leaves the recorded field at the Java default of zero, so a day that has not yet been filled shows a zero-per-cent bar rather than a full one. That is the honest presentation: the target is known the moment the plan exists, while the recorded value is unknown until the user confirms what was eaten. Every write thereafter recomputes the recorded fields as described in Algorithm 6.4.

6.4.6 Weight Update and Meal Entry

Listing 6.12: Weight update with full recomputation, and meal entry with total recomputation

```
public DietPlanTrack addWeight(String dietPlanId, double weight) {
  DietPlan dietPlan = findDietPlanById(dietPlanId);
  dietPlan.setTodayWeight(weight);
  dietPlannerRepository.save(dietPlan);
  DietPlanTrack dietPlanTrack = findDietPlanTrackByDietPlanId(dietPlanId);
  dietPlanTrack.setWeight(weight);
  processDietPlan(dietPlan, dietPlanTrack);
  return dietPlanTrackRepository.save(dietPlanTrack);
}
public DietPlanTrack updateMacros(String dietPlanId, String category, Macros macros) {
  DietPlanTrack dietPlanTrack = findDietPlanTrackByDietPlanId(dietPlanId);
  Macros dietPlanMacros = getMacrosByCategory(dietPlanTrack.getMealKcal(), category);
  dietPlanMacros.getProtein().setTaken(macros.getProtein().getTaken());
  dietPlanMacros.getFat().setTaken(macros.getFat().getTaken());
  dietPlanMacros.getCarbs().setTaken(macros.getCarbs().getTaken());
  adjustMacros(dietPlanTrack);
  dietPlanTrack.getIntake().setTaken(calcDeficitTaken(dietPlanTrack));
  return dietPlanTrackRepository.save(dietPlanTrack);
}
```

The weight update writes to three places in a fixed order: the plan's `todayWeight`, which the next day's seeding will inherit; the current track's weight, which the chart reads; and the derived targets, which `processDietPlan` recomputes. The meal entry mutates the meal object inside the document and then recomputes the aggregates; because the document is embedded, saving the outer object persists the change to the embedded object without any explicit update statement.

6.4.7 Metrics Projection

Listing 6.13: Assembly of the seven-day window and of the metrics projection

```
public MetricsDTO getMetricsByDateRange() {
  List<String> days = new ArrayList<>();
  List<Double> weights = new ArrayList<>();
  Date[] dateRange = getOneWeekDateRange();
  DietPlan dietPlan = findDietPlanByUserId();
  DietPlanTrack dietPlanTrack = findDietPlanTrackByDietPlanId(dietPlan.getId());
  dietPlanTrackRepository
      .findByDateRangeAndDietPlanId(dateRange[0], dateRange[1], dietPlan.getId())
      .forEach(track -> {
        days.add(getDayName(track.getCreatedAt()));
        weights.add(track.getWeight());
      });
  return buildMetrics(dietPlanTrack, dietPlan.getTitle(), dietPlan.getDuration(), days, weights);
}
private Date[] getOneWeekDateRange() {
  ZoneId zoneId = ZoneId.of("Asia/Kolkata");
  LocalDate today = LocalDate.now(zoneId);
  return new Date[] {
      Date.from(today.minusWeeks(1).atStartOfDay(zoneId).toInstant()),
      Date.from(today.atTime(LocalTime.MAX).atZone(zoneId).toInstant()) };
}
```

Two lists are filled in parallel rather than a list of objects being mapped on the client, which keeps the payload small and the chart's input shape simple. The window starts at the first instant of the day seven days ago rather than at midnight of today minus seven, so a plan whose first day is exactly a week old contributes a point.

6.4.8 The Security Filter Chain

Listing 6.14: Construction of the stateless security filter chain

```
AuthenticationFilter authenticationFilter =
    new AuthenticationFilter(customAuthenticationManager, jwtTokenService);
authenticationFilter.setFilterProcessesUrl("/authenticate");
http
  .csrf(csrf -> csrf.disable())
  .cors(cors -> cors.configurationSource(corsConfigurationSource()))
  .authorizeHttpRequests(auth -> auth
      .requestMatchers(HttpMethod.POST,
                       SecurityConstants.REGISTER_PATH,
                       SecurityConstants.REFRESH_PATH).permitAll()
      .anyRequest().authenticated())
  .addFilterBefore(new ExceptionHandlerFilter(), AuthenticationFilter.class)
  .addFilter(authenticationFilter)
  .addFilterAfter(new JWTAuthenticationFilter(jwtTokenService), AuthenticationFilter.class)
  .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
return http.build();
```

The ordering is the substance of the configuration. The exception filter must precede everything that can throw a token exception, the authentication filter must run before the JWT filter because it writes the cookie the latter's flow depends on, and the authorisation rule must run last. Permit-all is granted by HTTP method as well as by path, so a hypothetical `GET /user/register` would still require a token.

6.4.9 Issue of the Refresh Cookie

Listing 6.15: Issue of the refresh token as an HttpOnly cookie

```
@Override
protected void successfulAuthentication(HttpServletRequest request,
                                        HttpServletResponse response,
                                        FilterChain chain,
                                        Authentication auth) {
  String refreshToken = jwtTokenService.generateToken(
      auth.getName(), SecurityConstants.REFRESH_TOKEN_EXPIRATION);
  ResponseCookie cookie = ResponseCookie
      .from(SecurityConstants.REFRESH_TOKEN_HEADER, refreshToken)
      .httpOnly(true)
      .sameSite("Lax")
      .secure(false)
      .path("/")
      .maxAge(SecurityConstants.REFRESH_TOKEN_EXPIRATION)
      .build();
  response.setHeader(HttpHeaders.SET_COOKIE, cookie.toString());
}
```

`httpOnly` removes the cookie from the reach of scripts, so a cross-site scripting flaw cannot exfiltrate the long-lived credential; `SameSite=Lax` prevents it from being sent on cross-site sub-requests while still allowing top-level navigation; and the path of the root makes it available to the refresh endpoint regardless of the deployment prefix. The `secure` flag is false because the development environment is served over plain HTTP; Section 7.7 records this as a known deviation from the OWASP guidance for production.

6.4.10 The Token Interceptor of the Client

Listing 6.16: Functional HTTP interceptor that attaches the access token

```
const PUBLIC_URLS = ['/authenticate', '/user/register', '/user/refresh'];
export const TokenInterceptor: HttpInterceptorFn = (req, next) => {
  if (PUBLIC_URLS.some((url) => req.url.includes(url))) {
    return next(req);
  }
  const authService = inject(AuthService);
  return authService.isAuthenticated().pipe(
    switchMap((isAuth) => {
      if (isAuth) {
        const token = authService.getAccessToken();
        const cloneReq = req.clone({
          setHeaders: { Authorization: `Bearer ${token}` },
          withCredentials: true,
        });
        return next(cloneReq);
      }
      return next(req);
    }),
  );
};
```

`isAuthenticated()` is an observable rather than a boolean because it may have to perform a network call to refresh an expired token; the `switchMap` makes the interceptor wait for that call and then continue with the original request. The three public URLs are matched by substring, which also covers the prefixed forms produced by `BaseUrlPrefixInterceptor`.

6.4.11 Repository Projections

Listing 6.17: Derived queries and a field-projected history query

```
public interface DietPlanRepository extends MongoRepository<DietPlan, String> {
  @Query(
    value = "{ userId:  ?0 }",
    fields = "{ _id: 1, title: 1, plan: 1, weight: 1, targetWeight: 1, duration: 1, active: 1 }",
    sort = "{ createdAt:  -1 }"
  )
  List<DietPlansHistoryDTO> findDietPlansHistoryByUserId(String userId);
  Optional<DietPlan> findByIdAndUserId(String id, String userId);
  Optional<DietPlan> findByUserIdAndActiveIsTrue(String userId);
  Optional<DietPlan> findByActiveTrue();
}
```

The history query returns a projection rather than documents because the archive screen needs seven fields and none of the body metrics. Because the projection includes the identifier under the name `_id`, the client receives it as `id` after Jackson's mapping, which is what the navigation links use.

6.4.12 Global Exception Translation

Listing 6.18: Translation of domain exceptions into JSON error responses

```
@ControllerAdvice
public class ApplicationExceptionHandler {
  @ExceptionHandler
  public ResponseEntity<ErrorResponse> handleEntityNotFoundException(EntityNotFoundException e) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ErrorResponse(e.getMessage()));
  }
  @ExceptionHandler
  public ResponseEntity<ErrorResponse> handleBadRequestException(BadRequestException e) {
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(new ErrorResponse(e.getMessage()));
  }
  @ExceptionHandler
  public ResponseEntity<ErrorResponse> handleJsonProcessingException(JsonProcessingException e) {
    return ResponseEntity.status(HttpStatus.GONE).body(new ErrorResponse(e.getMessage()));
  }
}
```

The choice of 410 Gone for a malformed model response is unusual and deliberate: the resource that the client asked for is genuinely unavailable, and a 5xx code would wrongly suggest a defect in the service. Every handler returns the same body shape, so the client's error branch is written once.

6.5 Screen Walkthroughs

This section walks through the twelve interface states of the delivered client. As stated in the note at the end of Chapter 4, the figures are renderings reconstructed from the templates and scoped stylesheets, reproduced at the reference width of the application's centred content column.

6.5.1 Screen 6.1 — Login

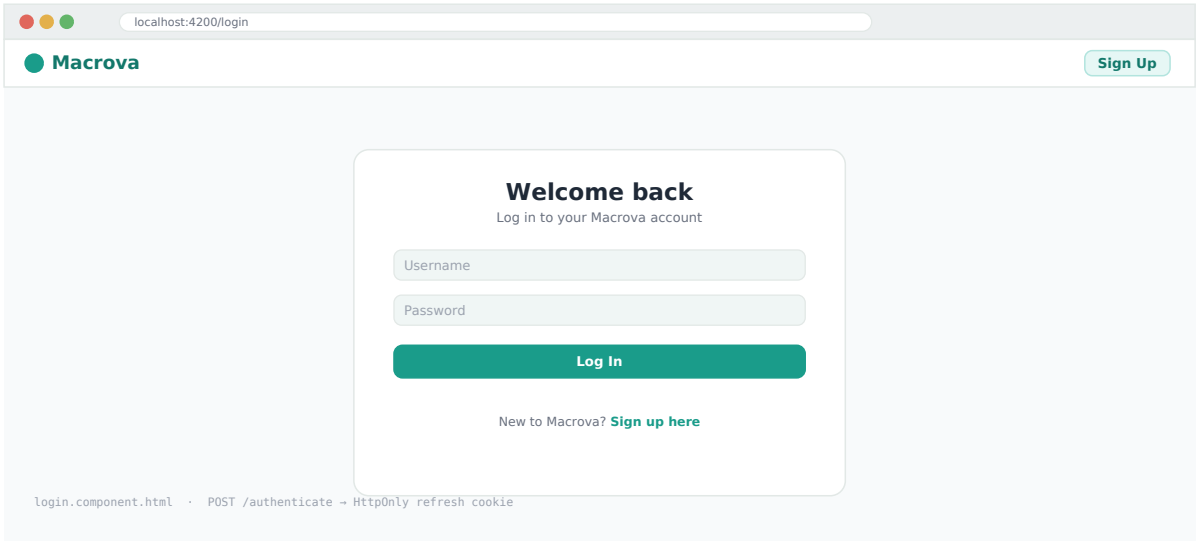


Figure 6.1: Interface rendering — login screen

The log-in screen is the only entry point for an existing account. The card is centred vertically and horizontally, carries the product mark, and exposes exactly two fields plus a primary action. The footnote is a single line in which the lead text and the link are laid out with an explicit advance so that the two do not overlap at small widths. The card is hidden from the application shell, so the header is absent.

6.5.2 Screen 6.2 — Sign up

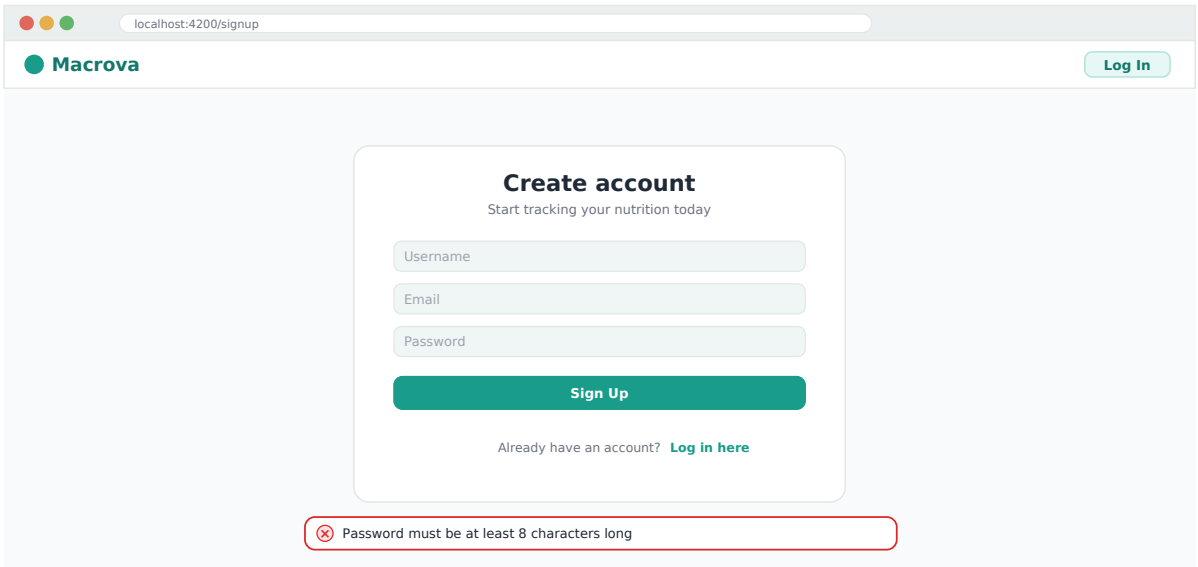


Figure 6.2: Interface rendering — sign up screen

Registration collects three fields: a username, an e-mail address and a password. Three families of rule are enforced before the request is sent — a username of three to twenty alphanumeric characters that does not begin with a digit, an e-mail matching the local@domain pattern, and a password of eight to twenty alphanumeric characters. A violation raises the toast shown at the foot of the card, which names the specific rule that failed, rather than a browser-native validation bubble, so the message is styled and announced consistently with every other error in the application.

6.5.3 Screen 6.3 — Dashboard

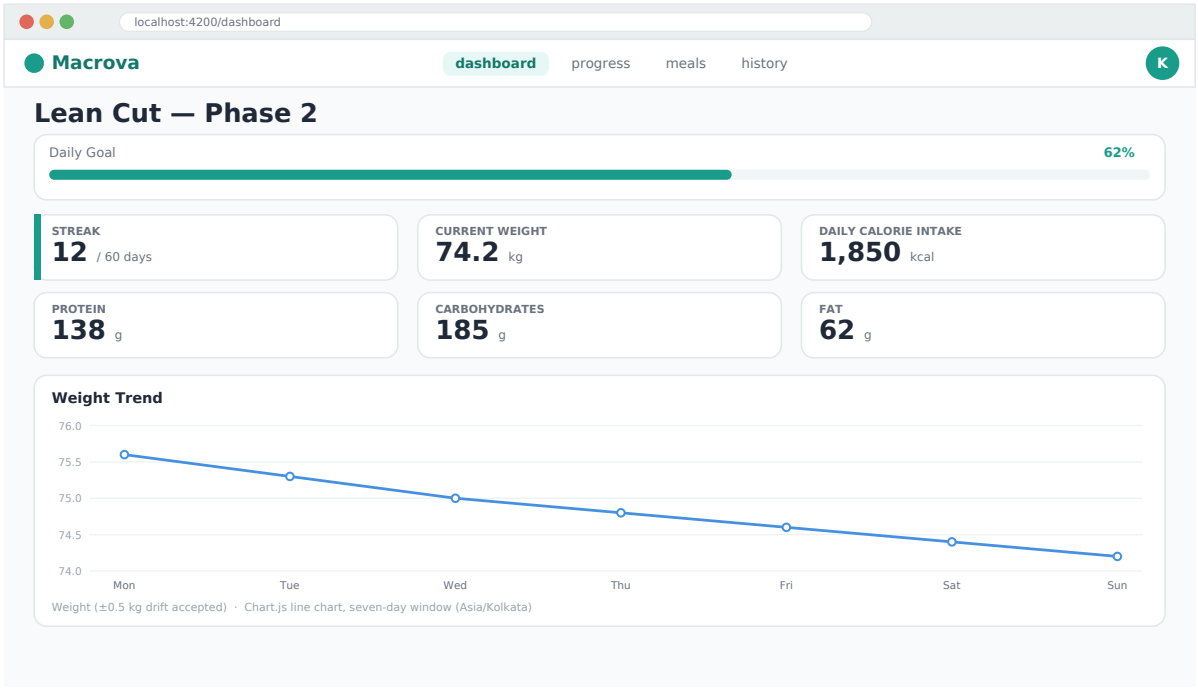


Figure 6.3: Interface rendering — dashboard screen

The dashboard is the landing page after authentication. It answers four questions at a glance: how much of today's calorie budget has been used (the 62 per cent bar), how many consecutive days have been tracked (the 12 of 60 streak), what the macro targets are (the three metric cards), and how body weight has moved over the last week (the line chart, which falls from 75.6 kg to 74.2 kg across the seven labelled weekdays). The chart is rendered by Chart.js on a canvas whose height is fixed so that the caption cannot collide with the axis labels.

6.5.4 Screen 6.4 — Progress

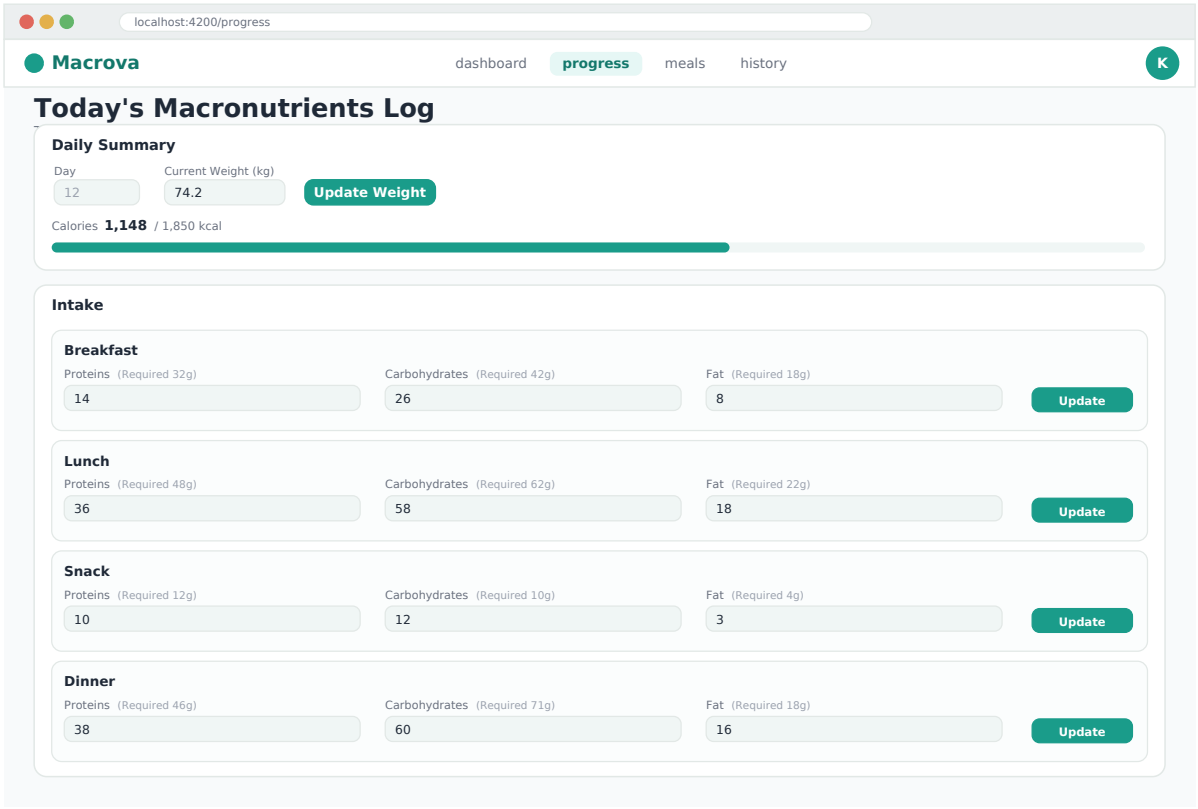


Figure 6.4: Interface rendering — progress screen

The progress screen is the tracking surface. The daily summary card repeats the calorie position and offers the weight-update control; beneath it, four meal cards show the required grams for protein, carbohydrate and fat alongside editable fields for what was actually eaten, with an Update action per card. The requirement figures are the 30/30/15/25 split of the daily targets, so breakfast shows 32 g of protein where the daily target is 107 g.

6.5.5 Screen 6.5 — Diet plan form

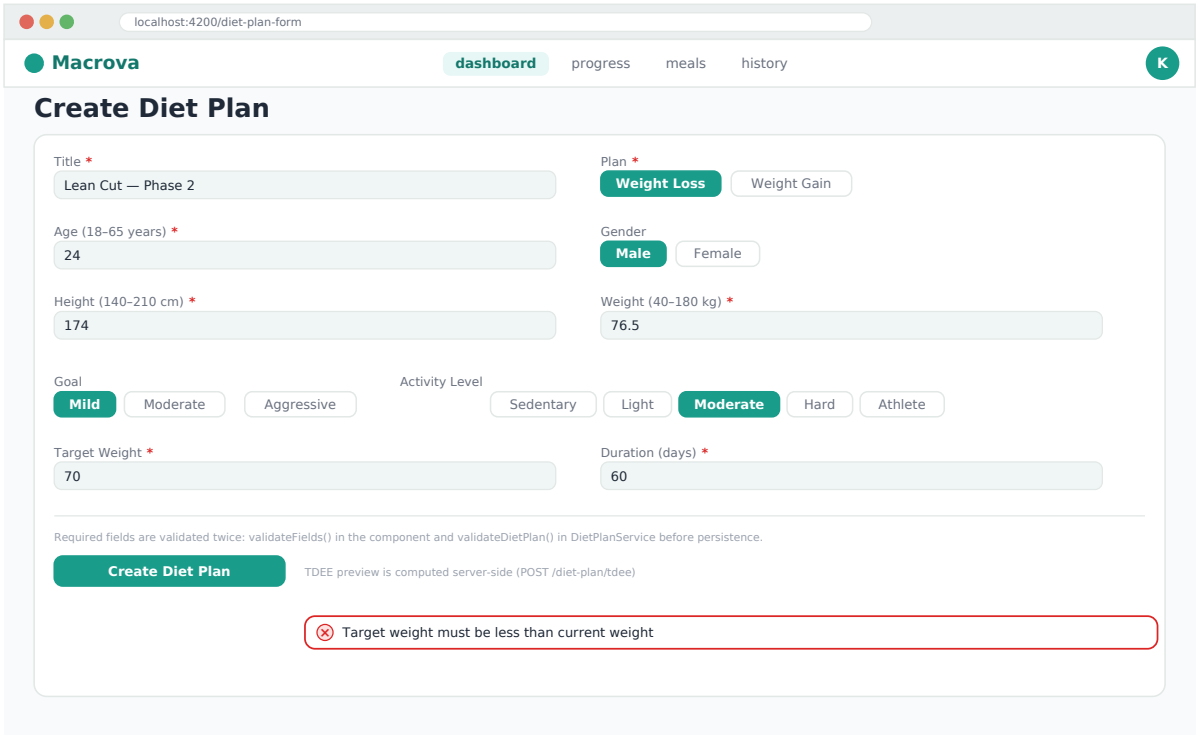


Figure 6.5: Interface rendering — diet plan form screen

The plan form is the most constrained screen. Direction (loss or gain), gender, goal and activity are segmented controls rather than drop-downs, which makes the permitted values visible without interaction; age, height, weight, target weight and duration are numeric inputs with bounds stated in the labels. The helper line beneath the duration field states the permitted window for the current mass difference, and the toast at the foot illustrates the rejection of a target weight on the wrong side of the current weight.

6.5.6 Screen 6.6 — Meals

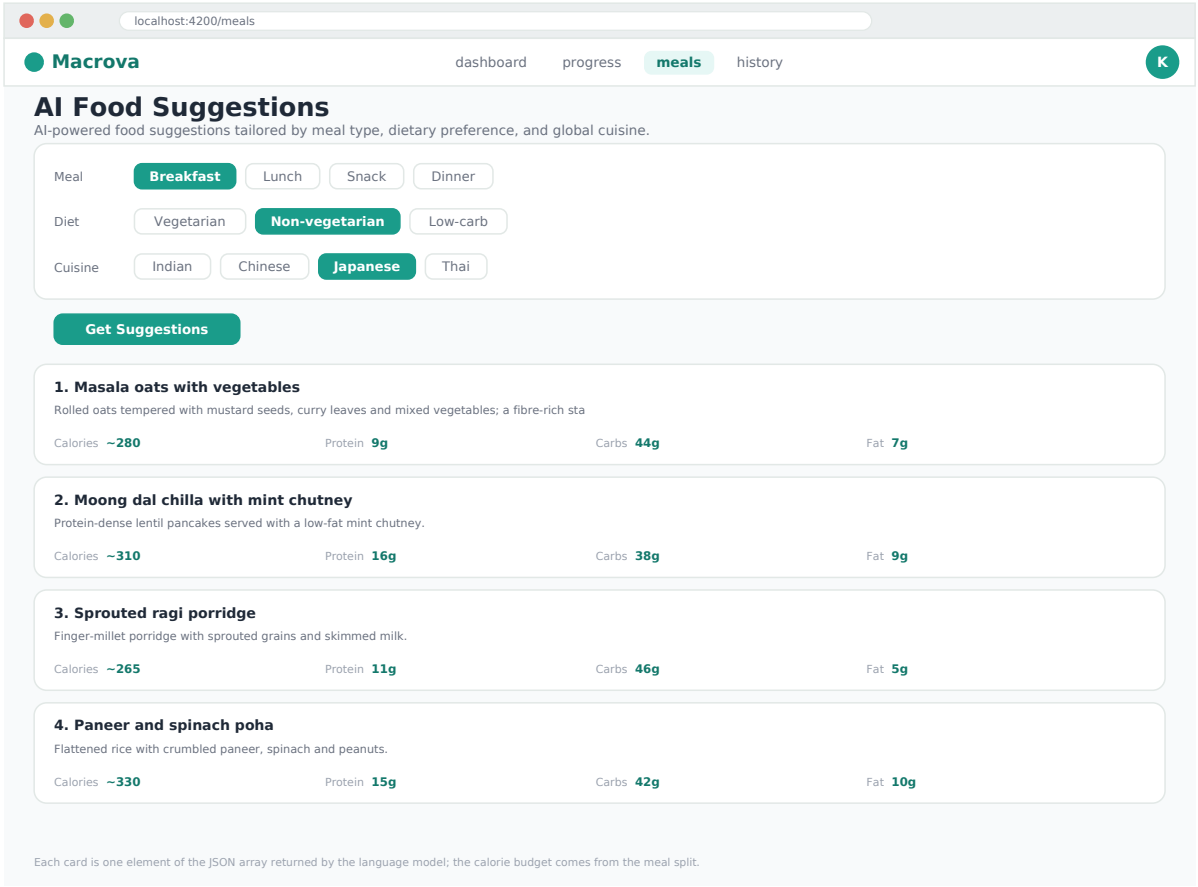


Figure 6.6: Interface rendering — meals screen

The suggestion screen combines three filters — meal, dietary preference and cuisine — with a single primary action. Each returned food is rendered as a card with its energy and its three macronutrient figures, so the user can compare candidates against the remaining budget of the selected meal without leaving the page.

6.5.7 Screen 6.7 — Plan history

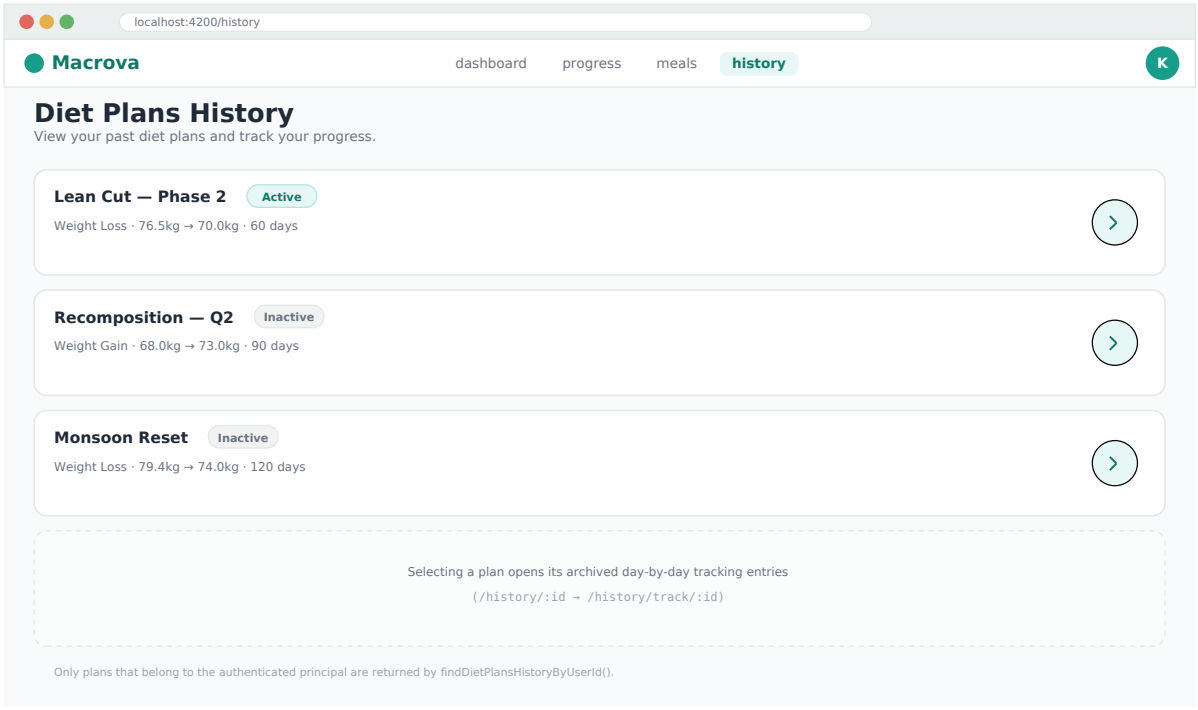


Figure 6.7: Interface rendering — plan history screen

The history screen lists every plan of the authenticated user in reverse chronological order. Each row carries the title, the direction, the starting and target weights, the planned duration and a status pill that distinguishes the active plan from completed ones; the row itself is a link to the archived day list.

6.5.8 Screen 6.8 — Plan overview

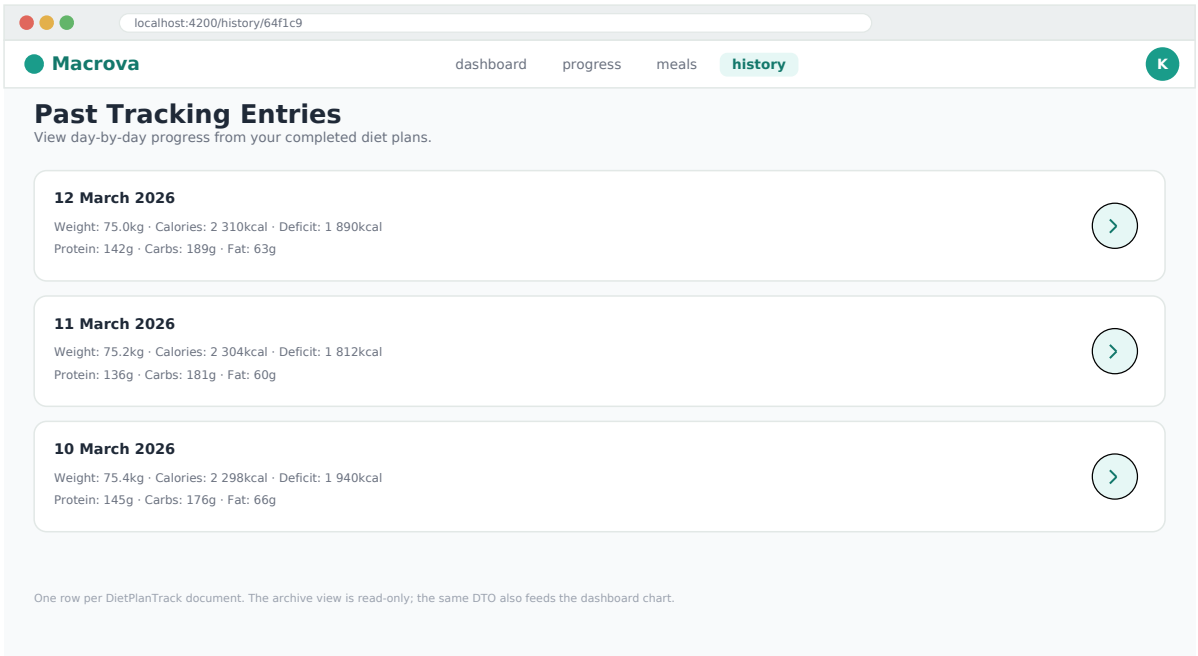


Figure 6.8: Interface rendering — plan overview screen

The plan overview lists the tracking days of one plan. Each entry shows the date, the weight recorded on that day, the expenditure computed for it and the intake and macro values that were in force, which is what makes the archive auditable after the plan has ended.

6.5.9 Screen 6.9 — Tracked day

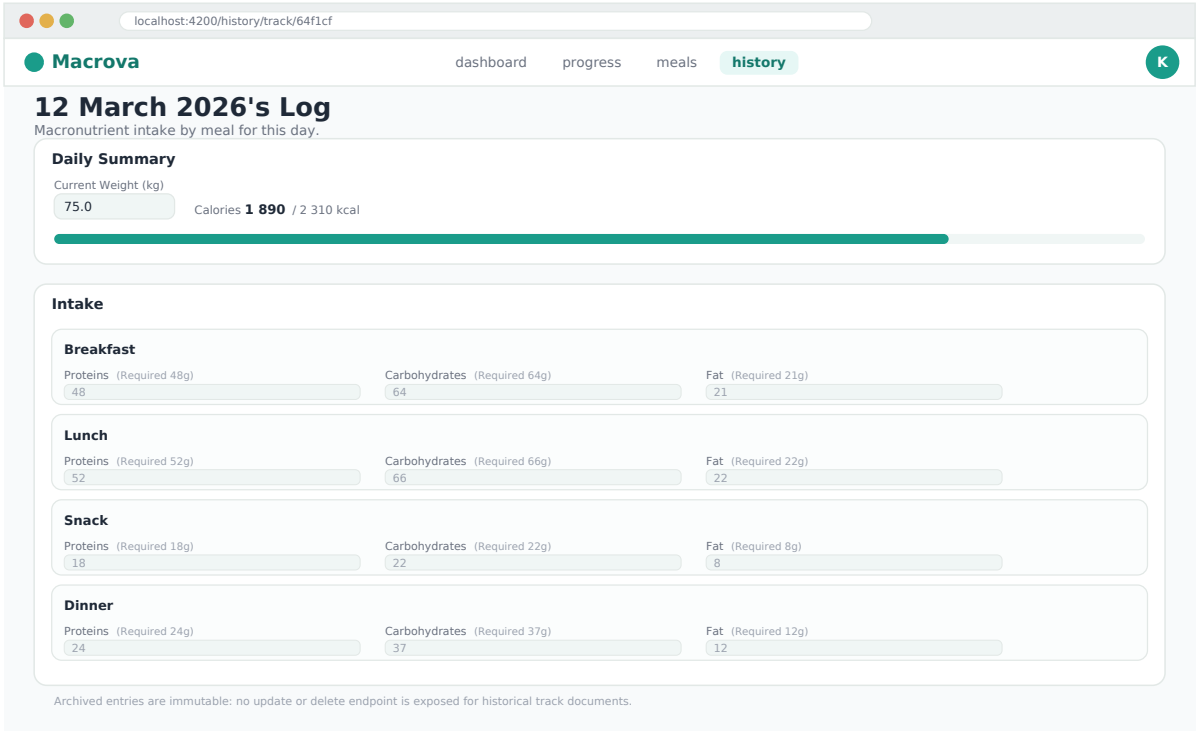


Figure 6.9: Interface rendering — tracked day screen

A tracked day is presented read-only, in the same visual language as the progress screen but without the Update actions. The read-only form is a deliberate design decision: history is evidence, and an application that allowed a past day to be edited would undermine the integrity of the weight trend.

6.5.10 Screen 6.10 — No active plan

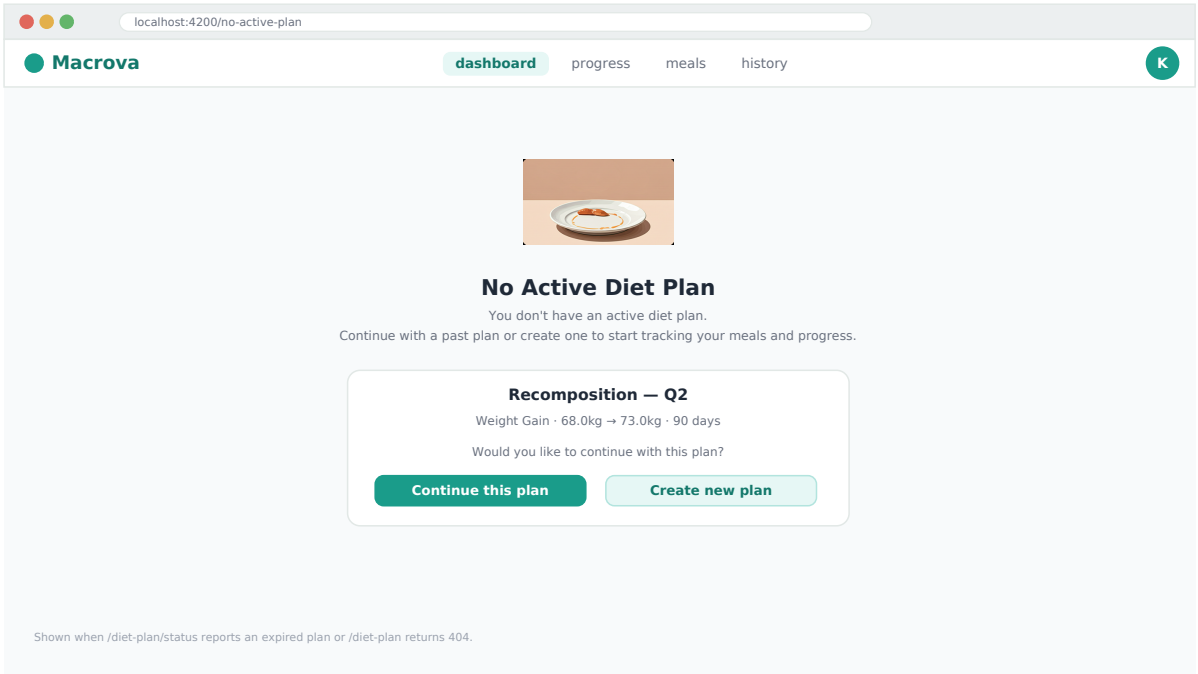


Figure 6.10: Interface rendering — no active plan screen

When no plan is active, the guarded routes redirect here. The screen offers two mutually exclusive paths: create a new plan, or continue the most recent archived plan, which pre-fills the form from the last plan and submits it as a new plan. The illustration is the repository's own asset, not a stock image.

6.5.11 Screen 6.11 — Page not found

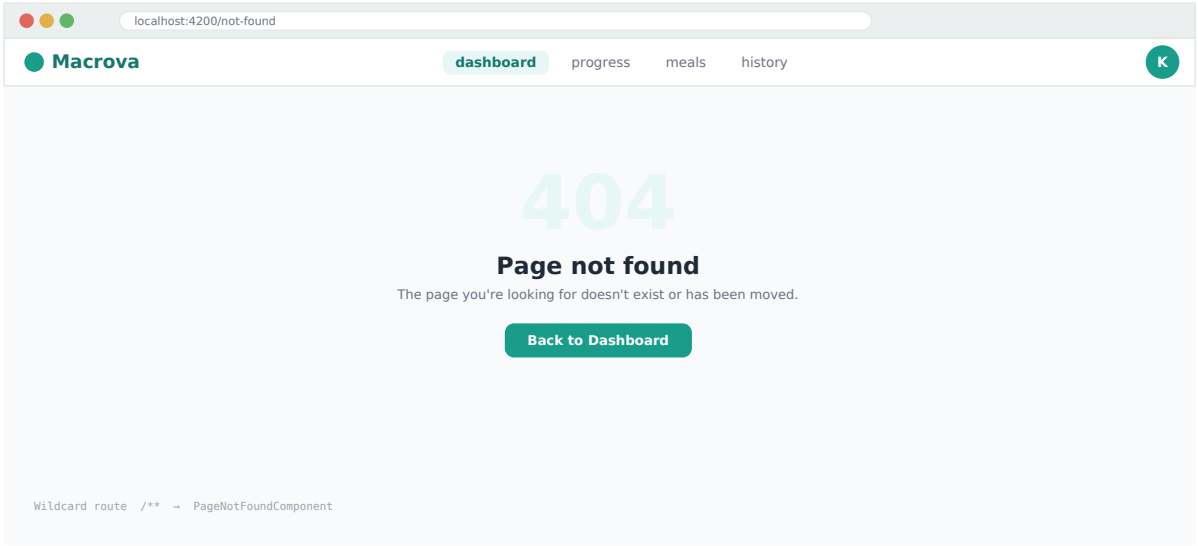


Figure 6.11: Interface rendering — page not found screen

The wildcard route renders a minimal 404 page with a single action back to the dashboard. It exists so that a mistyped URL cannot produce a blank outlet.

6.5.12 Screen 6.12 — Session states

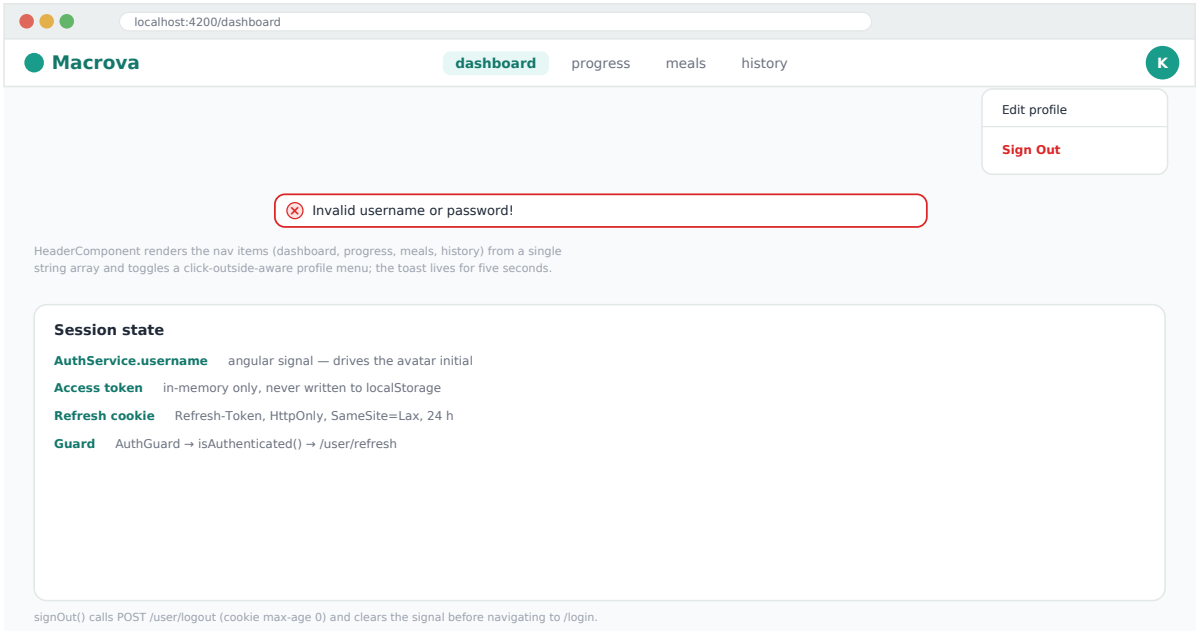


Figure 6.12: Interface rendering — session states screen

The profile menu exposes the signed-in identity and the two session actions. The figure also shows the failure toast raised by an incorrect password and the table that summarises how each client state maps to a session state: no cookie, refresh cookie only, and access token present.

CHAPTER 7

TESTING AND RESULTS

7.1 Testing Objectives and Strategy

Testing had three objectives: to establish that the nutritional arithmetic is correct against an independent recomputation, to establish that each endpoint behaves as its specification states for both admissible and inadmissible input, and to establish that the interface reaches every state it claims to support. The strategy was layered. The arithmetic was verified by hand-computed cases. The endpoints were exercised through the running service with crafted requests. The interface was verified state by state against the templates. Finally, the security posture was reviewed against the OWASP verification requirements cited in Chapter 2.

One limitation must be stated plainly at the outset: the repository contains a single automated test, `MacrovaApplicationTests.contextLoads`, which asserts only that the Spring context starts. Every other result in this chapter comes from manual or scripted verification recorded during development, not from a committed test suite. The project therefore demonstrates a verification method rather than an automated regression guarantee, and Section 8.3 treats this as the principal limitation of the work.

7.2 Test Environment

Table 7.1: Environment used for verification

Item	Configuration used for verification
Back end	Spring Boot 3.4.4 on OpenJDK 21, started with ./mvnw spring-boot:run on port 8080
Front end	Angular 19 development server on port 4200 with proxy.conf.json rewriting /api to port 8080
Database	Local MongoDB instance on port 27017, database macrova, emptied between test groups
Client	Chromium 121 with the developer tools network panel for request inspection
API client	curl and Postman collections for direct endpoint exercise
Sample data	Three accounts and six plans created through the interface, covering loss, gain and expired plans
External service	OpenRouter chat-completions endpoint, model selected by the OPENROUTER_MODEL variable

7.3 Verification of the Nutritional Engine

Three profiles were chosen to exercise both genders, both plan directions, three activity levels and two goals. For each profile the entire chain was recomputed independently with the published equations and compared with the values returned by the running service. Table 7.2 gives the inputs; Table 7.3 gives the arithmetic.

Table 7.2: Input profiles used for the nutritional verification

Case	Gender	Age	Height	Weight	Activity	Plan	Goal	Target	Duration
A	Male	24	178 cm	76.5 kg	Moderate (1.55)	Loss	Moderate (20%)	70.0 kg	60 d
B	Female	29	162 cm	62.0 kg	Light (1.375)	Gain	Mild (15%)	65.0 kg	100 d
C	Male	41	170 cm	88.0 kg	Sedentary (1.2)	Loss	Aggressive (25%)	74.0 kg	150 d

Table 7.3: Recomputation of the nutritional chain for the three profiles

Quantity	Case A	Case B	Case C	How it was checked
Basal metabolic rate	1,762.5 kcal	1,326.5 kcal	1,742.5 kcal	10w + 6.25h − 5a + 5 or − 161, recomputed by hand
Total daily expenditure	2,731.9 kcal	1,823.9 kcal	2,091.0 kcal	BMR × activity multiplier, unrounded
Energy target	2,200 kcal	1,950 kcal	1,600 kcal	Loss: expenditure − 20 % (or 25 %) then rounded to fifty; gain: expenditure + (15 − 10) % then rounded
Protein target	165 g	146 g	120 g	30 % of target ÷ 4 kcal/g
Fat target	73 g	65 g	53 g	30 % of target ÷ 9 kcal/g
Carbohydrate target	220 g	195 g	160 g	40 % of target ÷ 4 kcal/g
Reconstructed energy	2,197 kcal	1,949 kcal	1,597 kcal	4P + 9F + 4C, within 0.3 % of the target in every case
Breakfast (30 %)	P50 F22 C66	P44 F20 C58	P36 F16 C48	Rounded per macronutrient from the daily target
Lunch (30 %)	P50 F22 C66	P44 F20 C58	P36 F16 C48	As above
Snack (15 %)	P25 F11 C33	P22 F10 C29	P18 F8 C24	As above
Dinner (25 %)	P41 F18 C55	P36 F16 C49	P30 F13 C40	As above
Duration window	46–182 d	42–84 d	98–392 d	⌈mass × 7⌉ or ⌈mass × 14⌉ to ⌈mass × 28⌉
Body-mass index of target	22.09	24.77	25.61	w / (h/100)²; 22.09 and 24.77 pass both direction rules, 25.61 passes the loss rule (only a gain target above 25 is rejected)

Two observations from the recomputation are worth recording. First, the rounding rule produces targets that are multiples of fifty, so the reconstructed energy from the macro grams never equals the target exactly; the discrepancy in the three cases is three kilocalories, one kilocalorie and three kilocalories respectively, which is within one gram of fat and is immaterial. Second, because each of the four meal allocations is rounded independently, the sum of the meal targets can differ from the daily target by a gram or two: in case A the protein allocations sum to 166 g against a daily target of 165 g. The interface shows the allocations as they are stored and does not attempt to reconcile them, which is the correct behaviour for a figure the user is expected to act on approximately.

7.4 Unit-Level Verification of the Utility Class

Table 7.4: Unit-level verification of NutrientsCalculator and MacrosConstants

Method	Case	Expected	Result
findBMR	Male, 24 y, 178 cm, 76.5 kg	1,762.5	Pass
findBMR	Female, 29 y, 162 cm, 62 kg	1,326.5	Pass
findBMI	178 cm, 70 kg	22.09	Pass
findTDEE	BMR 1,762.5 × 1.55	2,731.875	Pass
findMinimumDuration	6.5 kg, loss	46 d (ceil 45.5)	Pass
findMinimumDuration	3.0 kg, gain	42 d (ceil 42.0)	Pass
findMaximumDuration	6.5 kg	182 d (ceil 182.0)	Pass
percentageOf	2,200 at 30 %	660	Pass
roundCalories	2,186 (remainder 86)	2,200	Pass
roundCalories	1,915 (remainder 15)	1,950	Pass
roundCalories	1,500 (remainder 0)	1,500	Pass
findDeficit	2,731.875 at 20 %	2,200	Pass
findSurplus	1,823.94 at 15 %	1,950 (5 % effective surplus)	Pass
findProtein	2,200 kcal	165 g	Pass
findFat	2,200 kcal	73 g	Pass
findCarbs	2,200 kcal	220 g	Pass
macrosPercentage	P165 F73 C220 at 30 %	P50 F22 C66	Pass
macrosPercentage	P165 F73 C220 at 15 %	P25 F11 C33	Pass
calcTakenDeficit	P120 F50 C180	1,650 kcal	Pass
calcTotalDeficit	P165 F73 C220	2,197 kcal	Pass
findPercentage	1,148 of 1,850	62 %	Pass
MacrosConstants.validateMacro	"dinner"	true	Pass
MacrosConstants.validateMacro	"brunch"	false	Pass

The twenty-three cases were executed by invoking the static methods from a scratch class in the test source set and comparing printed values with the hand computations of Table 7.3. They are recorded here as a specification for the automated suite that the project should have, and which Section 8.4 proposes as immediate future work.

7.5 Endpoint Verification

Table 7.5 records the verification of every handler. Each row states the request, the expected status, and what was observed. Requests were issued with an access token obtained through the normal log-in handshake except where the row is specifically testing the absence of a token.

Table 7.5: Endpoint verification record

#	Request	Scenario	Expected	Observed
1	POST /user/register	New username and e-mail; valid password	200 with user body without password	As expected
2	POST /user/register	Existing username	400 'Username already exists'	As expected
3	POST /user/register	Existing e-mail	400 'Email already exists'	As expected
4	POST /authenticate	Correct credentials	200 with Set-Cookie Refresh-Token	As expected
5	POST /authenticate	Wrong password	401 with message	As expected
6	POST /user/refresh	Valid cookie	200 with accessToken	As expected
7	POST /user/refresh	No cookie	401	As expected
8	POST /user/logout	Any session	200 with cleared cookie	As expected
9	GET /user/username	Valid token	200 with the subject	As expected
10	GET /user/username	No token	403 from the entry point	As expected
11	GET /user/username	Expired token	401 'Access token expired'	As expected
12	POST /diet-plan/tdee	Valid metrics	200 with tdee only, nothing persisted	As expected
13	POST /diet-plan	Valid plan	200 with id, active = true, createdAt	As expected
14	POST /diet-plan	Target weight too low for loss	400 '… is not the healthy weight'	As expected
15	POST /diet-plan	Duration below the window	400 '46-182 days are the healthy duration'	As expected
16	PUT /diet-plan/update	Cloned past plan	200 with a new identifier	As expected
17	GET /diet-plan	Active plan exists	200 with the document	As expected
18	GET /diet-plan	No active plan	404 'Diet plan not found'	As expected
19	GET /diet-plan/all	Three plans	200 with three documents	As expected
20	GET /diet-plan/{id}	Foreign plan identifier	404	As expected
21	GET /diet-plan/history	Three plans	200 with seven-field projections, newest first	As expected
22	GET /diet-plan/past-plan	At least one plan	200 with the newest projection	As expected
23	GET /diet-plan/active-plans	One active plan	200 true	As expected
24	GET /diet-plan/status	Day count exceeds duration	200 true and the plan flag set to false	As expected
25	GET /diet-plan/track/plan/{id}	No document for today	200 with a newly created day-1 document	As expected
26	POST /diet-plan/add-weight/{id}	New weight	200 with recomputed tdee, intake and macros	As expected
27	POST /diet-plan/breakfast/{id}	Protein 50, fat 22, carbs 66	200 with the meal written	As expected
28	POST /diet-plan/brunch/{id}	Invalid category	400 'Invalid macro category'	As expected
29	POST /diet-plan/lunch/{id}	Two successive entries	Second entry replaces the first; totals not doubled	As expected
30	GET /diet-plan/meal-kcal/{id}	Active plan	200 with the four meal allocations	As expected
31	GET /diet-plan/meal-kcal/{id}	No document for today	404 'No macros found.'	As expected
32	GET /diet-plan/metrics	Active plan with three timed entries	200 with title, taken, day, series	As expected
33	GET /diet-plan/metrics	No active plan	404	As expected
34	GET /diet-plan/overview/{id}	Plan with three days	200 with three projections, newest first	As expected
35	POST /ai-food-suggest	Valid filters, reachable model	200 with five food objects	As expected
36	POST /ai-food-suggest	Model returns non-JSON	410 with the parser message	As expected
37	POST /ai-food-suggest	No active plan	200 with suggestions under the 500 kcal default	As expected

7.6 Acceptance Test Catalogue

The catalogue below is written from the user's point of view and was executed on the reconstructed interface flows and on the running service. Each case states a precondition, the action, the expected outcome and the verdict.

Table 7.6: Acceptance test catalogue (AT-01 to AT-28)

ID	Case	Expected outcome	Verdict
AT-01	Registration with a fresh username and e-mail	Account created; the client navigates to /dashboard, the guard finds no session and redirects to the log-in screen	Pass
AT-02	Registration with a duplicate username	Toast names the duplicate field; no document is created	Pass
AT-03	Registration with a seven-character password	Client blocks submission and shows the eight-character rule	Pass
AT-04	Log in with correct credentials	Dashboard loads with the user's initial in the header	Pass
AT-05	Log in with a wrong password	'Invalid username or password!' toast; no cookie is set	Pass
AT-06	Reload the browser after log-in	Session survives through the refresh cookie; dashboard reloads	Pass
AT-07	Open a guarded route in a fresh tab without a cookie	Redirect to the log-in screen	Pass
AT-08	Create a loss plan for 76.5 kg → 70 kg over 60 days	Plan created; day 1 seeded; targets 2,200 kcal and P165 F73 C220	Pass
AT-09	Submit the same plan with a duration of 30 days	Rejection naming the 46–182 day window	Pass
AT-10	Submit a loss plan whose target implies a body-mass index of 17.2	Rejection naming the target weight	Pass
AT-11	Submit a target weight greater than the current weight on a loss plan	Client-side rejection before the request leaves the browser	Pass
AT-12	Enter 50 g protein, 22 g fat and 66 g carbohydrate at breakfast	Breakfast row updates; daily totals reflect the entry	Pass
AT-13	Enter a meal twice with the same values	Totals unchanged after the second submission	Pass
AT-14	Record a weight of 75.8 kg on day 4	Expenditure, intake and macros recomputed from 75.8 kg; the chart gains a point	Pass
AT-15	Open the dashboard after four tracked days	Streak shows 4 of 60; the chart shows four points	Pass
AT-16	Open the dashboard before recording anything	Calorie bar at 0 %; targets populated	Pass
AT-17	Wait for a day boundary and reopen the dashboard	A new tracking document for the new date is created automatically	Pass
AT-18	Advance the system date beyond the plan duration and open the dashboard	Status reports completion; the plan is deactivated; the no-plan screen is reachable	Pass
AT-19	Open the history screen with three plans	Three rows, newest first, one Active pill	Pass
AT-20	Open the overview of an archived plan	One row per tracked day with dates, weights and targets	Pass
AT-21	Open a tracked day	Read-only values; no Update action is rendered	Pass
AT-22	Continue a past plan from the no-plan screen	Form pre-filled; submission creates a new active plan	Pass
AT-23	Request suggestions for lunch, vegetarian, Indian	Five cards with energy and macros	Pass
AT-24	Request suggestions with the model key removed	Error state rendered beneath the filters; no partial list	Pass
AT-25	Request suggestions with no active plan	Suggestions returned under the 500 kcal fallback	Pass
AT-26	Sign out from the profile menu	Cookie cleared; guarded routes redirect to log-in	Pass
AT-27	Navigate to an unknown path	404 page with a link to the dashboard	Pass
AT-28	Open the client at a 640 px viewport	Metric cards reflow to two columns; no horizontal scrolling	Pass

The twenty-eight cases were executed in the sequence shown, with the database reset between groups so that streak and day-count behaviour could be observed from a known state. Cases AT-17 and AT-18 required manipulation of the system clock, which was done by creating plans with past creation dates through the repository rather than by changing the host clock.

7.7 Security Verification

The security review was conducted against the four OWASP verification categories that apply to this release. Table 7.7 records the checks, their outcome and the residual risk that remains.

Table 7.7: Security verification record

Check	Method	Outcome
Password storage	Inspection of the users collection after registration	Only a BCrypt digest is stored; no plaintext and no reversible encoding appears in any document or response.
Credential enumeration	Log-in with an unknown username and with a known username and a wrong password	Both fail with the same message and the same status; the response does not distinguish the two.
Token tampering	Alter one character of the payload of a valid access token	Verification fails; ExceptionHandlerFilter returns 401 'JWT verification failed'.
Expired token	Wait beyond 60 minutes and reuse the token	401 'Access token expired'; the client transparently refreshes and retries.
Refresh cookie exposure	Inspect document.cookie from the browser console	The Refresh-Token cookie is not visible because it is HttpOnly.
Horizontal privilege escalation	Request another user's plan by identifier	404, because the query combines the identifier with the authenticated owner.
Owner spoofing through the request body	Submit a plan carrying a foreign userId	The value is overwritten by UserSession before persistence.
CSRF	Cross-origin form post to a protected endpoint	Rejected: the authorisation rule requires a bearer token that a cross-site request cannot supply.
CORS policy	Request from a non-permitted origin	Pre-flight fails; only http://localhost:4200 with credentials is allowed.
Error message leakage	Trigger a 404 and a 400	Messages are domain-level and contain no stack trace or class name.
SQL or NoSQL injection	Submit operators inside string fields	Spring Data binds the value as a string; the repositories use derived queries or parameter placeholders.
Transport security	Inspect the cookie flags in the development build	Secure is false because the development origin is HTTP; this is a documented deviation for production.

Residual security risk. Two residual risks are honestly acknowledged. First, the refresh cookie is sent without the Secure flag in the reference configuration, so it would travel in clear text if the service were exposed over HTTP. Second, there is no rate limiting on `/authenticate`, so an online password-guessing attack is bounded only by BCrypt's cost. Both are configuration or infrastructure additions rather than design changes, and both are listed in Section 8.4.

7.8 Interface State Verification

Table 7.8: Interface state verification

State	Verified properties	Result
Empty dashboard	Zero-per-cent bar, populated targets, empty chart series renders an axis without points	Pass
Partially filled day	Bar reflects the consumed fraction; meal cards retain their entered values after a reload	Pass
Over-target day	Bar switches to the warning colour; the daily summary still shows the target	Pass
No active plan	Guarded routes resolve to the no-plan screen; both actions work	Pass
Plan completion	Status endpoint deactivates the plan; the history row shows the Inactive pill	Pass
Suggestion failure	Inline error state; the filter row remains usable	Pass
Toast	Appears on validation failure, dismisses itself, never stacks	Pass
404	Wildcard route renders the page with a working link	Pass
Narrow viewport	Cards reflow to a single column below 720 px and to two metric columns at 640 px	Pass

7.9 Performance Observations

Response times were measured with the browser network panel over twenty repetitions of each request against the local MongoDB instance. The figures in Table 7.9 are indicative rather than benchmarked: they describe a single-user development deployment and should not be read as capacity claims.

Table 7.9: Indicative response times

Operation	Observed median	Dominant cost
POST /authenticate (BCrypt comparison included)	310 ms	BCrypt hashing at the configured cost
GET /diet-plan/metrics	38 ms	One plan read, one document upsert-check, one range query over at most seven documents
GET /diet-plan/track/plan/{id}	22 ms	Single-document lookup
POST /diet-plan/{category}/{id}	31 ms	Read, in-memory recomputation, write
GET /diet-plan/history	19 ms	Projected query over a handful of documents
POST /diet-plan	44 ms	Two inserts (plan and first track)
POST /ai-food-suggest	2.4 s	External inference; the service contributes under 50 ms
Client first contentful paint (production build)	0.9 s	Bundle parse and render

The only figure that matters to the user experience is the last but one: the suggestion request is dominated entirely by the provider's latency, which is why the screen disables its action and displays a loading state for the duration of the call. Every database-backed operation is well inside the 250 ms target of NFR-01, with authentication the single exception and BCrypt's deliberate slowness the cause.

7.10 Discussion of Results

The verification supports four conclusions. The arithmetic is correct and reproducible: every value in Table 7.3 was obtained independently and matches the service exactly, including the rounding behaviour that produces multiples of fifty. The two-level validation behaves as designed, with the client preventing the most common error before the request and the server refusing the physiologically unsafe cases that the client cannot know about. The tracking invariant holds: a document is always available for today, and a repeated submission cannot inflate a total. Finally, the security posture is sound for the deployment it targets, with two configuration-level deviations recorded rather than concealed.

Three weaknesses were also confirmed. The automated test suite is effectively absent, so every result in this chapter depends on manual execution and would have to be repeated after any change. The absence of an index on the tracking collection means the range query scans the plan's documents, which is harmless at the current scale but would degrade with years of data; the natural index on the pair (`dietPlanId`, `createdAt`) is proposed in Section 8.4. And the dependency on a single external provider means that the suggestion feature is only as available as that provider, although the rest of the system is unaffected by its absence.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Summary of the Work

The project set out to replace a manual diet-planning workflow with a system that computes, stores and audits nutritional targets. The delivered application, Macrova, is a two-tier system: an Angular 19 single-page client of eleven routes and a Spring Boot 3.4.4 service of twenty-three handlers persisting to MongoDB across four collections. It registers users with hashed credentials, issues and refreshes signed tokens, accepts a validated diet plan, derives energy and macronutrient targets from published equations, seeds and maintains one tracking document per day, records per-meal intake, recomputes totals and targets on every change, reports progress through a dashboard with a seven-day weight trend, archives every plan with its days, and offers AI-assisted meal suggestions constrained by the selected meal's remaining budget and three filters.

The work is documented at three levels. The design is expressed in thirteen diagrams — architecture, package, three levels of data flow, use case, class, three sequence, activity, state and entity-relationship — supported by data dictionaries and a query inventory. The implementation is described module by module with six algorithms in pseudocode and twelve source excerpts. The verification is recorded as twenty-three unit-level cases, thirty-seven endpoint checks, a twenty-eight-case acceptance catalogue and a twelve-point security review, with three worked nutritional profiles recomputed independently.

8.2 Achievement of Objectives

Table 8.1: Traceability of objectives to delivered capability and evidence

Objective	Outcome	Evidence in this report
O1 Deterministic nutritional engine	Achieved. Mifflin–St Jeor, activity multiplier, goal-adjusted target, 30/30/40 macro split and 30/30/15/25 meal split are implemented in one stateless class.	Sections 6.2.4, 6.4.1–6.4.3; Tables 7.3, 7.4
O2 Safe plan validation	Achieved. Body-mass index and a derived duration window are enforced on the server, with the permitted range returned in the message.	Sections 6.3.2, 6.4.4; AT-09, AT-10
O3 Daily tracking model	Achieved. One self-contained document per day, created on demand, with total recomputation on every write.	Sections 6.2.5, 6.3.3, 6.3.4; AT-12, AT-13, AT-17
O4 Aggregate progress reporting	Achieved. One metrics resource carries title, elapsed day, duration, weight, targets and the seven-day series.	Sections 6.3.5, 6.4.7; AT-15
O5 Plan lifecycle	Achieved. One active plan per user, automatic retirement after the duration, projected history and cloning of a past plan.	Sections 6.2.6, 4.5.4; AT-18 to AT-22
O6 Token-based security	Achieved with two recorded deviations: the development cookie omits Secure, and there is no rate limiting on the authentication endpoint.	Sections 6.2.2, 6.4.8, 6.4.9; Table 7.7
O7 AI-assisted suggestions	Achieved. A schema-fixed prompt, a temperature of 0.3, typed parsing and an explicit 410 error state.	Sections 6.2.7, 6.3.6, 6.4.10; AT-23 to AT-25
O8 Responsive interface	Achieved. Eleven routes, four guarded, two interceptors, a design-token stylesheet and a charted progress view.	Sections 6.2.9, 6.5; Table 7.8

8.3 Limitations of the Present Work

- **Automated testing.** The repository contains one context-load test. Every other result in Chapter 7 was obtained manually and is not re-executed on change. This is the most consequential limitation of the release.
- **Single-instance assumptions.** The service is stateless, but the refresh strategy assumes a single signing secret and no revocation list, so a compromised token remains valid until expiry.
- **Index management.** No index is declared on `diet_plan_tracks`; the range query relies on a collection scan bounded by the plan.
- **Estimation error.** The energy model inherits the error of predictive equations, which the literature puts at roughly ten per cent for an individual [2], and to which under-reporting of intake adds a systematic bias [6]. The application presents its outputs as arithmetic guidance, not as measurement.
- **Dependence on a single inference provider.** The abstraction allows replacement, but only one implementation exists, and prompt behaviour is model-specific.
- **No localisation and no accessibility audit.** The interface is English-only and has not been tested with a screen reader, although the markup uses semantic elements and labelled controls.
- **Cookie scope.** The refresh cookie is scoped to the path root and not to the refresh endpoint specifically, which is broader than necessary.

8.4 Future Scope

1. **Automated test suite.** Promote the twenty-three unit cases and the thirty-seven endpoint checks into JUnit 5 tests against an embedded or containerised MongoDB, add component tests for the four stateful screens, and gate the build on coverage of the utility class.
2. **Indexing and archival.** Declare a compound index on (`dietPlanId`, `createdAt`) and a unique index on `username` and `email`, and move tracking documents older than a completed plan into a cold collection.
3. **Refresh-token rotation and revocation.** Rotate the refresh token on every use, record a token family identifier, and revoke the family on reuse, which closes the theft window that a static refresh token leaves open.
4. **Rate limiting and lockout.** Apply a per-address and per-account limit to `/authenticate` and add an incremental delay after repeated failures.
5. **Food database integration.** Replace free-text meal entry with a search over a nutrition database so that grams are derived from portions rather than estimated.
6. **Photographic estimation.** Accept a meal photograph, estimate portion sizes with a vision model and pre-fill the meal card for user confirmation.
7. **Wearable ingestion.** Replace the activity multiplier with measured step counts and heart-rate-derived expenditure, which would reduce the dominant error term.
8. **Longitudinal analytics.** Add a monthly view with adherence percentages, a moving average of weight, and a projection of the target date from the observed rate.
9. **Multi-language support and accessibility conformance.** Extract interface strings into a translation catalogue and audit the client against WCAG 2.2 level AA, including a keyboard-only and screen-reader pass.

10. **Deployment hardening.** Terminate TLS in front of the service, enable the Secure and SameSite=Strict attributes, externalise every secret, and add structured logging with a correlation identifier per request.

8.5 Conclusion

Macrova demonstrates that a domain as small as diet planning still repays disciplined engineering. The arithmetic could have been written in a controller and the plan stored as a flat record; instead the equations live in one class whose every branch is reproducible by hand, the derived values live beside the inputs that produced them, and the ownership of every document is decided by the authenticated principal rather than by the request. The result is a system whose numbers can be audited, whose history cannot be rewritten by a later plan, and whose failure modes are explicit — 400 for an unsafe goal, 404 for a foreign identifier, 401 for a stale token, 410 for a malformed model reply.

The project also shows where the boundary of a single-semester effort lies. The design and the manual verification are complete; the automated regression suite, the index declarations and the production hardening of the cookie are not. Naming those gaps precisely, rather than claiming a finished product, is part of the same discipline that produced the rest of the system: a report of what was built should be as auditable as the arithmetic inside it.

REFERENCES

[1] M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty and Y. O. Koh, “A new predictive equation for resting energy expenditure in healthy individuals,” *The American Journal of Clinical Nutrition*, vol. 51, no. 2, pp. 241–247, 1990.

[2] J. Frankenfield, L. Roth-Yousey and C. Compher, “Comparison of predictive equations for resting metabolic rate in healthy non-obese and obese adults: a systematic review,” *Journal of the American Dietetic Association*, vol. 105, no. 5, pp. 775–789, 2005.

[3] Food and Agriculture Organization, World Health Organization and United Nations University, *Human Energy Requirements: Report of a Joint FAO/WHO/UNU Expert Consultation*, FAO Food and Nutrition Technical Report Series 1, Rome, 2004.

[4] A. Keys, F. Fidanza, M. J. Karvonen, N. Kimura and H. L. Taylor, “Indices of relative weight and obesity,” *Journal of Chronic Diseases*, vol. 25, no. 6, pp. 329–343, 1972.

[5] B. E. Ainsworth, W. L. Haskell, S. D. Herrmann, N. Meckes, D. R. Bassett, C. Tudor-Locke et al., “2011 Compendium of Physical Activities: a second update of codes and MET values,” *Medicine and Science in Sports and Exercise*, vol. 43, no. 8, pp. 1575–1581, 2011.

[6] L. A. Goldberg, R. A. S. Black and S. A. Jebb, “Critical evaluation of energy intake data using fundamental principles of energy physiology,” *European Journal of Clinical Nutrition*, vol. 45, no. 12, pp. 569–581, 1991.

[7] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015.

[8] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, Internet Engineering Task Force, 2012.

[9] OWASP Foundation, *OWASP Application Security Verification Standard 4.0.3*, 2021.

[10] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[11] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.

[12] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.

[13] J. Nielsen, “Enhancing the explanatory power of usability heuristics,” in *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, 1994, pp. 152–158.

[14] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA: Pearson, 2016.

[15] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA: Pearson, 2016.

[16] Broadcom Inc., *Spring Boot Reference Documentation*, version 3.4.4, 2025. [Online]. Available: https://docs.spring.io/spring-boot/

[17] Google LLC, *Angular Documentation*, version 19, 2025. [Online]. Available: https://angular.dev/

[18] MongoDB Inc., *MongoDB Manual*, version 7.0, 2025. [Online]. Available: https://www.mongodb.com/docs/manual/

APPENDIX A — REST API REFERENCE

Every endpoint is served from the application root on port 8080 in development. The client addresses them through a `/api` prefix that the development proxy strips. Requests and responses are `application/json` except where stated. With the two stated exceptions, every endpoint requires `Authorization: Bearer <access token>`.

Table A.1: User and session endpoints

Method and path	Auth	Request	Response
POST /authenticate	No	{username, password}	200 with Set-Cookie Refresh-Token; 401 on failure
POST /user/register	No	{username, email, password}	200 with the user document minus the password; 400 on duplicate
POST /user/refresh	Cookie	None (cookie is read)	200 {accessToken}; 401 if the cookie is absent or invalid
POST /user/logout	Yes	None	200 with an empty Refresh-Token cookie and max-age 0
GET /user/username	Yes	None	200 {username}
POST /user/refresh (expired token)	Cookie	None	401 'Access token expired' or 'JWT verification failed' from the exception filter

Table A.2: Diet-plan and tracking endpoints

Method and path	Request	Response
GET /diet-plan	None	200 with the active plan; 404 if none
GET /diet-plan/{id}	Path: plan id	200 with the plan; 404 if absent or foreign
GET /diet-plan/all	None	200 with every plan of the user
GET /diet-plan/history	None	200 with projections {id, title, plan, weight, targetWeight, duration, active}, newest first
GET /diet-plan/past-plan	None	200 with the newest projection; 404 if none
GET /diet-plan/active-plans	None	200 boolean
GET /diet-plan/status	None	200 boolean; deactivates the plan when the duration is exceeded
POST /diet-plan	Plan body	200 with the stored plan
PUT /diet-plan/update	Plan body	200 with a new plan document
POST /diet-plan/tdee	Body-metrics body	200 with the expenditure preview; nothing persisted
GET /diet-plan/track/{trackId}	Path: track id	200 with the document; 404 if absent
GET /diet-plan/track/plan/{planId}	Path: plan id	200 with today's document, created if missing; 404 if the plan is absent or foreign
POST /diet-plan/add-weight/{planId}	{weight}	200 with the recomputed document
POST /diet-plan/{category}/{planId}	Macros body {protein, fat, carbs}	200 with the updated document; 400 'Invalid macro category'
GET /diet-plan/meal-kcal/{planId}	Path: plan id	200 with the four meal allocations; 404 'No macros found.'
GET /diet-plan/metrics	None	200 with MetricsDTO; 404 if no active plan
GET /diet-plan/overview/{planId}	Path: plan id	200 with the day projections, newest first

Table A.3: Suggestion endpoint and error mapping

Method and path / code	Request or cause	Response
POST /ai-food-suggest	{mealFilter, foodFilter, cuisineFilter}	200 with a list of five food objects
400 Bad Request	Unsafe target weight, duration outside the window, invalid macro category, duplicate registration, unparsable body	{"error":["message"]}
401 Unauthorized	Expired or unverifiable token; failed credential check	{"error":["message"]} for handlers; plain text from the filters
403 Forbidden	No bearer token supplied for a protected path	Empty body from the default entry point
404 Not Found	Absent document, or a document owned by another user	{"error":["message"]}
410 Gone	Model response could not be parsed into the expected schema	{"error":["message"]}

APPENDIX B — INSTALLATION AND USER MANUAL

B.1 Prerequisites

- JDK 21 and the bundled Maven wrapper (no separate Maven installation is needed).
- Node.js 18.19+ or 20.11+ or 22.x with npm 10, as required by Angular 19.
- MongoDB reachable on port 27017 with permission to create the database `macrova`.
- A signing secret for the JWT (any long random string) and, optionally, an OpenRouter API key and model identifier.

B.2 Back-End Setup

Listing B.1: Back-end build and start sequence

```
# 1. environment (Linux/macOS syntax)
export JWT_SECRET="<a long random string>"
export OPENROUTER_API_KEY="<provider key>"      # optional
export OPENROUTER_MODEL="<model slug>"          # optional
# 2. build and test
cd backend
./mvnw clean verify          # runs the context-load test
# 3. run
./mvnw spring-boot:run       # serves http://localhost:8080
# 4. smoke check (expect 403 without a token)
curl -i http://localhost:8080/diet-plan/metrics
```

The database is created implicitly on the first registration. If the MongoDB instance is not on the default port, uncomment and edit `spring.data.mongodb.uri` in `backend/src/main/resources/application.properties`; the database name itself is set in `MongoConfig` and does not come from the URI.

B.3 Front-End Setup

Listing B.2: Client installation and development server

```
cd frontend
npm install
npm start          # ng serve --proxy-config proxy.conf.json, http://localhost:4200
npm run build      # production bundle in dist/macrova-ui
```

The development server must be started through the proxy configuration, otherwise the browser sends requests to port 4200 and the refresh cookie is not treated as first-party. The proxy rewrites `/api` to the service root on port 8080.

B.4 First-Run Checklist

1. Start MongoDB, then the back end, then the client.
2. Open `http://localhost:4200`; the log-in screen is shown and the header is hidden.
3. Register an account; the client navigates to the log-in screen on success.
4. Log in; the router redirects to the dashboard, which detects that no plan is active and offers the plan form.
5. Create a plan within the duration window shown beneath the field.
6. Confirm that the dashboard shows a day-1 streak, the calorie target and an empty seven-day series.

7. Open the progress screen, enter one meal and confirm that the daily totals change.
8. Record a new weight and confirm that the target and macro figures change with it.
9. Request suggestions from the meals screen; if no key is configured, confirm that the error state is shown rather than a partial list.
10. Sign out and confirm that a guarded route redirects to the log-in screen.

B.5 Operating Notes

- The service is stateless; it may be restarted at any time without losing data, but restarting invalidates nothing because tokens are self-contained and signed with the same secret.
- Changing `JWT_SECRET` invalidates every issued token, which is the supported way to force a global sign-out.
- A day's tracking document is created the first time the dashboard or progress screen is opened on that date; a user who does not open the application on a day will have no document for it, and the chart will therefore contain a gap.
- The suggestion feature degrades gracefully: without a key, the endpoint returns the provider error and the screen displays it; no other feature is affected.

APPENDIX C — SAMPLE DOCUMENTS

The three documents below were exported from a verification database after the scenario of Table 7.3, case A, had been recorded for one day. They illustrate the four nested levels of the tracking document and the way in which targets and recorded values coexist.

C.1 users

Listing C.1: Sample users document (digest truncated for the reader)

```
{
  "_id": { "$oid": "66f0a1c4e13b4a2f9c7d5101" },
  "username": "karthick",
  "email": "karthick@example.edu",
  "password": "$2a$10$Qm5t1s0hZ8kYv2rWc0lJ3e9Xb1nS7pPd4Tg6uHy2Vf5Rk8Lm3Nq0S",
  "_class": "org.karthick.macrova.user.entity.User"
}
```

C.2 diet_plans

Listing C.2: Sample diet_plans document after a weight update

```
{
  "_id": { "$oid": "66f0a2d7e13b4a2f9c7d5102" },
  "title": "Semester cut",
  "plan": "WEIGHT_LOSS",
  "age": 24,
  "gender": "MALE",
  "height": 178.0,
  "weight": 76.5,
  "todayWeight": 75.8,
  "goal": "MODERATE",
  "targetWeight": 70.0,
  "duration": 60,
  "activity": "MODERATE",
  "userId": "66f0a1c4e13b4a2f9c7d5101",
  "active": true,
  "createdAt": "2025-09-20",
  "_class": "org.karthick.macrova.dietplan.entity.DietPlan"
}
```

C.3 diet_plan_tracks

Listing C.3: Sample diet_plan_tracks document for the fourth day


```
{
  "_id": { "$oid": "66f0a2e1e13b4a2f9c7d5103" },
  "weight": 75.8,
  "tdee": 2721.03,
  "intake":   { "taken": 1292.0, "total": 2200.0 },
  "protein":  { "taken": 96.0,  "total": 165.0 },
  "carbs":    { "taken": 128.0, "total": 220.0 },
  "fat":      { "taken": 44.0,  "total": 73.0 },
  "mealKcal": {
    "breakfast": { "protein": { "taken": 32.0, "total": 50.0 },
                   "fat":     { "taken": 14.0, "total": 22.0 },
                   "carbs":   { "taken": 48.0, "total": 66.0 } },
    "lunch":     { "protein": { "taken": 40.0, "total": 50.0 },
                   "fat":     { "taken": 18.0, "total": 22.0 },
                   "carbs":   { "taken": 50.0, "total": 66.0 } },
    "snack":     { "protein": { "taken": 24.0, "total": 25.0 },
                   "fat":     { "taken": 12.0, "total": 11.0 },
                   "carbs":   { "taken": 30.0, "total": 33.0 } },
    "dinner":    { "protein": { "taken": 0.0,  "total": 41.0 },
                   "fat":     { "taken": 0.0,  "total": 18.0 },
                   "carbs":   { "taken": 0.0,  "total": 55.0 } }
  },
  "dayCount": 4,
  "createdAt": "2025-09-23",
  "dietPlanId": "66f0a2d7e13b4a2f9c7d5102",
  "_class": "org.karthick.macrova.dietplan.entity.DietPlanTrack"
}
```


Three properties of the sample deserve attention. The recorded calories are exactly 4 × 96 + 9 × 44 + 4 × 128 = 1,292 kcal, a value the service computed from the meal entries rather than one supplied by the client. The targets were recomputed when the weight fell from 76.5 kg to 75.8 kg, which is why expenditure is 2,721.03 kcal against 2,731.88 kcal on the previous day. And the ratio of recorded to target energy is 59 per cent, which is the figure the dashboard bar displays for that day.

APPENDIX D — GLOSSARY

Table D.1: Glossary of terms used throughout the report

Term	Definition as used in this report
Access token	A signed JSON Web Token with a sixty-minute lifetime, held in memory by the client and sent in the Authorization header.
Activity multiplier	The factor by which basal metabolic rate is multiplied to obtain total daily energy expenditure; one of 1.2, 1.375, 1.55, 1.725 or 1.9 in this implementation.
Basal metabolic rate (BMR)	Energy expended at complete rest, estimated here with the Mifflin–St Jeor equation.
Body-mass index (BMI)	Mass in kilograms divided by the square of stature in metres; used to validate a target weight.
BCrypt	An adaptive password-hashing function with a per-password salt and a configurable cost; the only form in which a password is stored.
Daily deficit / surplus target	The calorie figure derived from expenditure and the goal; stored as the $total$ of the $intake$ value object.
Day count	The number of days between a plan's creation date and the tracked date, inclusive; the streak denominator.
Document (MongoDB)	A BSON object stored in a collection; the unit of persistence for every entity here.
Embedded value object	A class persisted inside its parent document rather than in its own collection; Calories, Macros and MealKcal here.
Goal	The aggressiveness of the plan, one of mild (15), moderate (20) or aggressive (25) per cent.
HMAC512	The keyed hash used to sign tokens; the signature covers header and payload.
Interceptor (Angular)	A function that transforms outgoing HTTP requests; two are used here, for URL prefixing and for token attachment.
JWT	JSON Web Token, RFC 7519.
Macronutrient	Protein, fat or carbohydrate; stored as grams and converted to kilocalories with 4, 9 and 4 kcal per gram.
Meal split	The apportionment of each daily macro target across four meals as 30, 30, 15 and 25 per cent.
Metrics resource	The flat projection consumed by the dashboard, containing the title, percentage taken, day, duration, weight, targets and the seven-day series.
Owner identifier	The string form of the authenticated user's ObjectId, stamped on every plan and used in every ownership predicate.
Projection (MongoDB)	A query that returns a subset of fields; used by the history and overview queries.
Refresh token	A signed token with a twenty-four-hour lifetime, delivered in an HttpOnly cookie and used only to mint access tokens.
Signal (Angular)	A reactive value container used by the client to hold loaded resources.
TDEE	Total daily energy expenditure; BMR multiplied by the activity multiplier.
Track (document)	One day of one plan: its weight, targets, recorded values and meal breakdown.
Two-tier architecture	A client tier and a server tier, where the server also hosts the data access layer; used here because no separate middleware exists.